

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕНА К ЗАЩИТЕ

Заведующая кафедрой

_____ / Е.В. Петрунина/

Руководитель образовательной
программы

_____ / А.В. Толстиков /

**«РАЗРАБОТКА МИНИ-САПР «СПИРАЛЬНЫЙ ТЕПЛООБМЕННЫЙ
АППАРАТ» С ИСПОЛЬЗОВАНИЕМ API C3D»**

Выпускная квалификационная работа

по направлению 09.03.01 Информатика и вычислительная техника

Образовательная программа (профиль) «Интеграция и программирование в
САПР»

Студент: _____ / Куратов Д.И., 221–325 /

подпись *ФИО, группа*

Руководитель ВКР: _____ / Леонтьев А.В., _____ /

подпись *ФИО, уч. звание и степень*

Москва, 2026

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль)
«Интеграция и программирование в САПР»

ТЕМА ВКР	Разработка мини-САПР «Спиральный теплообменный аппарат» с использованием API C3D
ПРАКТИЧЕСКОЙ РЕЗУЛЬТАТ	
Назначение	Программное обеспечение для построения параметрической твердотельной модели теплообменника
Основные функции	1. Тепловой расчёт теплообменного аппарата; 2. Определение геометрических параметров теплообменного аппарата; 3. Гидравлический расчет теплообменного аппарата; 4. Создание и редактирование твердотельной модели теплообменника; 5. Сохранение твердотельной модели теплообменника.
Используемые технологии и платформы	C++, Qt, C3D Toolkit, Microsoft Visual Studio

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Разработка методики теплового и гидравлического расчёта спирального теплообменного аппарата и её программная реализация; 2. Разработка параметрической 3D-модели спирального теплообменного аппарата с использованием API C3D; 3. Разработка программного приложения мини-САПР, обеспечивающего ввод исходных данных, выполнение расчёта и автоматическую генерацию модели.
Состав технической документации	— Пояснительная записка; — Техническое задание; — Исходный код программного продукта
Состав графической части	Презентация

ПЛАН РАБОТЫ НАД ВКР

ЗАДАЧИ	НЕДЕЛИ																	
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8
Обзор предметной области																		
Постановка задачи																		
Разработка алгоритма расчёта теплового и гидравлического расчёта теплообменника																		
Разработка архитектуры приложения																		
Разработка модели аппарата при помощи API C3D																		
Разработка приложения для расчёта теплообменника																		
Составление пояснительной записки																		

РУКОВОДИТЕЛЬ ВКР:

«__» _____ 2025, _____ / Леонтьев А.В. , _____ /
подпись ФИО, уч. звание и степень

СТУДЕНТ:

«__» _____ 2025, _____ / Куратов Д.И. , 221–325 /
подпись ФИО, группа

АННОТАЦИЯ

Наименование работы: Разработка мини-САПР «Спиральный теплообменный аппарат» с использованием API C3D.

Цель работы: создание програамы для расчёта и проектирования спирального теплообменного аппарата с использованием API C3D.

Выполнивший работу: Куратов Д.И., студент Московского политехнического университета, кафедра «СМАРТ-технологии».

Работа состоит из 7 глав, Введения, Заключения и Приложений. Общий объем работы 189 страницы, из них 5 приложений на 103 листах. В работу включены 14 рисунков, 3 таблицы. Библиографический список состоит из 25 источников.

Во Введении описаны цель и задачи работы, её актуальность.

Основная часть работы включает в себя проведение необходимых расчетов, программную реализацию разработанных решений. Кроме того, в данном разделе представлено технико-экономическое обоснование проекта.

В приложениях представлены полный листинг кода, техническое задание и руководство пользователя.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
Глава 1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	10
1.1 Спиральный теплообменный аппарат	10
1.2 Анализ научной и нормативной литературы	13
1.3 Анализ существующих программных решений	15
1.4 Проблематика проектирования	18
1.5 Используемые технологии	19
Глава 2 МЕТОДИКА РАСЧЁТА ТЕПЛООБМЕННОГО АППАРАТА.....	22
2.1 Определение тепловой нагрузки и материального баланса	22
2.2 Определение среднего температурного напора	22
2.3 Расчет коэффициентов теплоотдачи и теплопередачи	23
2.4 Геометрический расчет	25
2.5 Гидравлический расчёт	27
Глава 3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ РАЗРАБОТАННОГО АЛГОРИТМА	28
3.1 Архитектура программного комплекса и структуры данных	28
3.2 Реализация расчетных алгоритмов.....	29
3.3 Алгоритм выбора ширины	31
Глава 4 ПОСТРОЕНИЕ МОДЕЛИ ПРИ ПОМОЩИ API C3D	33
4.1 Моделирование спиральных каналов.....	33
4.2 Моделирование обечайки	38
4.3 Моделирование патрубков.....	43
4.4 Моделирование прокладок	45
4.5 Моделирование днища.....	48
4.6 Создание сборки.....	50
Глава 5 СТРУКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	54
5.1 Алгоритм работы системы	54
5.2 Интерфейс приложения	55
5.3 Класс AppWindow	56

5.4 Класс LicenseManager	59
5.5 Класс CalcManagerWidget.....	60
5.6 Класс CalcSceneWidget	67
Глава 6 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	72
Глава 7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ	75
7.1 Расчет трудоемкости разработки программного продукта	75
7.2 Расчёт себестоимости программного продукта	77
7.3 Расчёт экономического эффекта программного продукта	79
ЗАКЛЮЧЕНИЕ	82
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	83
Приложение А – Техническое задание.....	86
Приложение Б – Руководство пользователя	101
Приложение В.....	112
Приложение Г	118
Приложение Д.....	135

ВВЕДЕНИЕ

Теплообменное оборудование является важнейшим элементом технологических цепочек в химической, нефтехимической, пищевой и энергетической отраслях промышленности. Эффективность процессов нагрева, охлаждения и конденсации рабочих сред напрямую определяет энергоэффективность, безопасность и экономическую рентабельность всего производства. С развитием технологий к аппаратам предъявляются все более строгие требования по компактности, надежности и способности работать со сложными, вязкими или сильно загрязненными средами.

С такими задачами лучше всего справляются спиральные теплообменные аппараты. Данный тип оборудования отличается уникальной конструкцией, обеспечивающей высокую интенсивность теплопередачи при минимальных габаритах. Благодаря эффекту самоочистки и низкому гидравлическому сопротивлению, спиральные аппараты успешно справляются с задачами, где традиционные решения, например кожухотрубные, оказываются малоэффективными или быстро выходят из строя из-за засорения проходных сечений.

Конструктивной основой такого аппарата являются металлические листы, свернутые вокруг центральной трубы-керна в форме спирали Архимеда и образующие непрерывные концентрические каналы для встречного движения теплоносителей. Эта специфическая геометрия делает процесс конструирования подобных аппаратов в классических универсальных системах автоматизированного проектирования весьма трудоемким. Ручное построение сложных криволинейных поверхностей, расчет шага навивки и разверток требует от инженера-конструктора больших затрат времени и высокой квалификации, а также несет в себе риск возникновения пространственных коллизий и геометрических неточностей.

В современных условиях цифровизации промышленности и необходимости быстрого вывода изделий на рынок, автоматизация рутинных проектных задач становится критически важной. Существующие

универсальные САПР не всегда способны предложить быстрые инструменты для узкоспециализированных задач, а ручное моделирование спиралей финансово и по времени нецелесообразно. Кроме того, в связи с государственным курсом на импортозамещение программного обеспечения, особую актуальность приобретает разработка профильных инженерных инструментов на базе исключительно отечественных технологий.

Целью данной выпускной квалификационной работы является разработка специализированного программного приложения — мини-САПР, предназначенной для автоматизированного проектирования и построения трехмерных твердотельных моделей спиральных теплообменных аппаратов. Разрабатываемое приложение призвано обеспечить удобный пользовательский интерфейс для ввода исходных геометрических и технологических параметров, автоматизировать математические расчеты профиля и быстро генерировать точную 3D-модель аппарата без необходимости ручного вычерчивания каждого витка спирали.

Ключевой особенностью разрабатываемой программы является использование прикладного программного интерфейса отечественного геометрического ядра C3D (C3D Toolkit). Интеграция с этим математическим ядром позволяет на высочайшем уровне решать задачи параметрического моделирования кривых, создания листовых тел и сборочных единиц. Программная реализация интерфейса и алгоритмов выполняется с использованием современных языков объектно-ориентированного программирования, что в связке с мощными функциями C3D API обеспечивает высокую производительность генерации твердотельной геометрии и возможность её последующего экспорта.

ГЛАВА 1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Спиральный теплообменный аппарат

Теплообменный аппарат — устройство для передачи тепловой энергии между двумя рабочими средами без их взаимного смешения, обеспечивающее определённый тепловой поток при заданных термо- и гидравлических условиях. Основные требования к теплообменным аппаратам — высокая эффективность теплообмена, надёжность при эксплуатационных перепадах температур и давлений, устойчивость к коррозии и загрязнению теплообменных поверхностей.

Спиральные теплообменные аппараты отличаются от других типов конструкций (кожухотрубных, пластинчатых) благодаря особенностям геометрии и организации потоков. Такие аппараты изготавливаются из двух спирально изогнутых листов, между которыми формируется канал, по которому движутся рабочие среды. Пространства между листами закрываются по торцам крышками для обеспечения герметичности (Рисунок 1).

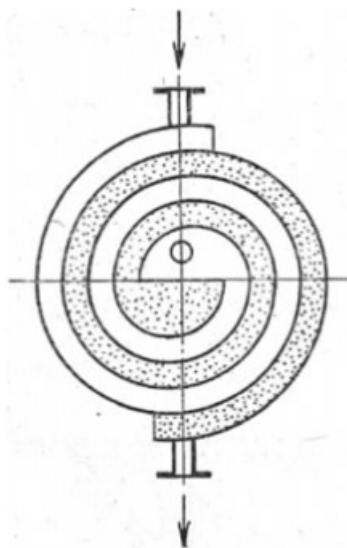


Рисунок 1 – Принципиальная схема спирального теплообменника

Спиральные теплообменники широко применяются в нефтехимической, химической, пищевой и энергетической промышленности. Их преимущества включают способность работать с вязкими, абразивными и загрязнёнными

потоками, а также высокую эффективность использования площади теплообмена при сравнительно небольших габаритах.

Рабочий процесс в спиральном теплообменнике базируется на непрерывном движении теплоносителей по криволинейным каналам. Как правило, аппарат подключается таким образом, чтобы обеспечить схему чистого противотока: один теплоноситель подается в центральный коллектор и движется по спирали к периферии, а второй поступает в периферийный канал и движется к центру. Такая схема обеспечивает максимальный температурный напор на всем протяжении теплообменной поверхности.

Одной из ключевых особенностей спиральных теплообменников является специфика их гидродинамики. При движении жидкости по криволинейному каналу возникают вторичные циркуляционные токи, которые интенсифицируют турбулентность потока даже при относительно низких скоростях (начиная с чисел Рейнольдса $Re \approx 1000$). Это приводит к значительному увеличению коэффициента теплоотдачи по сравнению с прямыми трубами кожухотрубных аппаратов.

Кроме того, спиральная геометрия одного канала обеспечивает так называемый «эффект самоочистки». Если на определенном участке канала начинает скапливаться осадок, живое сечение канала сужается. В результате локальная скорость потока в этом месте резко возрастает, что приводит к смыванию отложений с поверхности стенки. Это свойство делает спиральные аппараты незаменимыми при работе со шламами, суспензиями и загрязненными сточными водами.

Несмотря на ряд существенных достоинств, эксплуатация и производство спиральных теплообменников сопряжены с определенными недостатками и конструктивными ограничениями. В первую очередь, к ним относится ограничение по максимальному рабочему давлению (как правило, не превышающему 1,0–2,5 МПа), что связано со сложностью обеспечения прочности плоских торцевых крышек большого диаметра и обширных спиральных сварных швов. Кроме того, такие аппараты обладают крайне

низкой ремонтпригодностью: при возникновении разгерметизации или разрушении шва в глубине внутренних витков устранить дефект традиционными методами сварки практически невозможно. Техническое обслуживание также имеет свою специфику — из-за недоступности центральных участков каналов механическая очистка сильно затруднена или невозможна, что вынуждает применять для удаления стойких отложений исключительно методы химической промывки. Кроме этого, сам технологический процесс изготовления спиральных аппаратов требует наличия узкоспециализированного оборудования, что ограничивает круг их производителей и может повышать первоначальные капитальные затраты.

Конструктивные параметры (диаметры спиралей, шаг витков, ширина каналов и др.) регламентируются нормативными документами, такими как ГОСТ 12067–80 «Спиральные теплообменные аппараты» [10], который определяет типы, параметры и основные требования к таким аппаратам в рамках российского стандарта.

Спиральный теплообменник состоит из следующих основных элементов:

- корпус – цилиндрический кожух, внутри которого размещаются спиральные каналы;
- спиральные каналы – образуются двумя спирально изогнутыми листами металла, между которыми формируются каналы для теплоносителей;
- крышки и уплотнения – обеспечивают герметичное разделение потоков и предотвращают смешение сред;
- входные/выходные патрубки – обеспечивают подвод и отвод рабочих сред;
- перегородки и каналы распределения – задают направление потока и способствуют эффективному распределению теплоносителей по секциям аппарата.

1.2 Анализ научной и нормативной литературы

Для разработки алгоритмов теплового, гидравлического и конструктивного расчёта спиральных теплообменных аппаратов был проведён анализ научной и учебной литературы, а также нормативных документов, регламентирующих их конструкцию и параметры.

В качестве базового источника для формирования расчётных алгоритмов использовано учебное пособие «Процессы и аппараты химической технологии» под редакцией А. Н. Плановского, В. М. Рамма и С. З. Кагана [21]. В данном издании подробно изложены методы теплового и гидравлического расчёта теплообменных аппаратов, приведены зависимости для определения коэффициентов теплоотдачи, гидравлического сопротивления и тепловой эффективности, применимые к аппаратам со сложной геометрией каналов, включая спиральные конструкции.

Важным дополнительным источником при формировании расчётных алгоритмов послужил учебник А. Г. Касаткина «Основные процессы и аппараты химической технологии» [6]. В нём систематизированы общие теоретические основы тепловых и массообменных процессов, приведены зависимости для определения режимов течения и коэффициентов теплоотдачи, актуальные для аппаратов с каналами произвольного поперечного сечения.

Теоретическое обоснование применения уравнения для развитого турбулентного режима течения в каналах произвольного сечения подробно изложено в учебном пособии Ф. Ф. Цветкова «Тепломассообмен» [25]. Там же обосновывается степень влияния числа Прандтля на интенсивность конвективного теплообмена в зависимости от физических свойств среды.

Для более глубокого изучения специфики спиральных теплообменников в качестве профильного источника была изучена книга Н. В. Барановского, Л. М. Коваленко и А. Р. Ястребенецкого «Пластинчатые и спиральные теплообменники» [8]. В данном издании детально рассматриваются узкоспециализированные вопросы конструирования, гидродинамики и теплопередачи, характерные именно для этих типов аппаратов. В книге

подробно освещены практические аспекты компоновки аппаратов и специфика расчета их элементов на прочность.

Справочные значения теплофизических свойств рабочих сред — плотности, удельной теплоёмкости, теплопроводности и динамической вязкости — принимались в соответствии с данными, приведёнными в «Теплотехническом справочнике» [24]. Корректный выбор этих параметров непосредственно влияет на точность вычисления безразмерных критериев подобия и, следовательно, на итоговую площадь теплообмена.

При разработке последовательности расчётных шагов использовались методические указания А. М. Гусарова «Тепломассообмен» [15], содержащие инженерные методики и типовые примеры расчёта теплообменных аппаратов.

Практические аспекты проектирования, монтажа и технического обслуживания теплообменного оборудования рассмотрены в учебном пособии А. М. Бакластова «Проектирование, монтаж и эксплуатация тепломассообменных установок» [7]. Данный источник позволил учесть эксплуатационные ограничения аппаратов при постановке требований к геометрии каналов и конструктивным элементам разрабатываемой модели.

Теоретической основой при проектировании архитектуры программного комплекса послужил учебник И. П. Норенкова «Основы автоматизированного проектирования» [20]. В нём сформулированы принципы построения САПР: разделение системы на функциональные подсистемы, организация информационных потоков между расчётным ядром и модулем геометрического моделирования, а также требования к интерфейсу ввода-вывода инженерных данных.

Опыт разработки специализированных CAD-приложений на базе C3D Toolkit описан в работе В. Г. Муфтеева и соавторов «Прикладные САПР и приложения на основе геометрического ядра C3D для проектирования изделий с функциональными кривыми» [19].

Конструктивное исполнение и основные геометрические параметры спиральных теплообменных аппаратов определяются в соответствии с

требованиями нормативного документа ГОСТ 12067–80 [10], в котором регламентируются:

- типы спиральных теплообменников;
- конструкция каналов и корпуса;
- допустимые геометрические размеры;
- требования к материалам и элементам аппарата.

В процессе разработки расчётных алгоритмов и параметрических моделей спиральных теплообменных аппаратов используются следующие нормативные документы:

— ГОСТ 34347–2017 [13] – определяет общие требования к проектированию, изготовлению и эксплуатации стальных сварных аппаратов, работающих под давлением;

— ГОСТ 14249–89 [11]– регламентирует методы расчёта элементов аппаратов на прочность, устойчивость и герметичность при воздействии внутреннего давления и температуры;

— ГОСТ 5632–2014 [14]– содержит перечень марок сталей и сплавов, применяемых для изготовления элементов теплообменных аппаратов, и требования к их свойствам;

— ГОСТ 33259–2015 [12]– устанавливает типы, конструкцию и размеры фланцев, применяемых для присоединения теплообменных аппаратов к трубопроводам;

1.3 Анализ существующих программных решений

На сегодняшний день на рынке инженерного программного обеспечения существует ряд мощных комплексов, предназначенных для расчета и проектирования теплообменного оборудования. Как правило, такие системы ориентированы в первую очередь на технологические расчеты, определение оптимальной площади теплообмена и компоновки аппаратов.

1.3.1 HTRI Xchanger Suite

HTRI Xchanger Suite от компании Heat Transfer Research, Inc. является одним из наиболее авторитетных и широко используемых программных комплексов в нефтегазовой, химической и энергетической отраслях. Продукт представляет собой набор специализированных модулей для теплогидравлического расчета, симуляции и поверочного анализа теплообменников различных типов, включая кожухотрубные, пластинчатые, аппараты воздушного охлаждения и спиральные [4].

Для работы со спиральными пластинчатыми теплообменниками в составе комплекса предусмотрен специализированный модуль Xspe (Рисунок 2). Его алгоритмы базируются на многолетних эмпирических исследованиях HTRI.

Spiral Geometry	
☐ Total area	m2
☒ Plate width	1828.8 mm
☒ Plate length	71120 mm
Core diameter	304.8 mm
Stud square pitch	101.6 mm

Nozzles		
	Hot	Cold
Inlet ID	101.6 mm	101.6 mm
Outlet ID	101.6 mm	101.6 mm

Spiral Exchanger Type

Type I: Countercurrent spiral flow

Рисунок 2 – Панель ввода для указания геометрии спирального теплообменника.

Ключевые функциональные возможности Xspe и Xchanger Suite:

- Сложное теплогидравлическое моделирование: поддержка расчета потоков по схеме «спираль-спираль» (противоток, прямоток), точный расчет локальных профилей температур, давления, коэффициентов теплоотдачи и термического сопротивления по всей длине спирального канала.

- Широкий спектр режимов работы: возможность проведения расчетов как для однофазных потоков (нагрев/охлаждение), так и для процессов с изменением агрегатного состояния (конденсация, кипение).

— Генерация отчетности: формирование подробных спецификаций, таблиц с результатами и графиков распределения параметров вдоль поверхности теплообмена.

— Интеграция с внешними системами: возможность обмена данными с популярными программами для моделирования технологических процессов и базами данных физико-химических свойств веществ (например, REFPROP).

Несмотря на статус отраслевого стандарта в области теплотехники, HTRI Xchanger Suite имеет ряд критических ограничений с точки зрения конструкторской подготовки производства.

Во-первых, продукт генерирует лишь текстовые и табличные данные о геометрии (габариты, ширина канала, площадь), но в нем полностью отсутствует функционал для прямого CAD-моделирования. Инженеру-конструктору приходится вручную переносить расчетные данные в CAD-системы (Компас-3D, SolidWorks, NX) и заново выстраивать сложную трехмерную геометрию спирали Архимеда, что повышает риск ошибок.

Во-вторых, программа распространяется по жесткой коммерческой лицензии с крайне высокой стоимостью владения, что делает ее малодоступной для небольших компаний и учебных заведений.

В-третьих, отсутствие русскоязычного интерфейса и ограничения на поставку зарубежного ПО усложняют использование продукта на отечественных предприятиях.

1.3.2 AspenTech Heat Exchanger Software

Программные решения от американской компании Aspen Technology (AspenTech) представляют собой глобальную платформу для оптимизации производственных активов. В части теплообменного оборудования ключевым продуктом является семейство Aspen Exchanger Design & Rating (EDR), которое включает модули для расчёта различных типов аппаратов (Aspen Shell & Tube Exchanger, Aspen Plate Exchanger и др.) [1].

Данная система тесно интегрирована в общую экосистему процессного моделирования (Aspen HYSYS и Aspen Plus), что позволяет выполнять расчеты теплообменников непосредственно в составе сложных технологических схем заводов.

Функциональные возможности:

- расчёт и оптимизация проектных параметров теплообменников;
- интеграция с процессными моделями в Aspen HYSYS и Aspen Plus;
- поддержка различных типов теплообменников (пластинчатые, кожухотрубные, спиральные, воздушные);
- автоматизация расчётных процессов на основе широкого набора эмпирических моделей и корреляций.

Продукты AspenTech ориентированы исключительно на уровень инженеров-технологов. Их главная задача — выдать математическую модель теплообмена и базовый опросный лист (Data Sheet). Как и в случае с HTRI, система не предоставляет развитых механизмов для параметрического твердотельного моделирования узлов и деталей аппарата. Программа не способна сгенерировать точную геометрию спирального листа, сборочную единицу, развертки или чертежи, необходимые для передачи в производство. Кроме того, система имеет закрытую архитектуру: разработчикам стороннего ПО крайне сложно интегрироваться с расчетным ядром через API для извлечения геометрии.

1.4 Проблематика проектирования

Проектирование трёхмерных моделей спиральных теплообменных аппаратов представляет собой комплексную задачу, включающую расчёт тепловых и гидравлических параметров, создание параметрической геометрической модели, обеспечивающей точность конструирования, и последующую модификацию модели при изменении проектных параметров.

Ключевые проблемы текущих процессов проектирования:

- отсутствие интеграции расчётных методов и геометрического моделирования в едином рабочем пространстве;
- ручное создание трёхмерных моделей в CAD-средах при каждом изменении параметров изделия;
- ограниченные возможности автоматизации и параметризации моделей спиральных теплообменников.

Целью работы является разработка мини-САПР для автоматизированного создания твердотельных параметрических моделей спиральных теплообменных аппаратов с использованием API C3D Toolkit, обеспечивающего унифицированный интерфейс для геометрического моделирования, расчётных алгоритмов и управления моделью. Задачи включают анализ предметной области, выбор архитектуры ПО, реализации параметрического моделирования и функционала расчётов тепловых и гидравлических характеристик.

1.5 Используемые технологии

Разработка специализированных САПР, ориентированных на параметрическое моделирование и инженерные расчёты, требует использования программных технологий, обеспечивающих высокую вычислительную производительность, точность геометрических операций и переносимость между операционными системами.

1.5.1 Выбор языка программирования

В области разработки CAD- и CAE-систем наибольшее распространение получили языки программирования C++, C# и Python.

Языки C# и Python чаще используются для разработки надстроек, плагинов и пользовательских сценариев, однако обладают ограничениями по производительности и доступу к ядрам твердотельного моделирования, что ограничивает их применимость для создания автономных САПР.

Язык C++ является стандартом для разработки геометрических ядер и инженерных САПР. Он обеспечивает высокую производительность вычислений, прямой доступ к структурам данных геометрических моделей, возможность интеграции с низкоуровневыми API геометрических ядер, а также контроль над управлением памятью, критичный для сложных параметрических моделей. Изучение и эффективное использование богатого инструментария данного языка, включая объектно-ориентированные возможности, опирается на фундаментальные труды, такие как руководство С. Праты «Язык программирования C++» [22].

В рамках данной работы в качестве основного языка программирования выбран C++, как наиболее подходящий для реализации параметрического твердотельного моделирования и расчётных алгоритмов. При написании программной части мини-САПР особое внимание уделялось не только реализации заявленного функционала, но и качеству архитектуры приложения. Для создания надежного, читаемого и легко поддерживаемого исходного кода применялись передовые практики конструирования программного обеспечения, подробно изложенные в книге С. Макконнелла «Совершенный код» [17].

1.5.2 Графический фреймворк приложения

Фреймворк Qt широко применяется в профессиональных САД-системах благодаря следующим особенностям:

- поддержка кроссплатформенной разработки;
- встроенные механизмы работы с графикой и 3D-визуализацией;
- нативная интеграция с C++.

Использование Qt позволяет реализовать инженерный интерфейс с параметрическими формами ввода, визуализацией модели и управлением процессом проектирования в едином приложении. При проектировании архитектуры интерфейса, настройке инструментов разработчика и реализации

модулей на базе Qt Framework активно использовались официальные материалы и курсы образовательной платформы Qt Academy [5].

1.5.3 Выбор геометрического ядра

Ключевым компонентом любой САПР является геометрическое ядро, отвечающее за создание, хранение и модификацию трёхмерных моделей. Наиболее известные геометрические ядра:

- Parasolid (Siemens);
- ACIS (Dassault Systemes);
- C3D Toolkit (ООО «Цифровая платформа КОМПАС»).

Геометрическое ядро C3D Toolkit предоставляет API для:

- построения твердотельных и поверхностных моделей;
- создания параметрических зависимостей;
- работы с кривыми, сплайнами и сложной геометрией;
- реализации специализированных CAD-приложений.

Использование C3D Toolkit является обоснованным выбором для разработки отечественной САПР, ориентированной на инженерное проектирование, и обеспечивает независимость от зарубежных программных решений. Для изучения функционала ядра использовалось руководство для разработчиков, расположенное на сайте C3D [3].

1.5.4 Среда разработки

В качестве среды разработки применяется Visual Studio, которая обеспечивает удобные возможности для написания, тестирования и отладки программного кода, способствуя повышению качества процесса разработки.

ГЛАВА 2 МЕТОДИКА РАСЧЁТА ТЕПЛООБМЕННОГО АППАРАТА

Методика расчёта спирального теплообменного аппарата основана на последовательном определении тепловой нагрузки, подборе геометрических параметров каналов, итерационном вычислении коэффициента теплопередачи и расчёте гидравлических характеристик аппарата.

2.1 Определение тепловой нагрузки и материального баланса

На начальном этапе проектирования спирального теплообменного аппарата производится проверка полноты и согласованности входных данных. Ключевой задачей является определение тепловой нагрузки Q (Вт), необходимой для реализации заданного технологического процесса. В случае, если параметры потока на входе и выходе известны не для всех теплоносителей, расчет базируется на фундаментальном уравнении теплового баланса:

$$Q = G \cdot c_p \cdot \Delta t \quad (1)$$

где, G — массовый расход, кг/с;

c_p — удельная теплоемкость среды, Дж/(кг·К);

Δt — разность температур.

Из уравнения (1) определяется количество передаваемой теплоты Q , а также недостающий массовый расход горячей или холодной среды. Тепловая нагрузка Q , является определяющей величиной для последующего нахождения площади поверхности теплообмена.

2.2 Определение среднего температурного напора

Для определения движущей силы процесса теплопередачи используется среднелогарифмическая разность температур (LMTD), которая наиболее точно описывает распределение температур в теплообменных аппаратах при противоточном движении сред [18].

Для аппарата, работающего по схеме противотока, движущая сила рассчитывается по формуле:

$$\Delta t_{cp} = \frac{\Delta t_6 - \Delta t_m}{\ln \left(\frac{\Delta t_6}{\Delta t_m} \right)} \quad (2)$$

где, Δt_6 и Δt_m — большая и меньшая разности температур теплоносителей на концах аппарата.

2.3 Расчет коэффициентов теплоотдачи и теплопередачи

Основным этапом теплового расчета является определение интенсивности теплообмена в каналах аппарата. Выполняется последовательный расчет критериев подобия для каждой из рабочих сред (горячей и холодной), движущихся в каналах спирали прямоугольного сечения.

Сначала определяется минимально необходимая площадь проходного сечения каналов исходя из допустимой скорости движения теплоносителей:

$$f = \frac{G}{\rho w_{max}} \quad (3)$$

где, ρ — плотность среды, кг/м³;

w_{max} — максимально допустимая скорость потока, м/с.

Требуемая ширина канала определяется делением площади проходного сечения на заданный межлистовой зазор. Из полученных значений выбирается большее, после чего принимается ближайшее большее стандартное значение ширины.

После этого рассчитывается эквивалентный диаметр канала $d_{эkv}$ (м). Для прямоугольного канала эквивалентный диаметр определяется выражением:

$$d_{эkv} = \frac{2 \cdot a \cdot b}{a + b} \quad (4)$$

где, a — ширина зазора канала, м;

b — ширина спирального листа, м.

Затем уточняется действительная скорость теплоносителей:

$$w = \frac{G}{\rho \cdot a \cdot b} \quad (5)$$

Далее для каждой среды рассчитываются безразмерные критерии подобия. Их вычисление осуществляется по классическим эмпирическим зависимостям, широко применяемым в теории тепломассообмена [16]:

Число Рейнольдса (Re), характеризующее гидродинамический режим движения среды:

$$Re = \frac{w \cdot d_{\text{экв}} \cdot \rho}{\mu} \quad (6)$$

где, w — фактическая скорость потока в канале (м/с),

ρ — плотность среды (кг/м³),

μ — динамическая вязкость (Па·с).

Число Прандтля (Pr), отражающее влияние физических свойств теплоносителя на интенсивность теплообмена:

$$Pr = \frac{c_p \cdot \mu}{\lambda_{\text{ж}}} \quad (7)$$

где, c_p — удельная теплоемкость среды (Дж/(кг·К)),

$\lambda_{\text{ж}}$ — коэффициент теплопроводности среды (Вт/(м·К)).

Число Нуссельта (Nu), определяющее безразмерный коэффициент теплоотдачи. Для развитого турбулентного режима ($Re \geq 10000$) используется эмпирическое уравнение:

$$Nu = 0,023 \cdot Re^{0,8} \cdot Pr^{0,4} \quad (8)$$

На основе полученного числа Нуссельта вычисляются частные коэффициенты теплоотдачи α_1 и α_2 (Вт/(м²·К)) для каждой стороны поверхности теплообмена:

$$\alpha = \frac{Nu \cdot \lambda_{\text{ж}}}{d_{\text{экв}}} \quad (9)$$

Итоговый коэффициент теплопередачи K (Вт/(м²·К)) рассчитывается как величина, обратная сумме всех термических сопротивлений на пути теплового

потока. Алгоритм учитывает конвективное сопротивление обеих сред, теплопроводность металлической стенки и влияние загрязнений (фактор загрязнения):

$$K = \frac{1}{\frac{1}{\alpha_1} + \frac{1}{\alpha_2} + \frac{\delta_{\text{ст}}}{\lambda_{\text{ст}}} + r_{\text{загр}}} \quad (10)$$

где, α_1, α_2 — коэффициенты теплоотдачи со стороны горячего и холодного теплоносителей (Вт/(м²·К));

$\delta_{\text{ст}}$ — толщина металлического листа спирали (м);

$\lambda_{\text{ст}}$ — коэффициент теплопроводности материала стенки (Вт/(м·К));

$r_{\text{загр}}$ — коэффициент термического сопротивления загрязнений (м²·К/Вт).

Полученное значение коэффициента теплопередачи является ключевым для определения требуемой поверхности теплообмена аппарата

2.4 Геометрический расчет

Особенностью проектирования спирального теплообменника является сложная математическая взаимосвязь между расчетной площадью, длиной листа, количеством витков и итоговым внешним диаметром.

На первом этапе определяется расчетная поверхность теплообмена F (м²), необходимая для передачи заданной тепловой нагрузки:

$$F = \frac{Q}{K \cdot \Delta t_{\text{ср}}} \quad (11)$$

где:

Q — тепловая нагрузка (Вт);

K — коэффициент теплопередачи (Вт/(м²·К));

$\Delta t_{\text{ср}}$ — средне логарифмический температурный напор (К).

Исходя из полученной площади F и выбранной ширины листа (высоты аппарата) B (м), вычисляется общая расчетная длина спиральных листов L (м):

$$L = \frac{F}{2 \cdot B} \quad (10)$$

Множитель 2 в знаменателе обусловлен тем, что поверхность теплообмена образуется двумя сторонами листов, формирующими каналы.

Для реализации геометрической модели используется математическое описание спирали Архимеда. Переход от линейной длины листа к количеству витков требует введения вспомогательных параметров. Сначала определяется шаг спирали t (м), который равен сумме зазора канала и толщины металлического листа:

$$t = \delta_{\text{зaz}} + \delta_{\text{ст}} \quad (11)$$

Затем рассчитывается безразмерный коэффициент x , учитывающий влияние начального диаметра навивки (диаметра керна):

$$x = 0,5 \cdot \left(\frac{d_{\text{вн}}}{t} - 1 \right) \quad (12)$$

где $d_{\text{вн}}$ — диаметр центрального разделительного устройства (керн), вокруг которого производится навивка (м).

Количество витков спирали N находится путем решения уравнения длины спиральной кривой:

$$N = \sqrt{\frac{2 \cdot L}{\pi \cdot t} + x^2} - x \quad (13)$$

Полученное значение N подвергается округлению до ближайшего четного числа в большую сторону.

Итоговым параметром, необходимым для построения корпуса аппарата и его размещения на промышленной площадке, является наружный диаметр спирального блока D (м), который вычисляется по формуле:

$$D = d_{\text{вн}} + 2 \cdot N \cdot t + \delta_{\text{ст}} \quad (14)$$

Так как коэффициент теплоотдачи зависит от диаметра спирали, а диаметр определяется через длину канала и площадь теплообмена, расчёт выполняется итерационно. Расчёт повторяется до выполнения условия:

$$\frac{|K_i - K_{i-1}|}{K_{i-1}} < \varepsilon \quad (15)$$

2.5 Гидравлический расчёт

После определения конструктивных параметров можно провести гидравлический расчёт, для определения потерь давления [9]:

$$\Delta P = \frac{g \cdot L \cdot \rho \cdot w^2 \cdot 0.0113}{Re^{0.25} \cdot a} \quad (16)$$

где, g — ускорение свободного падения, m/s^2 ;

L — длина спирали, m ;

ρ — плотность теплоносителя, kg/m^3 ;

w — скорость теплоносителя, m/s ;

Re — число Рейнольдса;

δ — ширина канала, m .

Мощность, необходимая для преодоления гидравлических сопротивлений при прокачивании через теплообменник определяется по формуле:

$$N = \frac{G \cdot \Delta P}{\rho \cdot \eta} \quad (17)$$

ГЛАВА 3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ РАЗРАБОТАННОГО АЛГОРИТМА

3.1 Архитектура программного комплекса и структуры данных

Основой программного комплекса является класс `SpiralHeatExchanger`, который инкапсулирует в себе все необходимые данные и методы для проведения расчёта спирального теплообменника.

Для организации входных данных и результатов в программе реализованы специализированные структуры:

1. `Fluid` — содержит теплофизические свойства рабочей среды: массовый расход (`G`), плотность (`rho`), теплоёмкость (`cp`), теплопроводность (`lambda`), динамическую вязкость (`viscosity`), входную и выходную температуры (`t_in`, `t_out`), а также допустимую скорость потока в канале (`max_velocity`).

2. `Wall` — описывает характеристики стенки спирали: коэффициент теплопроводности материала (`lambda`), толщину стенки (`thickness`) и фактор термического сопротивления загрязнений (`fouling_factor`).

3. `Geometry` — хранит геометрические параметры каналов: ширины каналов горячей и холодной сред (`width_hot`, `width_cold`), диаметр внутреннего керна (`d_inner`), толщины корпуса и крышки (`shell_thickness`, `cover_thickness`), а также параметры автоматического выбора ширины теплообменника (`auto_width`, `width_min`, `width_max`).

4. `Flange` — описывает геометрические характеристики фланцевых соединений: название, внутренний и наружный диаметры, диаметр окружности болтов, толщину фланца, диаметр и количество болтов.

5. `CalculationResult` — комплексная структура, формируемая по итогам расчёта. Содержит сообщение об ошибке (`error_message`), тепловые параметры (`Q_required`, `LMTD_required`, `Q_real`), гидравлические характеристики (скорости потоков, числа Рейнольдса), коэффициенты теплоотдачи (`alpha_hot`, `alpha_cold`, `K`), геометрические характеристики

аппарата (channel_width, channel_length, turns, D), а также параметры патрубков и фланцев.

Структура Flange не используется в расчётах, но она будет нужна далее для создания модели.

3.2 Реализация расчетных алгоритмов

Математическая модель, описанная в Главе 2, реализована в виде набора приватных методов класса, которые последовательно вызываются в ходе расчёта.

Метод validateInputs (Листинг 1) выполняет проверку корректности входных данных перед началом расчёта. Метод проверяет положительность массовых расходов обоих потоков, а также корректность направления изменения температур: температура на входе горячей среды должна быть выше температуры на выходе, а холодной — ниже.

Листинг 1 – Метод проверки входных данных

```
1:     std::string SpiralHeatExchanger::validateInputs()
2:     {
3:         if (hotFluid.G <= 0 || coldFluid.G <= 0)
4:             return "Массовый расход должен быть положительным";
5:         if (hotFluid.t_in <= hotFluid.t_out)
6:             return "Температура входа горячей среды должна быть выше выхода";
7:         if (coldFluid.t_in >= coldFluid.t_out)
8:             return "Температура входа холодной среды должна быть ниже выхода";
9:
10:        return "";
11:    }
```

Метод calculateLMTD (Листинг 2) вычисляет среднелогарифмическую разность температур. Для реализации всех вычислительных операций в программе (таких как логарифмирование, возведение в степень, вычисление абсолютных значений) применялись функции стандартной математической библиотеки C++ <cmath> [2].

Листинг 2 – Метод вычисления температурного напора

```
1:  double SpiralHeatExchanger::calculateLMTD(double t_hot_in, double t_hot_out,
2:      double t_cold_in, double t_cold_out) const
3:  {
4:      double dt1 = std::abs(t_hot_in - t_cold_out);
5:      double dt2 = std::abs(t_hot_out - t_cold_in);
6:
7:      if (std::abs(dt1 - dt2) < 1e-9)
8:          return dt1;
9:
10:     double larger = std::max(dt1, dt2);
11:     double smaller = std::min(dt1, dt2);
12:     return (larger - smaller) / log(larger / smaller);
13: }
```

Метод calculateAlpha (Листинг 3) принимает на вход объект свойств среды, скорость потока, эквивалентный диаметр канала и текущий наружный диаметр спирали Dc. Метод вычисляет число Рейнольдса и проверяет режим течения: если $Re \leq 10\,000$, коэффициент теплоотдачи принимается равным нулю (развитый турбулентный режим не достигнут). Далее вычисляются числа Прандтля и Нуссельта с поправкой на кривизну канала, и на их основе определяется коэффициент теплоотдачи конвекцией.

Листинг 3 – Метод вычисления коэффициента теплоотдачи

```
1:  double SpiralHeatExchanger::calculateAlpha(const Fluid& f, double velocity,
2:      double d_eq, double Dc) const
3:  {
4:      if (velocity <= 0.0)
5:          return 0.0;
6:
7:      double Re = velocity * d_eq * f.rho / f.viscosity;
8:      if (Re <= ReConst1)
9:          return 0.0;
10:
11:     double Pr = f.cp * f.viscosity / f.lambda;
12:     double a_coof = calculateAlphaCoof(d_eq, Dc);
13:     double Nu = 0.023 * pow(Re, 0.8) * pow(Pr, 0.4) * a_coof;
14:
15:     return Nu * f.lambda / d_eq;
16: }
```

Метод calculateAlphaCoof (Листинг 4) вычисляет поправочный коэффициент к критерию Нуссельта, учитывающий влияние кривизны спирального канала. При нулевом диаметре спирали (начало итерационного процесса) метод возвращает значение 1,1; в остальных случаях используется

формула с отношением эквивалентного диаметра канала к текущему диаметру спирали.

Листинг 4 – Метод вычисления поправочного коэффициента теплоотдачи

```
1: double SpiralHeatExchanger::calculateAlphaCoof(double de, double Dc) const
2: {
3:     if (Dc <= 0.0)
4:         return 1.1;
5:
6:     return 1.0 + 3.54 * de / Dc;
7: }
```

Все промежуточные вычисления объединены в методе calculate, который поэтапно проверяет входные данные, вычисляет требуемые тепловые нагрузки и температурный напор, подбирает стандартную ширину канала, итерационно уточняет коэффициент теплопередачи и диаметр спирали, а затем определяет число витков, длину канала и гидравлические потери. Реализация данного метода представлена в Приложении А.

3.3 Алгоритм выбора ширины

В классе инициализирован вектор стандартных значений ширины в соответствии с нормативными требованиями.

Листинг 5 – Вектор стандартных значений ширины теплообменника

```
1: const std::vector<double> standard_width = { 0.2, 0.32, 0.5, 0.8, 1.0, 1.1, 1.25, 1.5 };
```

Главный публичный метод calculate (Листинг 6) реализует следующий алгоритм выбора ширины. Из условия не превышения максимально допустимых скоростей вычисляются минимально необходимые поперечные сечения каналов для горячей и холодной сред. Делением этих сечений на соответствующие ширины каналов получают минимально допустимое значение be. Из вектора standard_width выбирается первое значение, не меньшее полученного минимума и, при необходимости, лежащее в заданном диапазоне [width_min, width_max]. Если подходящее значение не найдено, метод возвращает CalculationResult с заполненным полем error_message.

Листинг 6 – Фрагмент метода calculate

```
1:  double f_hot = hotFluid.G / (hotFluid.rho * hotFluid.max_velocity);
2:  double f_cold = coldFluid.G / (coldFluid.rho * coldFluid.max_velocity);
3:
4:  double be_hot = f_hot / geo.width_hot;
5:  double be_cold = f_cold / geo.width_cold;
6:
7:  double be_min = std::max(be_hot, be_cold);
8:
9:  auto it = std::find_if(standard_width.begin(), standard_width.end(),
10:    [&](double h) {
11:      if (h < be_min) return false;
12:      if (!geo.auto_width) {
13:        if (h < geo.width_min || h > geo.width_max) return false;
14:      }
15:      return true;
16:    });
```

После выбора ширины метод переходит к итерационному уточнению коэффициента теплопередачи K и наружного диаметра спирали D_c . На каждой итерации вычисляются коэффициенты теплоотдачи с учётом кривизны канала, суммарное термическое сопротивление, требуемая площадь теплообменной поверхности и число витков. По результатам расчёта формируется структура `CalculationResult`, содержащая все тепловые, гидравлические и геометрические характеристики спирального теплообменника.

ГЛАВА 4 ПОСТРОЕНИЕ МОДЕЛИ ПРИ ПОМОЩИ API C3D

Глава посвящена детальному процессу построения электронных деталей с использованием модуля C3D Modeler, который является инструментом для создания трехмерных моделей.

4.1 Моделирование спиральных каналов

Центральным элементом проектируемого аппарата является спиральный канал. Для его построения разработана функция CreateSpiral, которая автоматизирует генерацию сложной геометрии на основе расчетных данных: внутреннего диаметра, количества витков и ширины каналов.

Процесс моделирования спиральных каналов начинается с математического описания траектории. Программа подготавливает массив точек `SArray<MbCartPoint>`, количество которых напрямую зависит от заданного числа витков. Использование 64 точек на каждый оборот позволяет добиться высокой точности аппроксимации при последующем построении кривой. В цикле `for` программа вычисляет полярный угол `phi` и соответствующий ему радиус `r`, который линейно увеличивается в зависимости от шага витка `step_per_turn` (Листинг 7).

Листинг 7 – Создание набора точек спирали

```
1:     size_t points_per_turn = 64;
2:     size_t total_points = static_cast<size_t>(points_per_turn * turns) + 1;
3:
4:     SArray<MbCartPoint> points(total_points, 1);
5:     double delta_phi = M_PI2 / (double)points_per_turn;
6:
7:     for (size_t i = 0; i < total_points; ++i) {
8:         double phi = i * delta_phi;
9:         double r = current_r_start + (thickness / 2.0) +
10:             (step_per_turn * phi / M_PI2);
11:         points.push_back(MbCartPoint(r * cos(phi + phase_shift),
12:                                     r * sin(phi + phase_shift)));
13:     }
```

После генерации набора координат код переходит к созданию гладкого геометрического объекта. С помощью метода `MbCubicSpline::Create` массив точек превращается в кубический сплайн, который обеспечивает непрерывность касательных векторов во всех узлах. Эта кривая затем

оборачивается в объект MbContour, который выступает в роли контейнера для составных сегментов профиля. На данном этапе спираль представляет собой чисто математическую линию в плоскости XY (Листинг 8).

Листинг 8 – Создание контура профиля

```
1:     SPtr<MbCurve> spiral(MbCubicSpline::Create(points, false));
2:     SPtr<MbContour> contour(new MbContour);
3:     contour->AddSegment(spiral);
```

Для того чтобы аппарат можно было технологически соединить с корпусом, в коде реализована логика достройки прямолинейного участка. Программа запрашивает параметры конечной точки сплайна через GetTMax() и вычисляет вектор касательной tangent. После нормализации вектора код рассчитывает необходимую длину «хвоста» tail_length, чтобы спираль гарантированно достигла нужного радиуса сопряжения с кожухом. Новый прямолинейный сегмент MbLineSegment создается на основе найденной точки и направления, после чего добавляется в общий контур (Листинг 9).

Листинг 9 – Достройка конца спирали

```
1:     double t_end = spiral->GetTMax();
2:     MbCartPoint p_end;
3:     MbVector tangent;
4:     spiral->GetLimitPoint(2, p_end);
5:     spiral->Tangent(t_end, tangent);
6:     tangent.Normalize();
7:
8:     double r_target = r_start + (step_per_turn * (turns + 0.5)) + thickness;
9:     double b = 2.0 * (p_end.x * tangent.x + p_end.y * tangent.y);
10:    double c = (p_end.x * p_end.x + p_end.y * p_end.y) -
11:               (r_target * r_target);
12:    double D = b * b - 4.0 * c;
13:    double tail_length = (D >= 0) ? (-b + sqrt(D)) / 2.0 :
14:                             (thickness + gap_1);
15:
16:    contour->AddSegment(new MbLineSegment(p_end,
17:        MbCartPoint(p_end.x + tangent.x * tail_length,
18:                    p_end.y + tangent.y * tail_length)));
```

Завершающим этапом является переход от двумерного эскиза к трехмерному твердому телу. Программа определяет опорную плоскость MbPlane и подготавливает данные для операции заметания в объекте MbSweptData. Основная настройка геометрии стенки происходит в структуре ExtrusionValues. Здесь код задает высоту спирали вдоль оси Z, распределяя

ширину канала `channel_width` поровну между параметрами `side1` и `side2` для симметричного расположения относительно плоскости построения. За толщину отвечают параметры `thickness1` и `thickness2`, которые превращают тонкую линию контура в лист металла (Листинг 10).

Листинг 10 – Операция выдавливания спирали

```
1:  SPtr<MbSurface> surface(new MbPlane(place));
2:  MbSweptData curvesData(*surface, contours);
3:
4:  ExtrusionValues extrudeParams;
5:  extrudeParams.side1.scalarValue = channel_width / 2;
6:  extrudeParams.side1.way = sw_scalarValue;
7:  extrudeParams.side2.scalarValue = channel_width / 2;
8:  extrudeParams.side2.way = sw_scalarValue;
9:  extrudeParams.thickness1 = thickness / 2.0;
10: extrudeParams.thickness2 = thickness / 2.0;
11:
12: MbSNameMaker snMakerExtrude(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone,
13: 0);
14:
15: PArray<MbSNameMaker> cNames(0, 1, false);
16:
17: MbSolid* surfaceSolid = nullptr;
18: ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
19:   extrudeParams, snMakerExtrude, cNames, surfaceSolid);
```

В результате вызова функции `::ExtrusionSolid` ядро C3D формирует твердотельный объект `MbSolid`. Этот объект представляет собой сложную винтовую поверхность с заданной толщиной стенки и прямолинейным выходом, полностью готовую для объединения с центральным делителем и внешним корпусом теплообменника.

Функция `CreateSpiralDivider` отвечает за построение начальных перегородок в центре аппарата. Эти элементы служат «точкой старта» для навивки листов и обеспечивают герметичное разделение потоков в центральной части теплообменника.

Сначала происходит вычисление вспомогательной величины — радиальной коррекции. Она необходима, так как зазоры между каналами могут быть разными, и начало спирали должно быть смещено, чтобы витки ложились корректно.

Логика программы построена на использовании условного оператора `if (is_second)`, который разделяет построение на два независимых блока. Для

первого блока координаты рассчитываются так, чтобы создать прямоугольники, примыкающие каждый к своей спирали. Это позволяет сместить геометрию и избежать пересечения деталей, обеспечивая точную стыковку с листами, имеющими разный шаг из-за различных зазоров между каналами (Листинг 11).

Листинг 11 – Нахождение точек профиля керна

```
1:         double radial_correction = (gap_2 - gap_1) / 2.0;
2:
3:         SArray<MbCartPoint> rectPoints(4, 1);
4:
5:         if (is_second) {
6:             double x_start = d_inner / 2.0 + thickness;
7:             double x_end = -(d_inner / 2.0 + radial_correction) - thickness / 2;
8:             rectPoints.push_back(MbCartPoint(x_start, 0));
9:             rectPoints.push_back(MbCartPoint(x_start, -thickness / 2.0));
10:            rectPoints.push_back(MbCartPoint(x_end, -thickness / 2.0));
11:            rectPoints.push_back(MbCartPoint(x_end, 0));
12:        }
13:        else {
14:            double x_start = d_inner / 2.0 + thickness / 2;
15:            double x_end = -(d_inner / 2.0 + radial_correction) - thickness;
16:            rectPoints.push_back(MbCartPoint(x_start, 0));
17:            rectPoints.push_back(MbCartPoint(x_start, thickness / 2.0));
18:            rectPoints.push_back(MbCartPoint(x_end, thickness / 2.0));
19:            rectPoints.push_back(MbCartPoint(x_end, 0));
20:        }
21:
22:        SPtr<MbContour> contour(new MbContour());
23:        for (size_t i = 0; i < 4; ++i)
24:            contour->AddSegment(new MbLineSegment(rectPoints[i], rectPoints[(i +
1) % 4]));
```

Завершающим этапом работы функции является преобразование плоского чертежа в объемную деталь (Листинг 12).

Листинг 12 – Настройка параметров выдавливания керна

```
1:  MbPlacement3D place;  
2:  RPArray<MbContour> contours(1, 1);  
3:  contours.push_back(contour);  
4:  SPtr<MbSurface> surface(new MbPlane(place));  
5:  MbSweptData curvesData(*surface, contours);  
6:  
7:  ExtrusionValues extrudeParams;  
8:  extrudeParams.side1.scalarValue = channel_width / 2;  
9:  extrudeParams.side1.way = sw_scalarValue;  
10: extrudeParams.side2.scalarValue = channel_width / 2;  
11: extrudeParams.side2.way = sw_scalarValue;  
12:  
13: MbSolid* dividerSolid = nullptr;  
14: MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);  
15: PArray<MbSNameMaker> cNames(0, 1, false);  
16: ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,  
17:     extrudeParams, snMaker, cNames, dividerSolid);
```

В результате выполнения этого кода создаются два блока, которые при последующей сборке образуют центральный узел, к которому «привариваются» спиральные листы.

На Рисунок 3 показана модель спиралей, соединённых с керном.

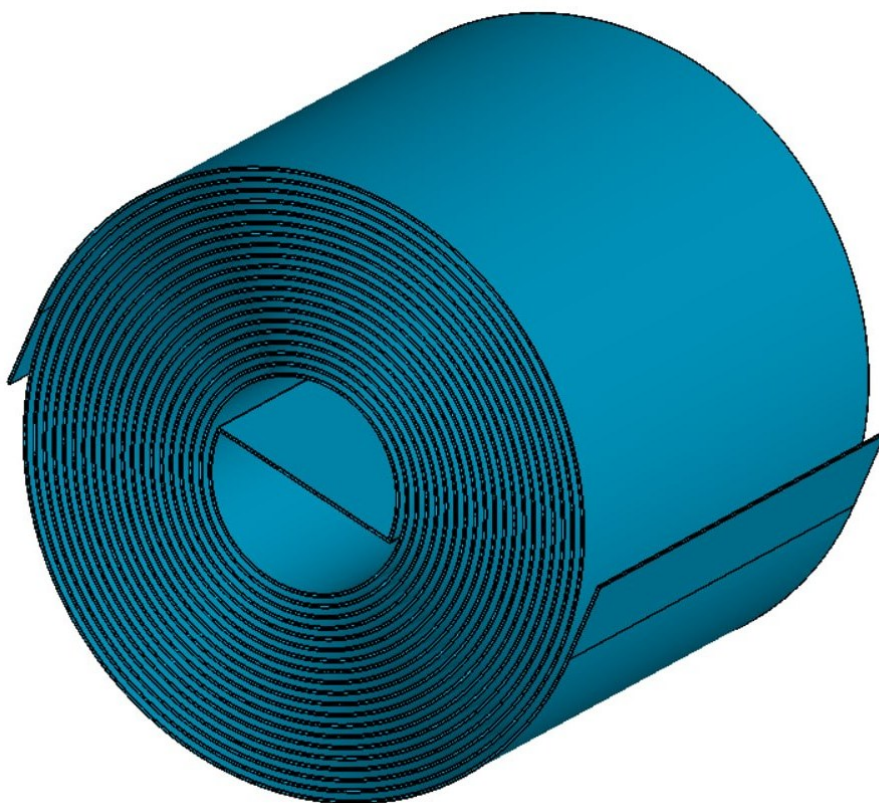


Рисунок 3 – Модель спиралей с керном

4.2 Моделирование обечайки

Обечайка в функции CreateShell представляет собой внешний защитный кожух, который не только удерживает давление теплоносителя, но и служит основой для крепления всей арматуры. Процесс её создания начинается с формирования кругового профиля. В коде инициализируется объект MbArc, радиус которого рассчитывается как средняя линия будущего металла. Программа оборачивает дугу в замкнутый контур MbContour, создавая идеальную окружность, которая станет сечением для последующего выдавливания (Листинг 13).

Листинг 13 – Создание окружности обечайки

```
1:         double r_mid = d_in_shell / 2.0 + shell_thickness / 2.0;
2:
3:         SPtr<MbArc> circle(new MbArc(r_mid));
4:         SPtr<MbContour> contour(new MbContour());
5:         contour->AddSegment(circle);
```

Переход к трехмерной модели осуществляется через уже знакомый механизм выдавливания. Программа учитывает не только основную ширину канала, но и длину посадочного места под прокладки (gasket_length). В коде значения side1 и side2 увеличиваются на величину зазора для прокладки, что делает обечайку чуть длиннее внутреннего спирального блока. Это необходимо для обеспечения герметичности при закрытии аппарата крышками. Для придания стенке физической толщины параметры thickness1 и thickness2 устанавливаются равными половине толщины листа, формируя твердотельную трубу (Листинг 14).

Листинг 14 – Выдавливание обечайки корпуса

```
1:  MbPlacement3D place;
2:  SPtr<MbSurface> surface(new MbPlane(place));
3:  MbSweptData curvesData(*surface, contours);
4:
5:  ExtrusionValues extrudeParams;
6:  extrudeParams.side1.scalarValue = channel_width / 2 + gasket_length;
7:  extrudeParams.side1.way = sw_scalarValue;
8:  extrudeParams.side2.scalarValue = channel_width / 2 + gasket_length;
9:  extrudeParams.side2.way = sw_scalarValue;
10: extrudeParams.thickness1 = shell_thickness / 2.0;
11: extrudeParams.thickness2 = shell_thickness / 2.0;
12:
13: MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
14: PArray<MbSNameMaker> cNames(0, 1, false);
15:
16: MbSolid* shellSolid = nullptr;
17: ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
18:   extrudeParams, snMaker, cNames, shellSolid);
```

Для создания отверстий под патрубки программа использует метод булевого вычитания. Сначала создается вспомогательный объект — цилиндр-инструмент. Для этого используется структура MbElementarySolidParams, где задаются размеры будущего отверстия. Тело поворачивается на 90 градусов вокруг оси Y с помощью метода Rotate, чтобы цилиндр-инструмент был направлен перпендикулярно стенке обечайки (Листинг 15).

Листинг 15 – Создание цилиндра

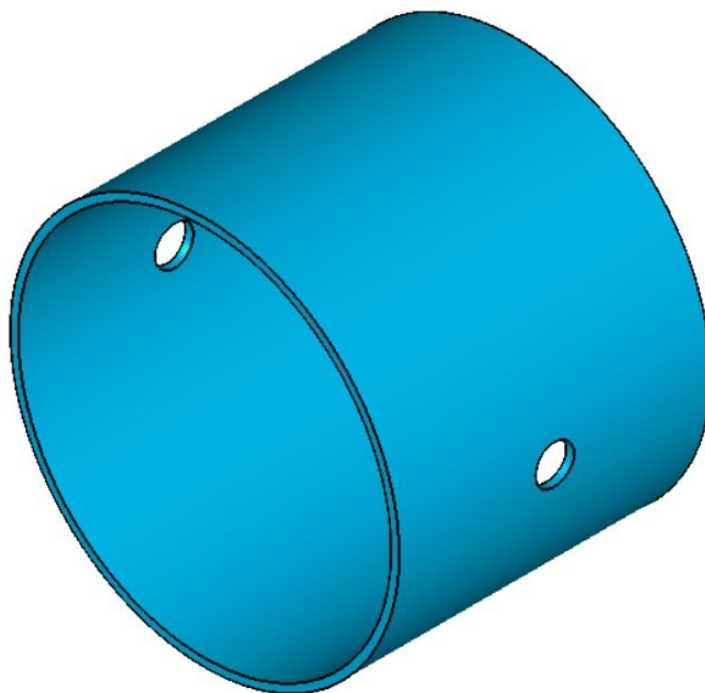
```
1:  MbPlacement3D holePlace;
2:  holePlace.Rotate(MbAxis3D::yAxis, M_PI_2);
3:
4:  MbSNameMaker snHole(ct_ElementarySolid, MbSNameMaker::i_SideNone, 1);
5:  MbElementarySolidParams cylParams(snHole);
6:
7:  double halfLength = r_mid + shell_thickness / 2.0 + 10.0;
8:  cylParams.InitCylinder(holePlace, hole_diameter / 2.0,
9:   halfLength, halfLength);
10:
11: MbElementarySolidResults results;
12: if (::ElementarySolid(cylParams, results) != rt_Success)
13:   return shellSolid;
```

В финальной части функции основной объект обечайки и вспомогательный цилиндр передаются в функцию ::BooleanResult (Листинг 16).

Листинг 16 – Операция вычитания

```
1:      MbSNameMaker boolSnMaker(ct_BooleanSolid, MbSNameMaker::i_SideNone, 0);
2:      MbBooleanOperationParams pDiff(bo_Difference, true, boolSnMaker);
3:      SolidSPtr shellPtr(shellSolid);
4:      SolidSPtr boolResult;
5:
6:      if (::BooleanResult(shellPtr, cm_Same, holeTool, cm_Copy,
7:                          pDiff, boolResult) == rt_Success && boolResult) {
8:          shellPtr = nullptr;
9:          shellSolid = boolResult.detach();
10:     }
```

Полученный результат — это деталь корпуса с заданной толщиной стенок и подготовленным местом под патрубок.



После того как обечайка сформирована, переходим к фланцу корпуса, за создание которых отвечает функция `CreateShellFlange`. Код начинается с расчета геометрии: программа определяет внешний радиус будущего кольца, отталкиваясь от диаметра обечайки и высоты самого фланца. Чтобы задать форму детали, код создает круговую дугу `MbArc`, которая затем упаковывается в контур для превращения в полноценный эскиз (Листинг 17).

Листинг 17 – Создание эскиза окружности фланца

```
1: double r_outer = d_in_shell / 2.0 + shell_thickness;
2: double r_flange_mid = r_outer + flange_height;
3:
4: SPtr<MbArc> circle1(new MbArc(r_flange_mid));
5: SPtr<MbContour> contour(new MbContour());
6: contour->AddSegment(circle1);
7:
8: RPAarray<MbContour> contours(1, 1);
9: contours.push_back(contour);
```

В отличие от симметричного выдавливания листов спирали, здесь параметры side1 и side2 в структуре ExtrusionValues используются для точного позиционирования детали относительно торца обечайки. Программа настраивает смещение так, чтобы фланец «наращивался» на край корпуса, создавая посадочную полку (Листинг 18). Значение thickness1 и thickness2 здесь определяет радиальную ширину фланца, превращая тонкую окружность в массивное плоское кольцо.

Листинг 18 – Выдавливание фланца

```
1: MbPlacement3D place;
2: SPtr<MbSurface> surface(new MbPlane(place));
3: MbSweptData curvesData(*surface, contours);
4:
5: ExtrusionValues extrudeParams;
6: extrudeParams.side1.scalarValue = channel_width / 2 + gasket_length;
7: extrudeParams.side1.way = sw_scalarValue;
8: extrudeParams.side2.scalarValue = -channel_width / 2 + gasket_length
9:                                     + flange_width;
10: extrudeParams.side2.way = sw_scalarValue;
11: extrudeParams.thickness1 = flange_height;
12: extrudeParams.thickness2 = flange_height;
```

Чтобы прорезать отверстия в созданном теле фланца, программа генерирует вспомогательное тело-инструмент holesTool. Этот объект представляет собой набор цилиндров, полученных путем выдавливания подготовленных малых окружностей. Финальный этап — выполнение булевой операции вычитания bo_Difference.

Функция ::BooleanResult принимает основное тело фланца и «вычитает» из него массив цилиндров. Если операция проходит успешно, код возвращает модифицированный объект MbSolid, который теперь имеет все необходимые

технологические отверстия и готов к стыковке с крышками аппарата в общей сборке (Листинг 19).

Листинг 19 – Создание отверстий в фланце

```
1:      ExtrusionValues holeParams;  
2:      holeParams.side1.scalarValue = flange_width + 4.0;  
3:      holeParams.side1.way = sw_scalarValue;  
4:  
5:      MbSNameMaker snHoles(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone,  
6:                                                                    3);  
7:      MbSolid* holesTool = nullptr;  
8:      ::ExtrusionSolid(holeData, holePlace.Normal() * -1.0, nullptr,  
9:                                                                nullptr, false,  
10:                     holeParams, snHoles, PArray<MbSNameMaker>(0, 1, false),holesTool);  
11:  
12:      if (holesTool) {  
13:          SolidSPtr resultBody;  
14:          MbSNameMaker bNames(ct_BooleanSolid, MbSNameMaker::i_SideNone,  
15:                                                                    20);  
16:          MbBooleanOperationParams bParams(bo_Difference, true, bNames);  
17:          SolidSPtr solid1(flangeSolid);  
18:          SolidSPtr solid2(holesTool);  
19:          if (::BooleanResult(solid1, cm_Same, solid2, cm_Same, bParams,  
20:                           resultBody) == rt_Success)  
21:              flangeSolid = resultBody.detach();  
22:      }
```

На Рисунок 4 представлена модель фланца.

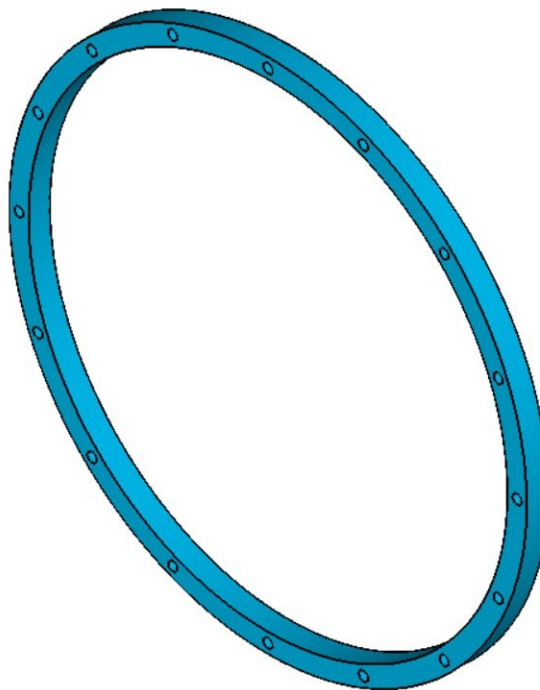


Рисунок 4 – Модель фланца обечайки

4.3 Моделирование патрубков

Для моделирования патрубков в коде предусмотрена универсальная функция `CreateNozzle`. Она отвечает за построение сложного узла, состоящего из трубы и фланца с отверстиями. Процесс начинается с формирования сечения трубы. Программа создает окружность `MbArc`, радиус которой вычисляется как среднее значение между внутренним и внешним диаметром стенки. Этот профиль упаковывается в контур и подготавливается к выдавливанию вдоль нормали, заданной в параметре `placement` (Листинг 20).

Листинг 20 – Создание эскиза патрубка

```
1: double r_pipe_mid = (d_in + wall_thickness) / 2.0;
2: SPtr<MbArc> pipeCircle(new MbArc(MbCartPoint(0, 0), r_pipe_mid));
3:
4: SPtr<MbContour> pipeContour(new MbContour());
5: pipeContour->AddSegment(pipeCircle);
6:
7: RPArray<MbContour> pipeContours(1, 1);
8: pipeContours.push_back(pipeContour);
9:
10: SPtr<MbSurface> pipeSurface(new MbPlane(placement));
```

После того как тело трубы готово, код приступает к созданию фланца патрубка. Чтобы фланец оказался на нужном конце трубы, программа использует объект `flangePlace`, который смещается вдоль оси трубы на расстояние, равное её длине за вычетом толщины самого фланца (Листинг 21). Геометрия фланца строится по той же логике экструзии: создается кольцевой профиль, который «раздувается» до нужного внешнего диаметра за счет параметров толщины в структуре `ExtrusionValues`.

Листинг 21 – Создание фланца патрубка

```
1: MbPlacement3D flangePlace(placement);
2: MbVector3D offset = flangePlace.Normal();
3: offset *= (pipe_len - flange_thickness);
4: flangePlace.Move(offset);
5:
6: double flange_width = (flange_d - d_in) / 2.0;
7: double r_flange_mid = (d_in + flange_width) / 2.0;
8:
9: SPtr<MbArc> flangeCircle(new MbArc(MbCartPoint(0, 0), r_flange_mid));
10: SPtr<MbContour> flangeContour(new MbContour());
11: flangeContour->AddSegment(flangeCircle);
12: RPArray<MbContour> flangeContours(1, 1);
13: flangeContours.push_back(flangeContour);
```

```

14:   SPtr<MbSurface> flangeSurface(new MbPlane(flangePlace));
15:   MbSweptData flangeData(*flangeSurface, flangeContours);
16:
17:   ExtrusionValues flangeParams;
18:   flangeParams.side1.scalarValue = flange_thickness;
19:   flangeParams.side1.way = sw_scalarValue;
20:   flangeParams.thickness1 = flange_width / 2.0;
21:   flangeParams.thickness2 = flange_width / 2.0;
22:
23:   MbSNameMaker snFlange(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 2);
24:   MbSolid* flangeSolid = nullptr;
25:   ::ExtrusionSolid(flangeData, flangePlace.Normal(), nullptr, nullptr, false,
26:     flangeParams, snFlange, PArray<MbSNameMaker>(0, 1, false), flangeSolid);

```

Продолжение Листинг 21

Когда труба и фланец созданы как два отдельных тела, программа объединяет их в единый монолит. Для этого используется функция `::UnionSolid` (Листинг 22). После объединения запускается цикл создания отверстий под болты. Код рассчитывает положение каждого отверстия на делительной окружности и формирует массив контуров, который затем превращается в массив цилиндров `holesTool`.

Листинг 22 – Создание отверстий фланца

```

1:   RPArray<MbContour> holeContours(num_holes, 1);
2:   double angle_step = M_PI2 / (double)num_holes;
3:
4:   for (int i = 0; i < num_holes; ++i) {
5:     double angle = i * angle_step;
6:     double x = (hole_pcd / 2.0) * cos(angle);
7:     double y = (hole_pcd / 2.0) * sin(angle);
8:     MbArc* holeCircle = new MbArc(MbCartPoint(x, y), hole_d / 2.0);
9:     MbContour* cnt = new MbContour();
10:    cnt->AddSegment(holeCircle);
11:    holeContours.push_back(cnt);
12:  }

```

Финальный шаг — это вычитание объема отверстий из тела патрубка. С помощью операции `bo_Difference` в функции `::BooleanResult` программа прорезает фланец (Рисунок 5).

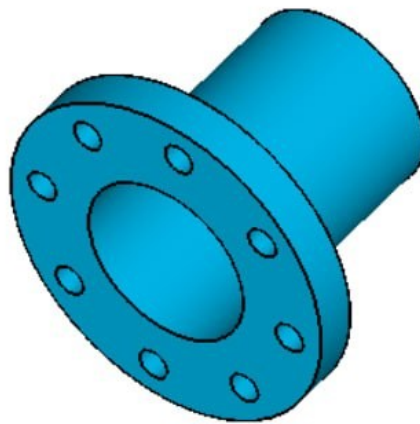


Рисунок 5 – Модель патрубка

Полная модель корпуса представлена на Рисунок 6.

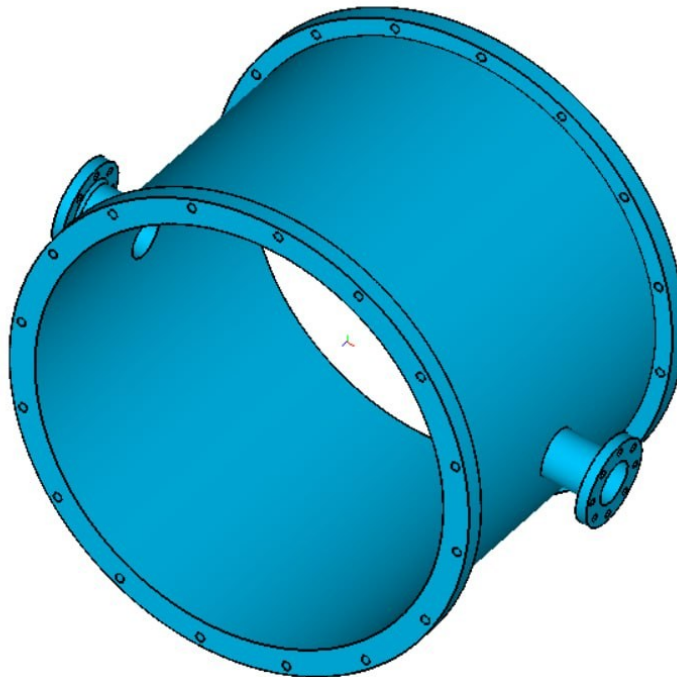


Рисунок 6 – Модель корпуса

4.4 Моделирование прокладок

Процесс начинается с вычисления координаты z_{start} . Программа автоматически определяет, где должна находиться прокладка (сверху или снизу), и смещает плоскость построения так, чтобы деталь точно легла на торец обечайки (Листинг 23).

Листинг 23 – Создание эскиза прокладки

```
1:         double r_outer_metal = d_in_shell / 2.0;
2:
3:         double z_start = is_top
4:             ? (channel_width / 2.0)
5:             : -(channel_width / 2.0 + gasket_length);
6:
7:         MbPlacement3D place;
8:         place.Move(MbVector3D(0.0, 0.0, z_start));
9:
10:        SPtr<MbArc> circle(new MbArc(r_outer_metal));
11:        SPtr<MbContour> contour(new MbContour());
12:        contour->AddSegment(circle);
13:
14:        RPArray<MbContour> contours(1, 1);
15:        contours.push_back(contour);
```

Геометрия прокладки создается как обычное кольцо путем экструзии кругового профиля MbArc. В параметрах выдавливания ExtrusionValues указывается толщина gasket_length, при этом выдавливание идет только в одну сторону (Листинг 24).

Листинг 24 – Выдавливание прокладки

```
1:        SPtr<MbSurface> surface(new MbPlane(place));
2:        MbSweptData curvesData(*surface, contours);
3:
4:        ExtrusionValues extrudeParams;
5:        extrudeParams.side1.scalarValue = gasket_length;
6:        extrudeParams.side1.way = sw_scalarValue;
7:        extrudeParams.side2.scalarValue = 0.0;
8:        extrudeParams.side2.way = sw_scalarValue;
9:
10:       MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
11:       PArray<MbSNameMaker> cNames(0, 1, false);
12:
13:       MbSolid* gasketSolid = nullptr;
14:       ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
15:           extrudeParams, snMaker, cNames, gasketSolid);
```

По аналогии с методами выше создается входное отверстие в прокладке. Код вычисляет положение центра отверстия, генерирует вспомогательный цилиндр через MbElementarySolidParams и «прорекает» им основное тело прокладки с помощью булевой операции вычитания bo_Difference (Листинг 25). В итоге получается готовый уплотнительный элемент.

Листинг 25 – Создание отверстия в прокладке

```
1: double r_hole = r_start / 2.0;
2: double hole_y = is_top ? r_hole : -r_hole;
3: double z_mid = z_start + gasket_length / 2.0;
4: double halfLen = gasket_length / 2.0 + 1.0;
5:
6: MbPlacement3D cylPlace;
7: cylPlace.Move(MbVector3D(0.0, hole_y, z_mid));
8:
9: MbSNameMaker snCyl(ct_ElementarySolid, MbSNameMaker::i_SideNone, 1);
10: MbElementarySolidParams cylParams(snCyl);
11: cylParams.InitCylinder(cylPlace, nozzle_d_in / 2.0, halfLen, halfLen);
12:
13: MbElementarySolidResults cylResults;
14: if (::ElementarySolid(cylParams, cylResults) == rt_Success) {
15:     MbSolid* holeTool = cylResults.GetResultSolid();
16:     SolidSPtr resultBody;
17:     MbSNameMaker bNames(ct_BooleanSolid, MbSNameMaker::i_SideNone, 20);
18:     MbBooleanOperationParams bParams(bo_Difference, true, bNames);
19:     SolidSPtr solid1(gasketSolid);
20:     SolidSPtr solid2(holeTool);
21:     if (::BooleanResult(solid1, cm_Same, solid2, cm_Same, bParams, resultBody)
== rt_Success)
22:         gasketSolid = resultBody.detach();
23: }
```

На Рисунок 7 изображена модель прокладки.

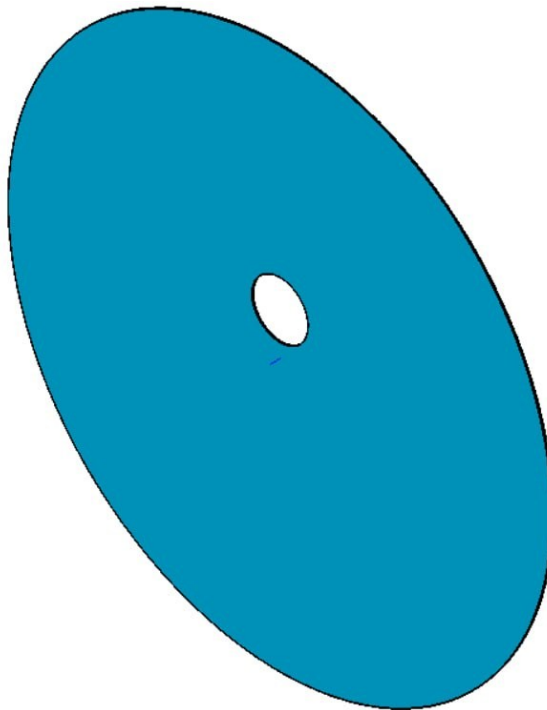


Рисунок 7 – Модель прокладки

4.5 Моделирование днища

Процесс начинается с создания основного диска крышки. Программа рассчитывает `r_cap` – радиус днища (Листинг 26). Построение ведется методом экструзии кругового профиля на толщину `cap_thickness`.

Листинг 26 – Создание крышки

```
1:  double r_outer = d_in_shell / 2.0 + shell_thickness;
2:  double r_cap = r_outer + flange_height * 2;
3:
4:  MbPlacement3D place;
5:  place.Move(MbVector3D(0.0, 0.0, z_inner));
6:
7:  SPtr<MbArc> circle(new MbArc(r_cap));
8:  SPtr<MbContour> contour(new MbContour());
9:  contour->AddSegment(circle);
10:
11:  RPAArray<MbContour> contours(1, 1);
12:  contours.push_back(contour);
13:
14:  SPtr<MbSurface> surface(new MbPlane(place));
15:  MbSweptData curvesData(*surface, contours);
16:
17:  ExtrusionValues extrudeParams;
18:  extrudeParams.side1.scalarValue = cap_thickness;
19:  extrudeParams.side1.way = sw_scalarValue;
20:  extrudeParams.side2.scalarValue = 0.0;
21:  extrudeParams.side2.way = sw_scalarValue;
22:
23:  MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
24:  PArray<MbSNameMaker> cNames(0, 1, false);
25:
26:  MbSolid* capSolid = nullptr;
27:  ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
28:    extrudeParams, snMaker, cNames, capSolid);
```

Для обеспечения болтового соединения с корпусом, в днище программно генерируется сетка отверстий. Код использует цикл, в котором на основе полярных координат вычисляются центры отверстий на заданном радиусе `hole_r`. Массив этих отверстий превращается в объемный инструмент `holesTool`, который затем вычитается из тела крышки через операцию `bo_Difference` (Листинг 27).

Листинг 27 – Создание отверстий

```
1:         double angle_step = M_PI2 / (double)hole_count;
2:         RPArray<MbContour> holeContours(hole_count, 1);
3:
4:         for (int i = 0; i < hole_count; ++i) {
5:             double angle = i * angle_step;
6:             double hole_r = r_outer + flange_height;
7:             MbArc* hc = new MbArc(MbCartPoint(hole_r * cos(angle), hole_r *
sin(angle)), d_hole / 2.0);
8:             MbContour* cnt = new MbContour();
9:             cnt->AddSegment(hc);
10:            holeContours.push_back(cnt);
11:        }
```

Завершается формирование детали операцией объединения ::UnionSolid. Программа берет подготовленный диск с отверстиями и «приваривает» к нему созданный патрубок (Листинг 28).

Листинг 28 – Создание патрубка для крышки

```
1:     double z_outer = is_top ? (z_inner + cap_thickness) : z_inner;
2:
3:     MbVector3D axisX(1.0, 0.0, 0.0);
4:     MbVector3D axisY(0.0, is_top ? 1.0 : -1.0, 0.0);
5:     MbCartPoint3D nozzleOrigin(0.0, hole_y, z_outer);
6:
7:     MbPlacement3D nozzlePlace(axisX, axisY, nozzleOrigin);
8:     MbSolid* nozzle = CreateNozzle(nozzlePlace,
9:         nozzle_d_in, nozzle_wall, nozzle_len,
10:        nozzle_flange_d, nozzle_flange_t,
11:        nozzle_hole_pcd, nozzle_num_holes, nozzle_hole_d,
12:        nozzle_chamfer);
13:
14:     if (nozzle) {
15:         RPArray<MbSolid> parts(2, 1);
16:         parts.Add(capSolid);
17:         parts.Add(nozzle);
18:         MbSolid* united = nullptr;
19:         MbSNameMaker snUnion(ct_BooleanSolid, MbSNameMaker::i_SideNone, 30);
20:         ::UnionSolid(parts, cm_Same, true, snUnion, false, united);
21:         if (united) capSolid = united;
22:     }
```

На Рисунок 8 представлена модель днища

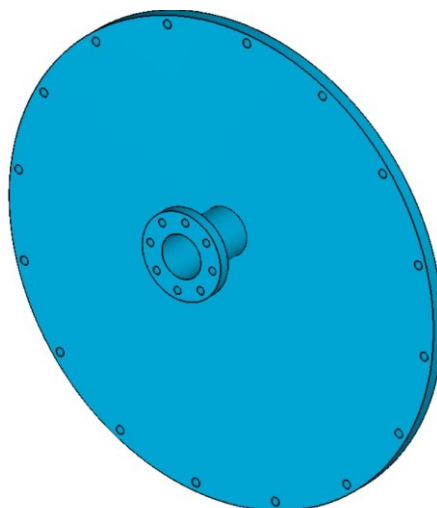


Рисунок 8 – Модель днища

4.6 Создание сборки

Заключительным этапом работы алгоритма является объединение всех созданных деталей в единую интеллектуальную модель в функции `AssembleHeatExchanger`. В отличие от простого копирования геометрии, этот процесс подразумевает создание структуры сборки, где каждая деталь занимает свое место на основе логических связей (Листинг 29). Программа инициализирует объекты `MbInstance`, которые являются ссылками на исходные твердотельные тела, и собирает их в массив для формирования объекта `MbAssembly`.

Листинг 29 – Добавление элементов в сборку

```

1:   SPtr<MbInstance> compBody(
2:       new MbInstance(*resultBody, MbPlacement3D()));
3:   SPtr<MbInstance> compGasket1(
4:       new MbInstance(*gasket1,
5:           MbPlacement3D(axX, axYp, MbCartPoint3D(0, 0, channel_width / 2))));
6:   SPtr<MbInstance> compGasket2(
7:       new MbInstance(*gasket2,
8:           MbPlacement3D(axX, axYm, MbCartPoint3D(0, 0, -channel_width / 2))));
9:   SPtr<MbInstance> compCover1(
10:      new MbInstance(*cover1,
11:          MbPlacement3D(axX, axYp, MbCartPoint3D(0, 0, L))));
12:  SPtr<MbInstance> compCover2(
13:      new MbInstance(*cover2,
14:          MbPlacement3D(axX, axYm, MbCartPoint3D(0, 0, -L))));
15:  std::vector<SPtr<MbInstance>> parts;
16:  parts.push_back(compBody);
17:  parts.push_back(compGasket1);
18:  parts.push_back(compGasket2);
19:  parts.push_back(compCover1);
20:  parts.push_back(compCover2);
  
```

Для того чтобы модель вела себя как реальное изделие, код накладывает на компоненты геометрические ограничения. Основная работа ведется с помощью системы GCM (Geometric Constraint Manager). Программа использует два типа связей:

— GCM_CONCENTRIC: обеспечивает строгую соосность цилиндрических поверхностей (например, фланца корпуса и прокладки).

GCM_DISTANCE: фиксирует детали «плоскость к плоскости» с нулевым смещением, имитируя плотное прилегание при сборке.

Для поиска граней в коде реализованы две функции. Функция FindCylindricalFace предназначена для поиска цилиндрической грани в трёхмерном твердотельном объекте MbSolid (Листинг 30). На вход функция принимает указатель на модель тела solid, значение требуемого радиуса radius, а также допустимое отклонение tol, используемое для сравнения радиусов с учётом погрешности вычислений.

Листинг 30 – Поиск цилиндрической грани по её радиусу

```
1: static MbFace* FindCylindricalFace(MbSolid* solid, double radius,
2:                                   double tol = 1.0)
3: {
4:     for (size_t i = 0; i < solid->GetFacesCount(); ++i)
5:     {
6:         MbFace* face = solid->GetFace(i);
7:         if (!face) continue;
8:         const MbSurface& surf = face->GetSurface().GetSurface();
9:         if (surf.IsA() == st_CylinderSurface)
10:        {
11:            const MbCylinderSurface& cyl =
12:                static_cast<const MbCylinderSurface&>(surf);
13:            if (fabs(cyl.GetRadius() - radius) < tol)
14:                return face;
15:        }
16:    }
17:    return nullptr;
18: }
```

Функция FindPlanarFaceAtZ предназначена для поиска плоской грани в твердотельной модели MbSolid, расположенной на заданной высоте по оси Z (Листинг 31). В качестве параметров функция принимает указатель на объект модели solid, координату z, соответствующую требуемому положению плоскости, а также значение допустимого отклонения tol.

Листинг 31 – Поиск плоской грани по координате Z

```
1: static MbFace* FindPlanarFaceAtZ(MbSolid* solid, double z, double tol = 1.0)
2: {
3:     for (size_t i = 0; i < solid->GetFacesCount(); ++i)
4:     {
5:         MbFace* face = solid->GetFace(i);
6:         if (!face) continue;
7:         const MbSurface& surf = face->GetSurface().GetSurface();
8:         if (surf.IsA() == st_Plane)
9:         {
10:            const MbPlane& plane = static_cast<const MbPlane&>(surf);
11:
12:            MbVector3D normal;
13:            plane._Normal(0.0, 0.0, normal);
14:            if (fabs(fabs(normal.z) - 1.0) > 0.01)
15:                continue;
16:
17:            MbCartPoint3D origin;
18:            plane._PointOn(0.0, 0.0, origin);
19:            if (fabs(origin.z - z) < tol)
20:                return face;
21:        }
22:    }
23:    return nullptr;
24: }
```

После определения и поиска необходимых сопрягаемых граней выполняется этап создания геометрических связей между компонентами сборки. Например, грань торца корпуса связывается с гранью прокладки, а та, в свою очередь, с внутренней плоскостью крышки (Листинг 32).

Листинг 32 – Наложение ограничений на модели

```
1: if (fBodyCyl && fG1CylNear) {
2:     MtGeomArgument argA(fBodyCyl, compBody);
3:     MtGeomArgument argB(fG1CylNear, compGasket1);
4:     assm->AddConstraint(GCM_CONCENTRIC, argA, argB);
5: }
6: if (fBodyFlatTop && fG1FlatNear) {
7:     MtGeomArgument argA(fBodyFlatTop, compBody);
8:     MtGeomArgument argB(fG1FlatNear, compGasket1);
9:     assm->AddConstraint(GCM_DISTANCE, argA, argB, 0);
10: }
11: if (fBodyCyl && fG2CylNear) {
12:     MtGeomArgument argA(fBodyCyl, compBody);
13:     MtGeomArgument argB(fG2CylNear, compGasket2);
14:     assm->AddConstraint(GCM_CONCENTRIC, argA, argB);
15: }
```

После программной расстановки всех сопряжений вызывается метод EvaluateConstraints, который заставляет систему пересчитать положение всех деталей так, чтобы ни одно правило не было нарушено. Если все условия соблюдены, сборка принимает свой финальный вид (Рисунок 9).

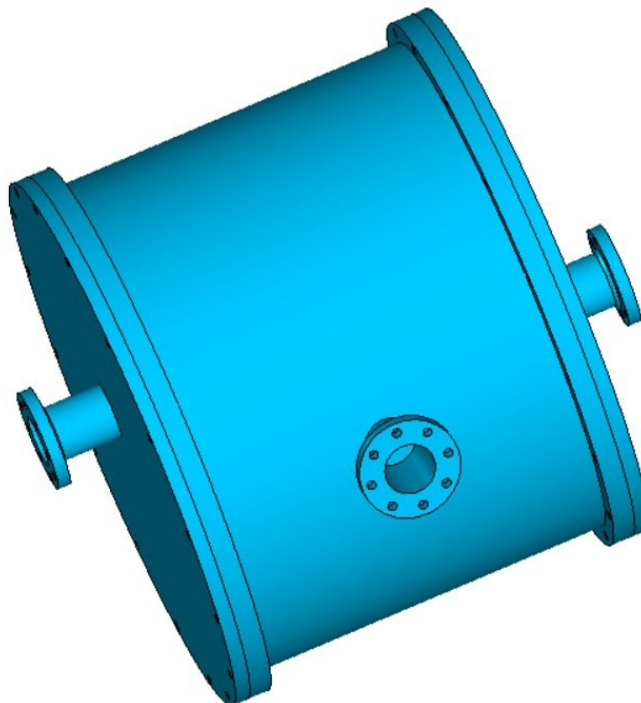


Рисунок 9 – Модель спирального теплообменника

ГЛАВА 5 СТРУКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

5.1 Алгоритм работы системы

Разработанное программное обеспечение представляет собой специализированную систему автоматизированного проектирования, предназначенную для параметрического построения спиральных теплообменных аппаратов (Рисунок 10). Архитектура системы базируется на модульном принципе, что обеспечивает разделение инженерных расчетов, геометрического моделирования и пользовательского интерфейса.

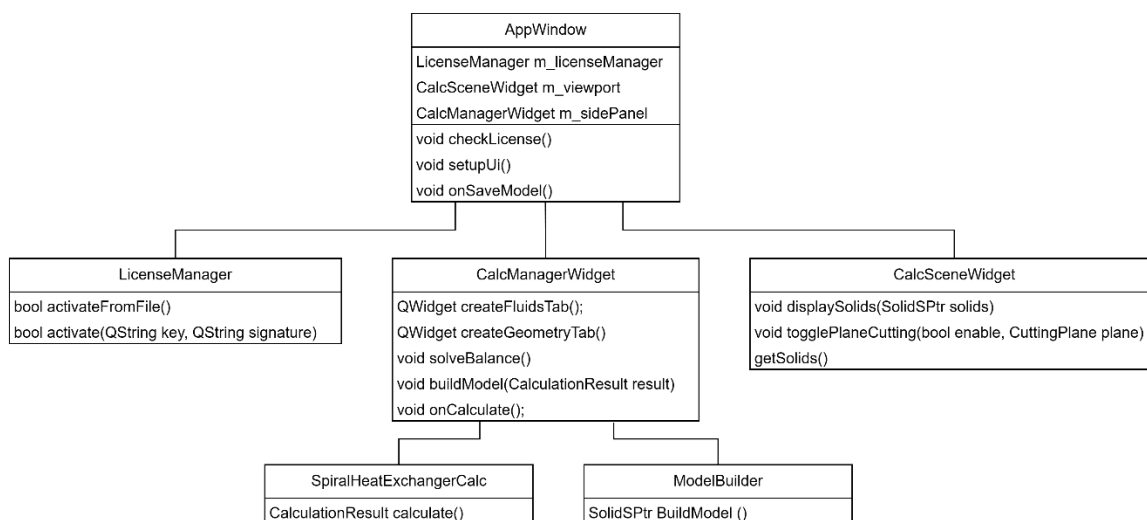


Рисунок 10 – Диаграмма классов приложения

Центральным звеном взаимодействия с пользователем выступает интерфейс **CalcManagerWidget**, где осуществляется ввод теплофизических параметров сред и геометрических ограничений будущего аппарата. На данном этапе алгоритм реализует механизм контроля валидности данных, блокируя передачу физически недостоверных значений в расчетное ядро. Важной особенностью является работа метода `solveBalance`, который автоматически выстраивает тепловой баланс системы, гарантируя математическую сходимость модели еще до начала основных вычислений.

SpiralHeatExchangerCalc выполняет тепловой и гидравлический расчет, по результатам которого проводится автоматическая проверка на соответствие заданным инженерным допускам. В случае успешного завершения расчетного цикла система переходит к этапу конструирования. Через интерфейс

FlangeConfigDialog производится подбор типоразмеров стандартных узлов, таких как фланцы, исходя из рассчитанного давления и диаметра корпуса.

Заключительный этап алгоритма связывает численные результаты с визуальным представлением. Программный модуль создает 3D-геометрии в ядре C3D, преобразуя расчетные параметры в твердотельные объекты — спирали, обечайки и патрубки. Готовая модель транслируется в модуль CalcSceneWidget, который с помощью C3D Vision визуализирует результат.

5.2 Интерфейс приложения

Интерфейс приложения представляет собой главное окно, в котором объединены средства управления, визуализации и взаимодействия с геометрическими данными. Он построен по принципу разделения на функциональные области, каждая из которых отвечает за отдельный этап работы пользователя — от формирования модели до её просмотра и экспорта.

Основу интерфейса составляет центральное окно, разделённое на две части. Одна часть предназначена для отображения трёхмерной сцены, где пользователь может наблюдать результат построения геометрических объектов. Вторая часть выполняет роль управляющей панели, через которую задаются параметры построения и формируются исходные данные для создания моделей.

В верхней части окна расположено меню, обеспечивающее доступ к основным функциям приложения. Через него выполняются операции сохранения результатов работы, а также настройка внешнего вида интерфейса, включая переключение между светлой и тёмной темами оформления (Рисунок 11).

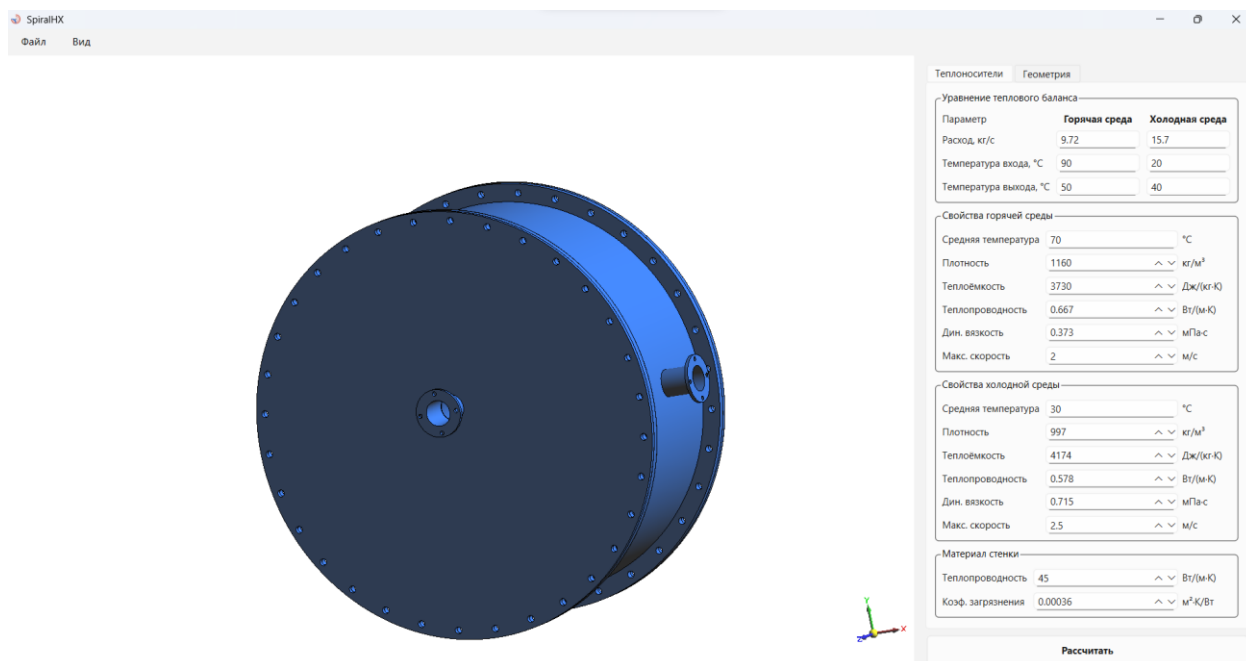


Рисунок 11 – Интерфейс приложения

5.3 Класс AppWindow

Точкой входа в программу является файл `main.cpp`, в котором инициализируются объекты `QApplication` и `QVision`. Здесь происходит базовая настройка окружения, необходимая для корректной работы высокоуровневых компонентов визуализации `C3D Vision`, и запускается главный цикл обработки событий (Листинг 33).

Листинг 33 — Код файла `main.cpp`

```

1:  #include <QApplication>
2:  #include <QWindow>
3:  #include "AppWindow.h"
4:  #include <vsn_application.h>
5:  int main(int argc, char* argv[])
6:  {
7:      QApplication app(argc, argv);
8:      VSN::Application vapp;
9:      AppWindow win;
10:     win.show();
11:     return app.exec();
12: }
```

Основная задача класса `AppWindow` — координация потоков данных. Класс обеспечивает передачу расчетных параметров из расчетного модуля в геометрическое ядро и последующее отображение полученных твердотельных моделей на сцене. Настройка главного окна происходит в методе `setUpUi`, где формируется компоновка элементов с использованием `QSplitter`, что позволяет

пользователю гибко настраивать размеры рабочей области и панели параметров (Листинг 34).

Листинг 34 — Создание рабочих виджетов

```
1: auto central = new QWidget(this);
2: setCentralWidget(central);
3:
4: m_splitter = new QSplitter(Qt::Horizontal, central);
5:
6: m_viewport = new CalcSceneWidget(m_splitter);
7: m_sidePanel = new CalcManagerWidget(m_splitter);
```

Взаимодействие между классами осуществляется через систему сигналов и слотов. Система сигналов и слотов в Qt — это реализация паттерна «Наблюдатель», предназначенная для безопасного и гибкого взаимодействия между объектами [23] (Листинг 35).

Листинг 35 — Создание сигналов

```
1: connect(m_sidePanel, &CalcManagerWidget::solidReady,
2:         this, &AppWindow::onSolidsReady);
```

Метод onSaveModel отвечает за экспорт сгенерированной сборки в файл одного из поддерживаемых форматов. Работа метода начинается с проверки наличия геометрии: программа обращается к классу сцены и получает указатель на текущую сборку MbAssembly. Если сборка не была построена, пользователю выводится предупреждение, и выполнение функции прерывается.

Листинг 36 — Получение ссылки на модель

```
1: SPtr<MbAssembly> assembly = m_viewport->getAssembly();
2:
3: if (!assembly) {
4:     QMessageBox::warning(this, "Сохранение", "Нет модели для
5:                                             сохранения.");
6:     return;
7: }
```

Далее формируется список доступных расширений, который охватывает как родной формат ядра (C3D), так и универсальные инженерные форматы, такие как STEP, PARASOLID и STL (Листинг 37). С помощью стандартного диалогового окна QFileDialog программа запрашивает у пользователя путь и имя файла. Чтобы визуально обозначить процесс обработки данных, на время экспорта курсор приложения меняется на режим ожидания (WaitCursor).

Листинг 37 — Выбор формата

```
1:   QStringList filters;
2:       filters
3:           << "File C3D (*.c3d)"
4:           << "File OBJ (*.obj)"
5:           << "File ACISC (*.sat)"
6:           << "File STEP (*.step *.stp)"
7:           << "File PARASOLID (*.x_b)"
8:           << "File JT (*.jt *.JT)"
9:           << "File STL (*.stl *.STL)";
10:
11:   QString fileName = QFileDialog::getSaveFileName(
12:       this,
13:       "Сохранить модель как...",
14:       "",
15:       filters.join(";;")
16:   );
17:
18:   if (fileName.isEmpty())
19:       return;
20:
21:   QApplication::setOverrideCursor(Qt::WaitCursor);
22:
23:   MbModel model;
24:   model.AddItem(*assembly);
25:
26:   MbeConvResType result = c3d::ExportIntoFile(
27:       model,
28:       c3d::WToPathstring(fileName.toStdWString())
29:   );
```

Завершается метод проверкой результата операции. Если экспорт прошел успешно (cnv_Success), выводится информационное сообщение (Листинг 38). В случае возникновения проблем (например, из-за отсутствия прав доступа к папке или ошибок конвертации) код выводит критическое окно с числовым кодом ошибки C3D.

Листинг 38 — Вывод результата

```
1:   if (result == cnv_Success) {
2:       QMessageBox::information(this, "Успех", "Файл успешно сохранен.");
3:   }
4:   else {
5:       QMessageBox::critical(
6:           this,
7:           "Ошибка",
8:           QString("Не удалось сохранить файл. Код ошибки C3D: %1")
9:               .arg(static_cast<int>(result))
10:       );
11:   }
```

5.4 Класс LicenseManager

Модуль LicenseManager отвечает за активацию математических библиотек. Архитектура класса спроектирована таким образом, чтобы минимизировать необходимость повторного ввода ключей пользователем. Для этого в модуле реализован механизм сохранения учетных данных в локальном хранилище операционной системы. Метод filePath() использует средства QStandardPaths, что обеспечивает кроссплатформенность приложения и позволяет хранить лицензионный файл в домашней директории пользователя.

Процесс активации реализован через двухступенчатую проверку. На первом этапе метод activateFromFile() пытается автоматически считать ключ и сигнатуру из сохраненного файла. Если файл отсутствует или данные недействительны, управление передается пользовательскому интерфейсу. Основная логика инициализации сосредоточена в методе activate(), где происходит вызов глобальных функций ядра EnableMathModules (Листинг 39).

Листинг 39 — Метод activate

```
1: LicenseManager::Result LicenseManager::activate(const QString& key,  
2:                                                  const QString& signature)  
3:     {  
4:  
5:         QByteArray keyB = key.toUtf8();  
6:         QByteArray sigB = signature.toUtf8();  
7:  
8:         EnableMathModules(  
9:             keyB.constData(),  
10:            keyB.size(),  
11:            sigB.constData(),  
12:            sigB.size()  
13:        );  
14:  
15:        m_active = IsMathVisionEnable();  
16:  
17:        if (m_active)  
18:            saveToFile(key, signature);  
19:  
20:        return m_active  
21:            ? Result::Success  
22:            : Result::Invalid;  
23:    }
```

При запуске приложения AppWindow делегирует проверку лицензии менеджеру, который, в свою очередь, может вызвать диалоговое окно LicenseDialog для ручного ввода данных (

Листинг 40). Только после получения статуса Success управление возвращается к методам настройки графической сцены и построения интерфейса. Таким образом гарантируется активация функция геометрического ядра C3D до запуска приложения.

Листинг 40 — Метод checkLicense

```
1: void AppWindow::checkLicense()
2: {
3:     if (m_licenseManager.activateFromFile())
4:     {
5:         return;
6:     }
7:
8:     while (true)
9:     {
10:        LicenseDialog dlg(this);
11:
12:        if (dlg.exec() != QDialog::Accepted)
13:        {
14:            QApplication::quit();
15:            return;
16:        }
17:
18:        auto result = m_licenseManager.activate(
19:            dlg.key(),
20:            dlg.signature()
21:        );
22:
23:        if (result == LicenseManager::Result::Success)
24:            return;
25:
26:        QMessageBox::warning(this, "Активация лицензии",
27:                               "Ошибка активации лицензии");
28:    }
29: }
```

5.5 Класс CalcManagerWidget

Класс CalcManagerWidget отвечает за сбор пользовательских данных и инициализацию процесса геометрического построения. Основная задача этого

модуля — преобразовать численные значения из полей ввода в набор твердых тел C3D, готовых к визуализации и экспорту.

Интерфейс класса CalcManagerWidget организован по модульному принципу с использованием виджета вкладок QTabWidget что позволяет логически разделить огромный массив инженерных параметров на управляемые блоки (Рисунок 12 – Интерфейс класса CalcManagerWidget). Каждая вкладка фокусируется на конкретном аспекте проектирования теплообменника, например геометрические размеры корпуса параметры спиральных листов или конфигурация входных патрубков.

Теплоносители Геометрия

Уравнение теплового баланса

Параметр	Горячая среда	Холодная среда
Расход, кг/с	9.72	15.7
Температура входа, °C	90	20
Температура выхода, °C	50	40

Свойства горячей среды

Средняя температура	70	°C
Плотность	1160	кг/м³
Теплоёмкость	3730	Дж/(кг·K)
Теплопроводность	0.667	Вт/(м·K)
Дин. вязкость	0.373	мПа·с
Макс. скорость	2	м/с

Свойства холодной среды

Средняя температура	30	°C
Плотность	997	кг/м³
Теплоёмкость	4174	Дж/(кг·K)
Теплопроводность	0.578	Вт/(м·K)
Дин. вязкость	0.715	мПа·с
Макс. скорость	2.5	м/с

Материал стенки

Теплопроводность	45	Вт/(м·K)
Козф. загрязнения	0.00036	м²·K/Вт

Рассчитать

Рисунок 12 – Интерфейс класса CalcManagerWidget

Для унификации внешнего вида и упрощения кода при создании вкладок применяется вспомогательная функция `addRowWithUnit` (Листинг 41). Эта функция инкапсулирует в себе создание горизонтального слоя `QHBoxLayout` в который помещается текстовая метка поле ввода и единица измерения.

Например, для ввода толщины листа функция автоматически создает подпись SpinBox с нужным диапазоном и текстовую метку мм в конце строки.

Листинг 41 – Метод addRowWithUnit

```
1: void CalcManagerWidget::addRowWithUnit(QFormLayout* form, const QString& name,
2:     QDoubleSpinBox* spin, const QString& unit)
3: {
4:     auto container = new QWidget;
5:     auto hLayout = new QHBoxLayout(container);
6:     hLayout->addWidget(spin);
7:     hLayout->addWidget(new QLabel(unit));
8:     hLayout->setContentsMargins(0, 0, 0, 0);
9:     hLayout->setSpacing(4);
10:
11:     form->addRow(name, container);
12: }
```

Использование QDoubleSpinBox вместо обычных текстовых полей гарантирует что в систему попадут только корректные числовые данные (Листинг 42).

Листинг 42 – Пример создания полей

```
1: auto makeSpinBox = []() {
2:     auto s = new CleanDoubleSpinBox;
3:     s->setRange(0.0, 1e9);
4:     s->setDecimals(6);
5:     s->setButtonSymbols(QAbstractSpinBox::NoButtons);
6:     return s;
7: };
8:
9: hot.rho = makeSpinBox();
10: hot.cp = makeSpinBox();
11: hot.lambda = makeSpinBox();
12: hot.viscosity = makeSpinBox();
13: hot.max_velocity = makeSpinBox();
14:
15: addRowWithUnit(hotForm, "Средняя температура", hot.t_avg, "°C");
16: addRowWithUnit(hotForm, "Плотность", hot.rho, "кг/м³");
17: addRowWithUnit(hotForm, "Теплоёмкость", hot.cp, "Дж/(кг·K)");
18: addRowWithUnit(hotForm, "Теплопроводность", hot.lambda, "Вт/(м·K)");
19: addRowWithUnit(hotForm, "Динамическая вязкость", hot.viscosity, "мПа·с");
20: addRowWithUnit(hotForm, "Макс. скорость", hot.max_velocity, "м/с");
```

В классе реализован метод solveBalance, который отвечает за автоматический расчет неизвестного параметра в уравнении теплового баланса (Листинг 43). После заполнения данных первого блока автоматически рассчитывается средняя температура сред, на основе которой пользователь заполняет остальные свойства теплоносителей.

Листинг 43 – Метод solveBalance

```

1: void CalcManagerWidget::solveBalance()
2: {
3:     if (G1 > 0 && cp1 > 0 && T1i != 0 && T1o != 0)
4:     {
5:         double Q = G1 * cp1 * (T1i - T1o);
6:
7:         if (T2o == 0 && G2 > 0 && cp2 > 0)
8:         {
9:             cold.t_out->blockSignals(true);
10:            cold.t_out->setValue(Q / (G2 * cp2) + T2i);
11:            cold.t_out->blockSignals(false);
12:        }
13:        else if (T2i == 0 && G2 > 0 && cp2 > 0)
14:        {
15:            cold.t_in->blockSignals(true);
16:            cold.t_in->setValue(T2o - Q / (G2 * cp2));
17:            cold.t_in->blockSignals(false);
18:        }
19:        else if (G2 == 0 && cp2 > 0 && T2o != 0 && T2i != 0)
20:        {
21:            cold.G->blockSignals(true);
22:            cold.G->setValue(Q / (cp2 * (T2o - T2i)));
23:            cold.G->blockSignals(false);
24:        }
25:    }
26:    else if (G2 > 0 && cp2 > 0 && T2i != 0 && T2o != 0)
27:    {
28:        double Q = G2 * cp2 * (T2o - T2i);
29:
30:        if (T1o == 0 && G1 > 0 && cp1 > 0)
31:        {
32:            hot.t_out->blockSignals(true);
33:            hot.t_out->setValue(T1i - Q / (G1 * cp1));
34:            hot.t_out->blockSignals(false);
35:        }
36:        else if (T1i == 0 && G1 > 0 && cp1 > 0)
37:        {
38:            hot.t_in->blockSignals(true);
39:            hot.t_in->setValue(T1o + Q / (G1 * cp1));
40:            hot.t_in->blockSignals(false);
41:        }
42:        else if (G1 == 0 && cp1 > 0 && T1i != 0 && T1o != 0)
43:        {
44:            hot.G->blockSignals(true);
45:            hot.G->setValue(Q / (cp1 * (T1i - T1o)));
46:            hot.G->blockSignals(false);
47:        }
48:    }
49: }

```

После нажатия на кнопку «Рассчитать» все данные преобразуются в систему СИ и отправляются в структуры для расчёта теплообменника (Листинг 44).

Листинг 44 – Пример инкапсуляции данных

```
1: Fluid CalcManagerWidget::toFluid(const FluidUI& ui)
2: {
3:     Fluid f;
4:     f.G = ui.G->value();
5:     f.t_in = ui.t_in->value();
6:     f.t_out = ui.t_out->value();
7:     f.rho = ui.rho->value();
8:     f.cp = ui.cp->value();
9:     f.lambda = ui.lambda->value();
10:    f.viscosity = ui.viscosity->value() / 1000.0;
11:    f.max_velocity = ui.max_velocity->value();
12:    return f;
13: }
```

Для расчёта спирального теплообменника используется класс `SpiralHeatExchanger`, в котором реализован алгоритм, описанный в главе 3 (Листинг 45).

Листинг 45 – Запуск расчёта спирального теплообменника

```
1: SpiralHeatExchanger hx(hotF, coldF, wallM, geomM);
2:
3: CalculationResult result = hx.calculate();
4: if (!result.error_message.empty()) {
5:     qDebug() << QString::fromStdString(result.error_message);
6:     return;
7: }
```

После расчёта вызывается диалоговое окно `ResultDialog`, в котором подробно описаны результаты расчёта (Листинг 46).

Листинг 46 – создание диалогового окна `ResultDialog`

```
1: auto w = new ResultDialog({ result });
2: w->setWindowTitle("Результат расчета");
3: w->resize(700, 400);
4: w->show();
5:
6: connect(w, &ResultDialog::selectedResultConfirmed, this, [=](const
CalculationResult& r) {
7:     buildModel(r);
8:     w->close();
9: });
```


Класс ResultDialog предназначен для отображения результатов теплогидравлического расчёта спирального теплообменника, а также для последующей настройки параметров конструкции перед построением трёхмерной модели (Рисунок 13).

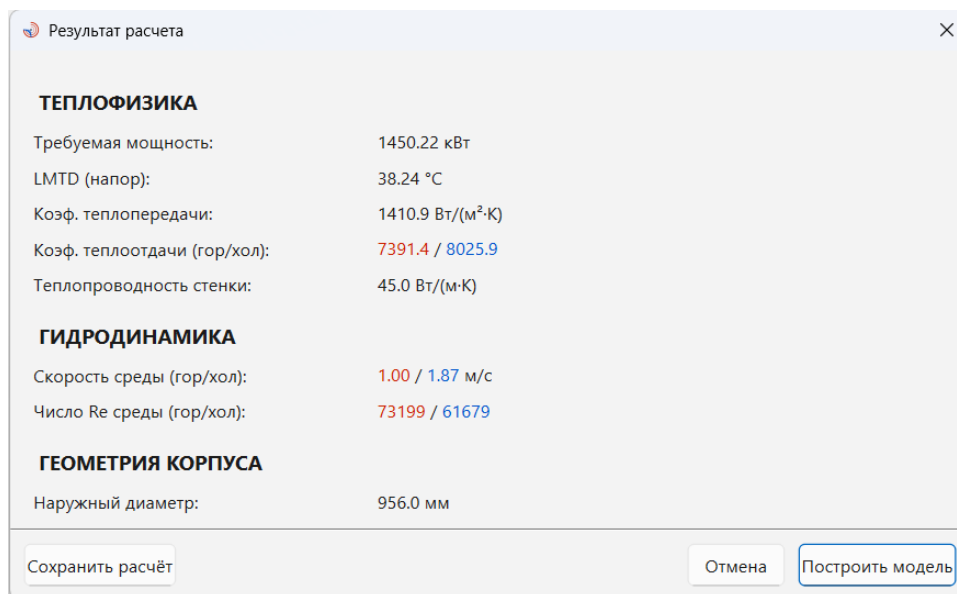


Рисунок 13 – Интерфейс класса ResultDialog

После создания объекта диалога выполняется инициализация пользовательского интерфейса, автоматический подбор фланцев и формирование элементов отображения.

Для сокращения дублирования кода в классе реализован вспомогательный метод addRow, предназначенный для автоматизированного формирования строк интерфейса (Листинг 47).

Листинг 47 – Создание строк

```

1: void ResultDialog::addRow(QFormLayout* layout, const QString& label,
2:   const QString& value, bool isHeader)
3:   {
4:       if (isHeader) {
5:           auto* lbl = new QLabel(label);
6:           lbl->setStyleSheet(
7:               "font-weight: bold; font-size: 11pt; "
8:               "margin-top: 10px; border-bottom: 2px solid palette(mid);");
9:           layout->addRow(lbl);
10:      }
11:      else {
12:          auto* val = new QLabel(value);
13:          val->setStyleSheet("font-weight: 500;");
14:          layout->addRow(new QLabel(label), val);
15:      }
16:  }

```

Для обеспечения корректного подбора элементов в программный модуль встроена база стандартных фланцевых соединений, сформированная на основе требований ГОСТ 33259–2015 для исполнения 1. База содержит типоразмеры фланцев в диапазоне условных проходов от DN 80 до DN 2200.

Класс ResultDialog содержит механизм автоматической проверки корректности рассчитанной конструкции. Для этого реализован метод buildWarnings(), выполняющий анализ геометрических и гидродинамических параметров.

Проверяются следующие условия:

- достаточность внутреннего диаметра спирали;
- соответствие размеров патрубков;
- корректность числа Рейнольдса;
- минимальное количество витков;
- допустимый наружный диаметр аппарата.

При обнаружении несоответствий пользователю выводятся предупреждающие сообщения (Рисунок 14).

Обнаружены несоответствия:

- Внутренний диаметр спирали (300.0 мм) меньше удвоенного расчётного диаметра входного отверстия ($187.1 \text{ мм} \times 2 = 374.2 \text{ мм}$). Рекомендуется увеличить внутренний диаметр спирали.

Рисунок 14 – Уведомления об ошибках

Класс поддерживает экспорт результатов расчёта в текстовый файл и PDF-документ. Для этого используется метод onSaveCalc(), в котором вызываются функции сериализации класса CalcResultSerializer (Листинг 48).

Листинг 48 – Сохранение расчёта

```
1:   QString selectedFilter;
2:   QString path = QFileDialog::getSaveFileName(
3:       this, "Сохранить расчёт", {},
4:       "Файл расчёта SpiralHX (*.txt);;"
5:       "PDF документ (*.pdf)",
6:       &selectedFilter);
7:   if (path.isEmpty()) return;
8:   QString err;
9:   if (selectedFilter.contains("*.pdf")) {
10:       if (!path.endsWith(".pdf", Qt::CaseInsensitive)) path += ".pdf";
11:       err = CalcResultSerializer::savePdf(snapshot, path);
12:   }
13:   else {
14:       if (!path.endsWith(".txt", Qt::CaseInsensitive)) path += ".txt";
15:       err = CalcResultSerializer::save(snapshot, path);
```

После сохранения пользователю выводится информационное сообщение об успешном завершении операции.

После выбора полной комбинации параметров активируется кнопка построения модели. При её нажатии выбранные параметры передаются в основной модуль программы для дальнейшего трёхмерного моделирования конструкции.

Для создания модели используется класс `ModelBuilder`, в котором реализован алгоритм, описанный в главе 4 (Листинг 49).

Листинг 49 – Метод `buildModel`

```
1: void CalcManagerWidget::buildModel(const CalculationResult& result)
2: {
3:     ModelBuilder builder;
4:     auto solids = builder.BuildModel(result);
5:
6:     emit solidReady(solids);
7: }
```

5.6 Класс `CalcSceneWidget`

Класс `CalcSceneWidget` является визуальным ядром приложения, представляя собой специализированный виджет для отображения трехмерной геометрии теплообменника. Он выступает в роли связующего звена между математическими объектами ядра C3D и графической подсистемой (OpenGL), обеспечивая взаимодействие пользователя с моделью.

За создание сцены отвечает функция `createScene`, которая инициализирует графическую среду (Листинг 50). Внутри этой функции происходит настройка параметров освещения, выбор фонового цвета и установка свойств материала для отображаемых объектов. Здесь же задаются параметры перспективы и начальное положение камеры, чтобы при запуске приложения пользователь видел корректную сетку координат.

Листинг 50 – Метод initScene

```
1: void CalcSceneWidget::initScene()
2: {
3:     setMouseTracking(true);
4:
5:     m_scene = new VSN::QtVision::QtOpenGLSceneWidget(this);
6:     m_scene->setMouseTracking(true);
7:     m_scene->setFocusPolicy(Qt::StrongFocus);
8:
9:     m_scene->viewport()->SetVisibleSceneWidgets(true);
10:    m_scene->viewport()->SetVisibleViewWidgets(true);
11:
12:    layout()->addWidget(m_scene);
13: }
```

Метод displaySolid является основным механизмом вывода геометрии на экран. Он связывает абстрактное твердое тело ядра C3D с визуальным движком приложения. Его работа заключается в преобразовании математического описания объекта в графическое представление (Листинг 51).

Логика работы метода базируется на последовательной трансформации данных из математического формата ядра в графическое представление вида. Сначала выполняется обязательная проверка входных указателей и полная очистка сцены через clearScene для исключения наложения новой модели на старую итерацию. Затем через фабрику геометрии иницируется процесс триангуляции, где твердое тело преобразуется в полигональную сетку с заданной точностью прогиба 0.1 мм, что обеспечивает необходимый баланс между детализацией спирали и производительностью системы. Полученная репрезентация упаковывается в новый сегмент сцены SceneSegment, который связывается с корневым узлом иерархии объектов. На финальном этапе метод принудительно создает рендер-объекты для видеокарты, устанавливает флаг видимости и иницирует перерисовку виджета командой update, благодаря чему пользователь мгновенно видит актуальный результат геометрического построения.

Листинг 51 – Метод displaySolid

```
1: void CalcSceneWidget::displaySolid(const SolidSPtr& solid)
2: {
3:     if (!m_scene || !solid) return;
4:
5:     clearScene();
6:
7:     auto root = m_scene->sceneContent()->GetRootSegment();
8:
9:     auto rep = GeometryFactory::Instance()->CreateMathRep(
10:         solid,
11:         CommandType::Synchronous,
12:         0.1
13:     );
14:
15:     if (!rep) return;
16:
17:     m_currentSegment = new SceneSegment(rep, root);
18:     root->AddSegment(m_currentSegment);
19:
20:     m_currentSegment->CreateRenderObject();
21:     m_currentSegment->SetVisible(true);
22:
23:     m_scene->update();
24: }
```

В классе CalcSceneWidget взаимодействие с трехмерной моделью реализовано через переопределение стандартных обработчиков событий мыши Qt.

Метод mousePressEvent отвечает за фиксацию начальной точки взаимодействия и определение режима навигации (Листинг 52). При нажатии кнопки мыши программа сохраняет текущие координаты курсора и проверяет, какая именно клавиша была задействована. Если нажата левая кнопка, система готовится к вращению модели, если правая или средняя — к перемещению. Этот обработчик служит триггером, который «захватывает» фокус ввода, позволяя последующим движениям мыши корректно интерпретироваться в контексте трехмерного пространства.

Листинг 52 – метод mousePressEvent

```
1: void CalcSceneWidget::mousePressEvent(QMouseEvent* event)
2: {
3:     m_lastMousePos = event->pos();
4:     m_firstMouse = false;
5:
6:     if (event->button() == Qt::RightButton) {
7:         QPointF p = event->position();
8:         start_position = QPointF(p.x() / width() - 0.5, p.y() / height() -
9:                                 0.5);
10:    }
11:
12:    m_scene->update();
13: }
```

Основная логика трансформации вида сосредоточена в методе `mouseMoveEvent`, который вычисляет дельту — разницу между текущим и предыдущим положением курсора (Листинг 53). Если зажата левая кнопка мыши, эта разница преобразуется в углы поворота камеры вокруг объекта, позволяя инженеру осмотреть модель под любым углом. Прижатой правой кнопке перемещение курсора трансформируется в линейный сдвиг плоскости обзора, что удобно для выравнивания модели по центру экрана. Метод постоянно вызывает обновление сцены, обеспечивая плавность и высокую скорость отклика визуализации на действия пользователя.

Листинг 53 – Метод mouseMoveEvent

```
1: void CalcSceneWidget::mouseMoveEvent(QMouseEvent* event)
2: {
3:     if (!m_scene || !(event->buttons() & (Qt::RightButton |
Qt::MiddleButton))) {
4:         m_firstMouse = true;
5:         return;
6:     }
7:
8:     auto cam = m_scene->camera();
9:     if (!cam) return;
10:
11:     if (m_firstMouse) {
12:         m_lastMousePos = event->pos();
13:         m_firstMouse = false;
14:         return;
15:     }
16:
17:     float dx = event->pos().x() - m_lastMousePos.x();
18:     float dy = m_lastMousePos.y() - event->pos().y();
19: }
```

```

20:     m_lastMousePos = event->pos();
21:
22:     if (event->buttons() & Qt::RightButton) {
23:         dx = dx / width() * rotate_sensitivity;
24:         dy = dy / height() * rotate_sensitivity;
25:
26:         cam->RotateAboutTarget(cam->GetUpVector(), -dx);
27:         cam->RotateAboutTarget(cam->GetSideVector(), dy);
28:     }
29:
30:     if (event->buttons() & Qt::MiddleButton) {
31:         float dist = cam->DistanceCameraToTarget();
32:         cam->Pan(Vector3DF(-dx * move_sensitivity * dist,
33:             -dy * move_sensitivity * dist,
34:             0));
35:     }
36:
37:     m_scene->update();
38: }

```

Продолжение Листинг 53

За управление масштабом отвечает метод `wheelEvent`, который обрабатывает вращение колесика мыши (Листинг 54). Каждое деление прокрутки генерирует коэффициент масштабирования, приближая или удаляя камеру относительно точки фокуса.

Листинг 54 – Метод `wheelEvent`

```

1: void CalcSceneWidget::wheelEvent(QWheelEvent* event)
2: {
3:     if (!m_scene) return;
4:     auto cam = m_scene->camera();
5:     if (!cam) return;
6:
7:     float scale = std::pow(2.0f, event->angleDelta().y() * scale_sensitivity);
8:     float dist = cam->DistanceCameraToTarget();
9:
10:    cam->SetZoom(scale);
11:    cam->Pan(Vector3DF((event->position().x() / width() - 0.5f) * (scale -
12:    1.0f) * dist,
13:        -(event->position().y() / height() - 0.5f) * (scale - 1.0f) * dist,
14:        0));
15:    m_scene->update();
16: }
17:

```

ГЛАВА 6 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

Целью тестирования являлась проверка корректности работы, разработанной мини-САПР для проектирования спиральных теплообменных аппаратов, а также подтверждение соответствия реализованного программного обеспечения требованиям, сформулированным в выпускной квалификационной работе. В ходе испытаний проверялись работоспособность расчетных алгоритмов, корректность построения трехмерной модели, устойчивость работы интерфейса и корректность взаимодействия с библиотекой C3D Toolkit.

Для проведения испытаний была составлена таблица методик тестирования, включающая сценарии проверки пользовательского интерфейса, корректности ввода данных и функциональных возможностей программного комплекса. Проверка выполнялась вручную путем последовательного выполнения тестовых сценариев (Таблица 1).

Процесс тестирования был разделен на три основных этапа:

1. Визуальная проверка интерфейса;
2. Проверка корректности обработки входных данных;
3. Проверка функциональных возможностей системы.

Таблица 1 – Методики проведения испытаний

№	Метод	Инструкция	Ожидаемый результат
1 Визуальная проверка			
1.1	Проверка русскоязычного интерфейса	Выполнить визуальную проверку элементов интерфейса приложения	Все элементы интерфейса отображаются на русском языке
1.2	Проверка взаимодействия с клавиатурой и мышью	Проверить работу элементов интерфейса с использованием клавиатуры и мыши	Интерфейс корректно реагирует на действия пользователя
1.3	Проверка расположения элементов интерфейса	Выполнить проверку отображения элементов при разрешении 1920×1080	Все элементы интерфейса располагаются в пределах рабочей области
1.4	Проверка единого стиля приложения	Открыть все вкладки приложения и проверить оформление компонентов	Компоненты интерфейса выполнены в едином стиле
1.5	Проверка отображения трехмерной сцены	Построить модель теплообменника и проверить отображение сцены	Модель корректно отображается в окне визуализации

Продолжение Таблица 1

2 Проверка ввода данных			
2.1	Ввод корректных параметров теплоносителей	Ввести допустимые значения расходов, температур и свойств среды	Система принимает данные и выполняет расчет
2.2	Ввод отрицательных значений	Попытаться ввести отрицательные значения расхода или температуры	Система не допускает ввод некорректных значений
2.3	Ввод текстовых данных в числовые поля	Попытаться ввести текст в поля QDoubleSpinBox	Система блокирует ввод недопустимых символов
2.4	Проверка автоматического расчета теплового баланса	Оставить один из параметров теплового баланса пустым	Система автоматически вычисляет неизвестный параметр
2.5	Проверка диапазонов значений	Ввести значения, превышающие допустимый диапазон	Система ограничивает ввод значений
2.6	Проверка обработки некорректного режима расчета	Ввести параметры, при которых невозможно обеспечить тепловой баланс	Система выводит сообщение об ошибке
3 Проверка функциональных возможностей			
3.1	Расчет теплообменника	Ввести корректные исходные данные и нажать кнопку «Рассчитать»	Выполняется тепловой и гидравлический расчет
3.2	Автоматический подбор геометрии	Выполнить расчет аппарата	Система подбирает допустимую геометрию из стандартного ряда
3.3	Построение трехмерной модели	После расчета запустить построение модели	Формируется корректная 3D-модель теплообменника
3.4	Экспорт модели	Выполнить сохранение модели	Файл успешно сохраняется и открывается в сторонних CAD-системах
3.5	Проверка отображения результатов расчета	Открыть окно результатов расчета	Все расчетные параметры отображаются корректно
3.6	Проверка выбора фланцев	Выполнить подбор фланцев после расчета	Система отображает допустимые стандартные варианты
3.7	Проверка работы лицензирования	Перезапустить приложение после активации лицензии	Лицензия автоматически загружается из локального файла
3.8	Проверка устойчивости работы приложения	Многokrатно выполнять расчеты и построение моделей	Приложение работает без аварийного завершения

При тестировании механизма лицензирования было обнаружено, что после ввода некорректного ключа окно активации закрывалось без повторного запроса. После исправления реализована повторная проверка и вывод сообщения об ошибке активации.

В процессе проверки автоматического расчёта теплового баланса обнаружилась проблема, при которой после изменения одного параметра не обновлялись автоматически связанные поля. Ошибка была устранена путём корректировки обработки сигналов `valueChanged`.

При построении трёхмерной модели была выявлена ошибка позиционирования патрубков: в отдельных случаях патрубки частично пересекались с корпусом аппарата. После корректировки координат и параметров смещения геометрия модели стала отображаться корректно.

Также была обнаружена проблема, связанная с повторным построением модели. При многократном нажатии кнопки расчёта предыдущая модель не удалялась со сцены, что приводило к наложению объектов друг на друга. Ошибка была исправлена добавлением очистки сцены перед генерацией новой модели.

ГЛАВА 7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

Технико-экономическое обоснование проекта (ТЭО) является инструментом анализа программного продукта, предназначенным для оценки затрат на его разработку, а также определения ожидаемой выгоды и эффективности для заказчика. При анализе изменений экономических показателей программного обеспечения следует учитывать следующие критерии:

- 1) общая трудоемкость разработки приложения;
- 2) себестоимость программного обеспечения;
- 3) экономический эффект от применения программного продукта.

7.1 Расчет трудоемкости разработки программного продукта

Общая трудоёмкость разработки программного продукта определяется по формуле:

$$T_{\text{общ}} = t_{\text{ТЗ}} + t_{\text{ЭП}} + t_{\text{ТП}} + t_{\text{РП}} + t_{\text{В}}, \quad (18)$$

где:

$t_{\text{ТЗ}}$ – затраты труда на стадии технического задания (в днях);

$t_{\text{ЭП}}$ – затраты труда на стадии эскизного проекта (в днях);

$t_{\text{ТП}}$ – затраты труда на стадии технического проекта (в днях);

$t_{\text{РП}}$ – затраты труда на стадии рабочего проекта (в днях);

$t_{\text{В}}$ – затраты труда на стадии внедрения (в днях).

Разрабатываемое программное обеспечение относится к категории расчётных систем и предназначено для автоматизации проектирования спирального теплообменного аппарата.

Степень новизны программного решения соответствует категории «В», так как разрабатываемая система имеет существующие аналоги.

Сложность алгоритмов относится к первой категории.

В разрабатываемом программном обеспечении используются:

- переменная информация (ПИ);
- нормативно-справочная информация (НСИ).

Количество наборов данных:

- $m = 2$ — количество наборов ПИ;
- $n = 1$ — количество наборов НСИ;

Сложность контроля входной информации принимается как 12 — входные данные однообразной формы и содержания.

Сложность контроля выходной информации принимается как 21 — вывод документов сложной многоуровневой структуры.

Трудоёмкость разработки по этапам без учёта поправочных коэффициентов составляет:

- техническое задание — 7 дней;
- эскизный проект — 11 дней;
- технический проект — 16 дней;
- рабочий проект — 38 дней;
- внедрение — 5 дней.

Подставив значения в формулу, получим:

$$T_{\text{общ}} = 7 + 11 + 16 + 38 + 5 = 77 \text{ дней}$$

Для стадий технического и рабочего проекта учитываются поправочные коэффициенты, зависящие от степени новизны и сложности алгоритмов.

С учётом поправочных коэффициентов формула принимает вид:

$$T' = t_{\text{ТЗ}} + t_{\text{ЭП}} + t'_{\text{ТП}} + t'_{\text{РП}} + t'_B \quad (19)$$

Поправочный коэффициент на использование различных видов информации рассчитывается по формуле:

$$K_{\Pi} = \frac{K_1 * m + K_2 * n + K_3 * p}{m + n + p}$$

Поправочный коэффициент для стадии технического проекта:

$$K_{\Pi}^{\text{ТП}} = \frac{1 * 2 + 0,72 * 1 + 2,08 * 0}{2 + 1 + 0} = 0,91 \quad (20)$$

Затраты труда на стадии технического проекта с учётом поправки:

$$t'_{\text{ТП}} = 14 * 1,26 * 0,91 = 18 \text{ дней}$$

Поправочный коэффициент для стадии рабочего проекта:

$$K_{\Pi}^{\text{РП}} = \frac{1,2 * 2 + 0,65 * 1 + 0,54 * 0}{2 + 1 + 0} = 1,02$$

Затраты труда на стадии рабочего проекта:

$$t'_{\text{РП}} = 38 * 1,36 * 1,02 * 1,08 = 57 \text{ дней}$$

Затраты труда на стадии внедрения:

$$t'_B = 5 * 1,16 * 1,21 = 8 \text{ дней}$$

Общие затраты труда на разработку программного продукта с учётом поправочных коэффициентов составят:

$$T'_{\text{ОБЩ}} = 7 + 11 + 18 + 57 + 7 = 100 \text{ дней}$$

Количество разработчиков определяется по формуле:

$$U = \frac{T_{\text{ОБЩ}}}{T'_{\text{ОБЩ}}}, \quad (21)$$

Затем следует подставить полученные ранее значения в формулу.

$$U = \frac{77}{100} \approx 1 \text{ человек}$$

Следовательно, для выполнения поставленной задачи достаточно одного разработчика.

7.2 Расчёт себестоимости программного продукта

Себестоимость программного продукта рассчитывается по формуле:

$$C = M_{\text{ВС}} + Э + 3_{\text{ЗП}}^0 + 3_{\text{ЗД}}^{\text{Д}} + 3_{\text{СН}} + Н, \quad (22)$$

где:

$M_{\text{ВС}}$ – затраты на вспомогательные материалы, р.;

$Э$ – затраты на электроэнергию на технологические цели, р.;

$3_{\text{ЗП}}^0$ – основная зарплата разработчика, р.;

$3_{\text{ЗД}}^{\text{Д}}$ – дополнительная зарплата разработчика, р.;

$3_{\text{СН}}$ – отчисления на социальные нужды, р.;

$Н$ – накладные расходы, р.

Затраты на электроэнергию рассчитываются по формуле:

$$\mathcal{E} = P * C_{\mathcal{E}} * T'_{\text{ОБЩ}} * R_{\text{ЗАГ}}, \quad (23)$$

где:

P – мощность потребляемой электроэнергии, кВт;

$C_{\mathcal{E}}$ – стоимость одного киловатт-часа электроэнергии, р./кВт-ч;

$T'_{\text{ОБЩ}}$ – общие затраты труда на разработку программного продукта, час;

$R_{\text{ЗАГ}}$ – коэффициент загрузки компьютера.

Подставим в формулу (25) числовые значения для вычисления затраты на электроэнергию:

$$\mathcal{E} = 0,08 * 7,5 * 672 * 0,6 = 230\text{р}$$

Основная заработная плата разработчика рассчитывается по формуле:

$$З_{\text{ЗП}}^0 = C_{\text{ЧТС}} * T'_{\text{ОБЩ}}, \quad (24)$$

где:

$C_{\text{ЧТС}}$ – часовая тарифная ставка разработчика, р./час;

$T'_{\text{ОБЩ}}$ – общие затраты труда на разработку программного продукта, час.

Тогда:

$$З_{\text{ЗП}}^0 = 476 * 800 = 380800\text{р}$$

Дополнительная заработная плата составляет 10% от основной:

$$З_{\text{ЗП}}^{\text{Д}} = 380800 * 0,1 = 38080\text{р}$$

Отчисления на социальные нужды рассчитываются по формуле:

$$З_{\text{СН}} = (З_{\text{ЗП}}^0 + З_{\text{ЗП}}^{\text{Д}}) * R_{\text{СН}}, \quad (25)$$

где:

$R_{\text{СН}}$ – коэффициент отчислений на социальные нужды, равный 0,26.

Отчисления на социальные нужды:

$$З_{\text{СН}} = (380800 + 38080) * 0,26 = 108908,8\text{р}$$

Накладные расходы рассчитываются по формуле:

$$Н = 0,1 * (З_{\text{ЗП}}^0 + З_{\text{ЗП}}^{\text{Д}}) \quad (26)$$

Накладные расходы:

$$H = 0,1 * (380800 + 38080) = 41888p$$

Таблица 2 – Калькуляция себестоимости разработки приложения

Наименование статей расходов	Затраты, р.
Основная заработная плата разработчика	380800
Дополнительная заработная плата разработчика	38080
Отчисления на социальные нужды	108908,8
Накладные расходы	41888
Затраты на электроэнергию	1800
Затраты на лицензию	500000
Вспомогательные материалы	0
Полная себестоимость программного продукта	1072476,8

7.3 Расчёт экономического эффекта программного продукта

Экономический эффект от внедрения программного продукта определяется по формуле:

$$\text{Э}_{\text{УГ}} = N * (t_1 - t_2) * C_{\text{ЧТС}} * K_{\text{ДОП}} * K_{\text{СН}} - E_{\text{Н}} * K, \quad (27)$$

где:

N – количество создаваемых моделей;

t_1, t_2 – трудоёмкость создания модели до и после внедрения программного продукта;

$C_{\text{ЧТС}}$ – часовая тарифная ставка пользователя р./час;

$K_{\text{ДОП}}$ – коэффициент отчислений на дополнительную заработную плату равный 0,1;

$K_{\text{СН}}$ – коэффициент отчислений на социальные нужды равный 0,26;

$E_{\text{Н}}$ – нормативный коэффициент окупаемости капитальных вложений равный 0,39;

K – дополнительные капитальные вложения, связанные с разработкой программного продукта.

Для разрабатываемого программного обеспечения принимаются следующие значения:

- $N = 1200$ моделей в год;
- $t_1 = 12$ часов — время проектирования теплообменника без использования программы;
- $t_2 = 4$ часа — время проектирования с использованием разработанной системы;
- $C_{\text{ЧТС}} = 476$ р./час.

Подставив значения в формулу, получим:

$$\text{Э}_{\text{УГ}} = 1200 * (12 - 4) * 476 * 0,1 * 0,26 - 0,39 * 1072476,8 = 128\,565,65$$

Срок окупаемости программного обеспечения рассчитывается по формуле:

$$T_{\text{ОК}} = \frac{C_{\text{ПР}}}{C_1 - C_2} \quad (28)$$

где: $C_{\text{ПР}}$ – себестоимость программного продукта, р.;

C_1 – затраты до внедрения разработанного ПО, р.;

C_2 – затраты после внедрения разработанного ПО, р.

Для расчёта затрат используем формулу:

$$C_1 = 1200 \cdot 12 \cdot 476 \cdot 0,1 \cdot 0,26 - 0,39 \cdot 1072476,8 = -6\,160,95$$

$$C_2 = 1200 \cdot 4 \cdot 476 \cdot 0,1 \cdot 0,26 - 0,39 \cdot 1072476,8 = -552\,992,55$$

Подставив полученные значения в формулу срока окупаемости, получим:

$$T_{\text{ОК}} = \frac{1072476,8}{-6\,160,95 - (-552\,992,55)} = 1,96 \text{ года или } 24 \text{ месяца}$$

Исходя из выполненных расчётов можно сделать вывод, что внедрение разработанной системы автоматизированного проектирования является экономически целесообразным. Использование программного продукта позволяет существенно сократить трудоёмкость проектирования спиральных теплообменников и уменьшить временные затраты на построение CAD-моделей.

Таблица 3 – Экономические показатели программного продукта

Наименование показателя	Ед. изм.	Значение
Основная заработная плата разработчика	р.	380800
Дополнительная заработная плата разработчика	р.	38080
Отчисления на социальные нужды	р.	108908,8
Накладные расходы	р.	41888
Затраты на электроэнергию	р.	1800
Вспомогательные материалы	р.	0
Полная себестоимость программного продукта	р.	1072476,8
Экономический эффект	р.	128565,65
Срок окупаемости программного продукта	лет	1,96

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была успешно решена задача — разработка специализированной мини-САПР для автоматизированного проектирования спиральных теплообменных аппаратов. Проведенный анализ предметной области показал, что существующие отраслевые решения ориентированы исключительно на технологические вычисления и не позволяют генерировать готовые 3D-модели. Для преодоления этого ограничения был создан независимый программный продукт, объединяющий тепловой расчет и параметрическое твердотельное моделирование в единой среде.

Программный комплекс реализован на языке C++ с использованием графического фреймворка Qt и отечественного геометрического ядра C3D Toolkit. Разработанные алгоритмы позволяют автоматически вычислять тепловой баланс, определять оптимальную геометрию каналов из стандартизированного ряда по ГОСТ и подбирать фланцевые соединения. На основе рассчитанных параметров система генерирует математически точную 3D-сборку аппарата, включающую сложный профиль спирали Архимеда, корпус, крышки и патрубки. Созданная геометрия отображается в интерактивном окне визуализации и может быть экспортирована в формат STL для дальнейшего использования в сторонних CAD-системах.

Практическая значимость работы заключается в автоматизации рутинного труда инженера-конструктора. Разработанная мини-САПР устраняет необходимость ручного построения сложной геометрии спирали, обеспечивает бесшовный переход от тепловых расчетов к готовой модели, минимизирует риск ошибок и значительно сокращает сроки конструкторской подготовки производства.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. AspenTech. Aspen Exchanger Design and Rating. — Текст: электронный // aspentech.com: [сайт]. — URL: <https://www.aspentech.com/en/products/engineering/aspen-exchanger-design-and-rating> (дата обращения: 17.03.2026).
2. C++ Reference. Стандартная библиотека C++: математические модули. — Текст: электронный // cppreference.com: [сайт]. — URL: <https://ru.cppreference.com/w/cpp/numeric/math> (дата обращения: 28.12.2025).
3. C3D Toolkit. — Текст: электронный // c3d.ascon.ru: [сайт]. — URL: <https://c3d.ascon.ru/doc/math/modules.html> (дата обращения: 17.03.2026).
4. Heat Transfer Research, Inc. Xspe. — Текст: электронный // htri.net: [сайт]. — URL: <https://www.htri.net/xspe> (дата обращения: 17.03.2026).
5. Qt Group. Qt Academy. — Текст: электронный // qt.io: [сайт]. — URL: <https://www.qt.io/academy> (дата обращения: 17.03.2026).
6. А.Г. Касаткин основные процессы и аппараты химической технологии: Учебник для вузов. - 10-е изд., стереотипное, доработанное. Перепеч. с изд. 1973 г. — М.: ООО ТИД «Альянс», 2004. - 753 с.
7. Бакластов А. М., Горбенко В. А., Удыма П. Г. Проектирование, монтаж и эксплуатация тепломассообменных установок : учеб. пособие для вузов / под ред. А. М. Бакластова. — М. : Энергоиздат, 1981. — 336 с.
8. Барановский Н. В. Пластинчатые и спиральные теплообменники / Н. В. Барановский, Л. М. Коваленко, А. Р. — М.: Машиностроение, 1973. — 288 с.
9. Виноградов, С. Н. Выбор и расчет теплообменников. Учебное пособие / С.Н. Виноградов, К.В. Таранцев, О.С. Виноградов — Пенза, Пензенский государственный университет, 2001. - 100 с.
10. ГОСТ 12067—80 Аппараты теплообменные спиральные. Типы, основные параметры и размеры. — Введ.1982-01-01 — Москва: Издательство стандартов, 1981 г.

11. ГОСТ 14249-89. Сосуды и аппараты. Нормы и методы расчета на прочность. — Москва : Изд-во стандартов, 1989. — 61 с. — Текст : непосредственный.

12. ГОСТ 33259-2015. Фланцы арматуры, соединительных частей и трубопроводов на номинальное давление до PN 250. Конструкция, размеры и общие технические требования. — Москва : Стандартинформ, 2016. — 107 с. — Текст : непосредственный.

13. ГОСТ 34347-2017. Сосуды и аппараты стальные сварные. Общие технические условия. — Москва : Стандартинформ, 2018. — 151 с. — Текст : непосредственный.

14. ГОСТ 5632-2014. Легированные нержавеющие стали и сплавы коррозионно-стойкие, жаростойкие и жаропрочные. Марки. — Москва : Стандартинформ, 2015. — 56 с. — Текст : непосредственный.

15. Гусаров А. М. Тепломассообмен: методические указания по выполнению курсовых проектов / сост. А. М. Гусаров, А. С. Марущак. — Витебск: УО «ВГТУ», 2026. — 89 с.

16. Карапузова Н. Ю. Расчет теплообменных аппаратов [Электронный ресурс] : методические указания к курсовому и дипломному проектированию / М-во образования и науки Рос. Федерации, Волгогр. гос. архит.-строит. ун-т ; сост. Н. Ю. Карапузова, В. М. Фокин. — Электронные текстовые и графические данные (2,11 Мбайт). — Волгоград : ВолгГАСУ, 2013

17. Макконнелл, С. Совершенный код : мастер-класс: пер. с англ. / С. Макконнелл. — Москва : Русская редакция, 2017. — 869 с.: ил.. — Библиогр.: с. 842-862

18. Михеев М. А., Михеева И. М. Основы теплопередачи : учеб. для вузов. М. : Бастет, 2010. 344 с.

19. Муфтеев, В. Г. Прикладные САПР и приложения на основе геометрического ядра C3D для проектирования изделий с функциональными кривыми / В. Г. Муфтеев , А. Максименко , Р. И. Ахметшин , А. П. Зелев , Р. А.

Зиатдинов , Ф. Т. Зиганшина , Р. И., Набиев , И. М. Фархутдинов , М. М. Фаттахов. — Нижний Новгород, 2021.

20. Норенков, И. П. Основы автоматизированного проектирования : учебник для вузов / И. П. Норенков. — 4-е изд., перераб. и доп. — Москва : Изд-во МГТУ им. Н. Э. Баумана, 2009. — 430 с.

21. Плановский, А. Н. Процессы и аппараты химической технологии : учебник для техникумов / А. Н. Плановский, В. М. Рамм, С. З. Каган. - 5-е изд., стереотипное. - М. : Химия, 1968. - 847 с. : ил. - Библиогр.: с. 828-831.

22. Прата, С. Язык программирования C++. Лекции и упражнения / С. Прата. — Москва : Вильямс, 2021.

23. Прохоренок Н. А. Qt 6. Разработка оконных приложений на C++. — СПб.: БХВ-Петербург, 2022. — 512 с.: ил. — (Профессиональное программирование)

24. Теплотехнический справочник / В. В. Галактионов, В. Ю. Пикус, Н. И. Горбунова и др. ; под ред. В. Н. Юренева, П. Д. Лебедева. — 2-е изд., перераб. — М. : Энергия, 1976. Т. 2. 896 с.

25. Цветков, Ф. Ф. Тепломассообмен : учебное пособие для вузов / Ф. Ф. Цветков, Б. А. Григорьев. — Москва : Издательство МЭИ, 2005.

1 ОБЩИЕ ПОЛОЖЕНИЯ

1.1 Наименование программы

1.1.1 Полное наименование программы

«Программное обеспечение для автоматизированного проектирования и построения трёхмерных моделей спиральных теплообменных аппаратов SpiralHX».

1.1.2 Краткое наименование программы

«SpiralHX».

1.2 Основания для проведения работ

Основанием для разработки программы «SpiralHX» является основная профессиональная образовательная программа по направлению 09.03.01 «Информатика и вычислительная техника», профиль «Интеграция и программирование в САПР», 2022 г.п., учебный план по направлению 09.03.01 «Информатика и вычислительная техника», профиль «Интеграция и программирование в САПР», 2022 г.п.

1.3 Наименование предприятий (объединений) разработчика и заказчика (пользователя) системы и реквизиты

1.3.1 Заказчик

Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет».

Адрес: 107023, г. Москва, ул. Большая Семёновская, д. 38.

Телефон: 8 (495) 223–05–23.

1.3.2 Разработчик

Факультет информационных технологий, кафедра СМАРТ-технологий.

Адрес: 127550, г. Москва, ул. Прянишникова, 2А.

1.4 Перечень документов, на основании которых создаётся система

Основанием для разработки программы «SpiralHX» являются следующие документы:

1. Лицензия на образовательную деятельность от 10.03.2021 №309-06 Московского политехнического университета.
2. Аккредитация A007-00115-77/00890767 Московского политехнического университета.
3. Устав Московского политехнического университета.
4. Положение Московского политехнического университета о порядке организации и осуществления образовательной деятельности.

1.5 Плановые сроки начала и окончания работы

Плановый срок начала работ по разработке программы «SpiralHX» – 2 февраля 2026 года. Окончание – 15 июля 2026 года.

1.6 Источники и порядок финансирования

Разработка ПО выполняется в рамках образовательного процесса на безвозмездной основе.

1.7 Порядок оформления и предъявления заказчику результатов работ

Система передаётся в виде функционирующего программного комплекса на базе средств вычислительной техники Заказчика и Исполнителя в сроки, установленные в пункте 1.5. Приёмка системы осуществляется комиссией в составе уполномоченных представителей Заказчика и Исполнителя.

Порядок предъявления системы, её испытаний и окончательной приёмки определён в пункте 6 настоящего ТЗ. Совместно с предъявлением системы производится сдача разработанного Исполнителем комплекта документации согласно пункту 8 настоящего ТЗ.

1.8 Состав используемой нормативно-технической документации

При разработке программы и создании проектно-эксплуатационной документации Исполнитель должен руководствоваться требованиями следующих нормативных документов:

1. ГОСТ 34.601-90. Комплекс стандартов на автоматизированные системы. Стадии создания;
2. ГОСТ 34.201-2020. Виды, комплектность и обозначение документов при создании автоматизированных систем;
3. ГОСТ Р 59853-2021. Автоматизированные системы. Термины и определения;
4. ГОСТ Р 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам;
5. ГОСТ 12067–80. Аппараты теплообменные спиральные. Типы, основные параметры и размеры;
6. ГОСТ 33259-2015. Фланцы арматуры, соединительных частей и трубопроводов на номинальное давление до PN 250;
7. ГОСТ 34347-2017. Сосуды и аппараты стальные сварные. Общие технические условия;
8. ГОСТ 14249-89. Сосуды и аппараты. Нормы и методы расчёта на прочность;
9. ГОСТ 12.1.030-81. Система стандартов безопасности труда. Электробезопасность. Защитное заземление, зануление;
10. СанПиН 2.2.2./2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы.

2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ ПРОГРАММЫ

2.1 Назначение программы

ПО «SpiralHX» предназначено для автоматизированного проектирования спиральных теплообменных аппаратов: выполнения теплового и гидравлического расчёта, подбора стандартных конструктивных элементов и автоматической генерации параметрической трёхмерной твердотельной модели аппарата на базе отечественного геометрического ядра C3D Toolkit.

2.2 Цель создания программы

Основной целью создания программы является разработка специализированного инженерного инструмента, обеспечивающего автоматизацию рутинных конструкторских задач при проектировании спиральных теплообменных аппаратов и устраняющего необходимость ручного построения сложной геометрии в универсальных САПР.

Для реализации поставленной цели необходимо решить следующие задачи:

1. Разработать методику и алгоритм теплового, геометрического и гидравлического расчёта спирального теплообменного аппарата.
2. Реализовать алгоритм автоматического построения параметрической 3D-модели аппарата с использованием API C3D Toolkit.
3. Разработать программное приложение с графическим интерфейсом пользователя на базе фреймворка Qt.
4. Обеспечить возможность экспорта созданной модели в стандартные форматы CAD-систем.
5. Провести тестирование разработанного программного комплекса.

3 ХАРАКТЕРИСТИКА ОБЪЕКТОВ АВТОМАТИЗАЦИИ

3.1 Объект автоматизации

Объектом автоматизации является процесс проектирования спирального теплообменного аппарата, включающий тепловой и гидравлический расчёт, подбор геометрических параметров каналов из стандартного ряда по ГОСТ 12067–80 и построение трёхмерной твердотельной модели конструкции.

3.2 Существующее программное обеспечение

В настоящий момент для разработки системы используется следующее программное обеспечение:

1. ОС Windows 11 — проприетарная операционная система.
2. Microsoft Visual Studio 2022 — среда разработки программного кода.
3. C3D Toolkit — отечественное геометрическое ядро для твердотельного моделирования.
4. Qt Framework — кроссплатформенный фреймворк для разработки графических приложений.

3.3 Существующее техническое обеспечение

Минимальная конфигурация компьютера:

1. Процессор – x86-64-битный с минимальным количеством ядер 4 и минимальной частотой ядра 2,3 ГГц.
2. Оперативная память 16.00 ГБ.
3. Видеокарта – Intel Iris Xe Graphics, 4 ГБ видеопамяти.
4. Дисковое пространство объёмом не менее 128 ГБ.
5. Монитор с разрешением не менее 1920×1080 точек.

3.4 Существующее нормативно-правовое обеспечение

Существующее нормативно-правовое обеспечение составляют федеральные нормативные правовые акты:

1. Конституция Российской Федерации.
2. Гражданский кодекс Российской Федерации.
3. Устав Московского политехнического университета.
4. Федеральный закон «Об информации, информационных технологиях и о защите информации» от 27.07.2006 № 149-ФЗ.
5. Закон РФ от 07.02.1992 № 2300-1 (ред. от 14.07.2022) «О защите прав потребителей».

4 ТРЕБОВАНИЯ К СИСТЕМЕ

4.1 Требования к системе в целом

4.1.1 Перспективы развития и модернизации системы

Разработка программы должна подразумевать возможность её модернизации: добавление новых типов теплообменных аппаратов, расширение базы стандартных типоразмеров, интеграция с внешними системами нормирования и планирования производства, а также поддержка новых версий геометрического ядра C3D Toolkit.

4.1.2 Требования к численности и квалификации персонала

Для эксплуатации ПО «SpiralHX» определены следующие роли:

1. Инженер-конструктор — основной пользователь системы.

Основными обязанностями инженера-конструктора являются:

1. Ввод исходных теплофизических и геометрических параметров проектируемого аппарата.

2. Анализ результатов расчёта и выбор подходящей конфигурации.

3. Запуск построения трёхмерной модели и её экспорт.

Инженер-конструктор должен иметь образование не ниже среднего профессионального по направлению «Химическая технология» или «Машиностроение» и базовые навыки работы с CAD-системами.

4.1.3 Требования к показателям назначения

Программа должна обеспечивать выполнение полного цикла проектирования — от ввода исходных данных до получения готовой 3D-модели — в пределах одного рабочего сеанса без необходимости использования сторонних расчётных программ. Время генерации трёхмерной модели аппарата не должно превышать 60 секунд при конфигурации оборудования, соответствующей требованиям раздела 4.3.

4.1.4 Требования к надёжности

Система должна сохранять работоспособность и обеспечивать восстановление своих функций при возникновении следующих внештатных ситуаций:

Сбои в системе электроснабжения, приводящие к перезагрузке ОС — восстановление программы должно происходить после перезапуска ОС и запуска исполняемого файла.

Ввод физически некорректных входных данных — система должна выводить диагностическое сообщение об ошибке и возвращаться в рабочее состояние без аварийного завершения.

Ошибки инициализации геометрического ядра C3D (например, при отсутствии или истечении срока действия лицензии) — система должна предоставить пользователю возможность ввода лицензионных данных.

4.1.5 Требования к безопасности

Все внешние элементы технических средств системы, находящиеся под напряжением, должны иметь защиту от случайного прикосновения, а сами технические средства — зануление или защитное заземление в соответствии с ГОСТ 12.1.030-81 и ПУЭ.

Факторы, оказывающие вредные воздействия на здоровье со стороны всех элементов системы, не должны превышать действующих норм (СанПиН 2.2.2./2.4.1340-03 от 03.06.2003 г.).

4.1.6 Требования к эксплуатации, техническому обслуживанию и хранению

Программа должна быть рассчитана на эксплуатацию в составе программно-технического комплекса Заказчика. Для нормальной эксплуатации должно быть обеспечено бесперебойное питание ПЭВМ. Все пользователи системы должны соблюдать правила эксплуатации электронной вычислительной техники. Квалификация персонала и его подготовка должны соответствовать технической документации.

4.1.7 Требования к защите информации

Система должна обеспечивать защиту от несанкционированного доступа к лицензионным ключам геометрического ядра C3D, хранящимся в локальном файле конфигурации. Доступ к исполняемым файлам и конфигурационным

данным программы должен быть ограничен средствами операционной системы.

4.1.8 Требования по патентной чистоте

Установка программы не должна предъявлять дополнительных требований к покупке лицензий на программное обеспечение сторонних производителей, за исключением лицензионной версии операционной системы Microsoft Windows и лицензии на геометрическое ядро C3D Toolkit, указанной в дополнительных соглашениях.

4.2 Требования к функциям, выполняемым программой

Программа «SpiralHX» должна обеспечивать выполнение следующих функций:

4.2.1 Подсистема ввода и валидации исходных данных

Обеспечивает ввод теплофизических параметров рабочих сред (расход, температуры, плотность, теплоёмкость, теплопроводность, вязкость), геометрических ограничений и параметров стенки. Включает автоматическую проверку корректности введённых данных, блокирование физически недопустимых значений и автоматический расчёт теплового баланса при наличии одного неизвестного параметра.

4.2.2 Подсистема теплового и гидравлического расчёта

Выполняет итерационный расчёт коэффициентов теплоотдачи и теплопередачи с учётом кривизны спирального канала, определение среднелогарифмического температурного напора, геометрический расчёт параметров спирали (длина листа, количество витков, наружный диаметр) и гидравлический расчёт потерь давления. По результатам расчёта формируется протокол с тепловыми, геометрическими и гидравлическими характеристиками аппарата.

4.2.3 Подсистема подбора стандартных элементов

Обеспечивает автоматический подбор ширины теплообменника из стандартного ряда по ГОСТ 12067–80 и типоразмеров фланцевых соединений

по ГОСТ 33259-2015 исходя из рассчитанного давления и диаметра корпуса. При невозможности подбора выводится предупреждающее сообщение.

4.2.4 Подсистема трёхмерного моделирования

Выполняет автоматическую генерацию параметрической 3D-сборки спирального теплообменного аппарата с использованием API C3D Toolkit. Модель включает: спиральные каналы (профиль спирали Архимеда), центральный разделитель (кern), обечайку с фланцами, входные и выходные патрубки, уплотнительные прокладки и крышки. Между компонентами накладываются геометрические ограничения (соосность, плотное прилегание) посредством системы GCM.

4.2.5 Подсистема визуализации и экспорта

Обеспечивает интерактивный просмотр построенной 3D-модели с возможностью вращения, масштабирования и панорамирования. Поддерживает экспорт модели в форматы C3D, STEP, PARASOLID, STL, OBJ, SAT, JT. Результаты расчёта могут быть сохранены в текстовый файл или PDF-документ.

4.3 Требования к техническому обеспечению

Для работы ПО «Мини-САПР СТА» необходим персональный компьютер со следующими минимальными характеристиками:

1. Процессор: Intel Core i5 или эквивалент (4 ядра, 2,0 ГГц и выше);
2. Оперативная память: не менее 8 ГБ;
3. Дисковое пространство: не менее 2 ГБ свободного места;
4. Видеоадаптер с поддержкой OpenGL 3.3 и выше;
5. Монитор с разрешением не менее 1920×1080.

Рекомендуемая конфигурация:

1. Процессор: Intel Core i7 или AMD Ryzen 7 (8 ядер, 3,0 ГГц и выше);
2. Оперативная память: 16 ГБ и более;
3. Дисковое пространство: SSD объёмом 256 ГБ и более.

4.3.1 Требования к программному обеспечению

Программа «Мини-САПР СТА» разработана с использованием Microsoft Visual Studio 2022 и совместима с операционными системами Windows.

Рекомендуется использовать следующие версии ОС:

1. Microsoft Windows 10 (64-разрядная версия);
2. Microsoft Windows 11 (любая версия).

Для работы программы необходима активная лицензия C3D Toolkit. Установка дополнительного программного обеспечения не требуется — все зависимости поставляются в составе дистрибутива.

5 СОСТАВ И СОДЕРЖАНИЕ РАБОТ ПО СОЗДАНИЮ ПРОГРАММЫ

Работы по созданию программы «Мини-САПР СТА» выполняются в несколько этапов:

1. Анализ предметной области. Изучение конструкции спиральных теплообменных аппаратов, нормативной документации и существующих программных решений. Результат: аналитическая записка. Срок: 2 февраля 2026 г. — 28 февраля 2026 г. Ответственный: программист.

2. Проектирование. Разработка технического задания, выбор архитектуры приложения, проектирование структур данных и алгоритмов. Результат: техническое задание, схема архитектуры. Срок: 1 марта 2026 г. — 14 марта 2026 г. Ответственный: программист.

3. Разработка расчётных алгоритмов. Программная реализация методики теплового, геометрического и гидравлического расчёта. Результат: реализованный и проверенный расчётный модуль. Срок: 15 марта 2026 г. — 31 марта 2026 г. Ответственный: программист.

4. Разработка модуля трёхмерного моделирования. Реализация алгоритмов генерации 3D-геометрии с использованием API C3D Toolkit. Результат: функционирующий модуль построения модели. Срок: 1 апреля 2026 г. — 15 апреля 2026 г. Ответственный: программист.

5. Разработка пользовательского интерфейса. Реализация интерфейса приложения на базе Qt, модуля визуализации C3D Vision, диалогов ввода и вывода данных. Результат: готовое приложение с полным интерфейсом. Срок: 16 апреля 2026 г. — 30 апреля 2026 г. Ответственный: программист.

6. Тестирование и доработка. Проведение функционального тестирования, устранение выявленных дефектов. Результат: протокол испытаний. Срок: 1 мая 2026 г. — 15 мая 2026 г. Ответственный: программист.

7. Разработка рабочей документации. Составление пояснительной записки, технического задания и руководства пользователя. Срок: 16 мая 2026 г. — 12 июня 2026 г. Ответственный: программист.

6 ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ СИСТЕМЫ

6.1 Порядок контроля и приёмки программы

1. Проверка работоспособности программы согласно методикам тестирования.
2. Проверка корректности расчётных алгоритмов на тестовых примерах.
3. Проверка корректности построения трёхмерной модели.
4. Проверка функции экспорта модели в сторонние CAD-системы.
5. Проверка сопутствующей документации и её комплектности.

Приёмно-сдаточные испытания проводятся согласно этапу тестирования, описанному в пункте 5.

6.2 Общие требования к приёмке работ

Сдача-приёмка работ производится поэтапно в соответствии с учебным планом по направлению 09.03.01 «Информатика и вычислительная техника», профиль «Интеграция и программирование в САПР», 2022 г.п.

Сдача-приёмка осуществляется комиссией, в состав которой входят представители Заказчика и Исполнителя. По результатам приёмки подписывается акт приёмочной комиссии.

Все создаваемые в рамках настоящей работы программные изделия передаются Заказчику как в виде готовых скомпилированных модулей, так и в виде исходных кодов, представляемых в электронной форме на стандартном машинном носителе.

6.3 Статус приёмочной комиссии

Статус приёмочной комиссии определяется Заказчиком до проведения испытаний.

7 ТРЕБОВАНИЯ К СОСТАВУ И СОДЕРЖАНИЮ РАБОТ ПО ПОДГОТОВКЕ ОБЪЕКТА АВТОМАТИЗАЦИИ К ВВОДУ СИСТЕМЫ В ДЕЙСТВИЕ

При подготовке к вводу в эксплуатацию программы «Мини-САПР СТА» Заказчик должен обеспечить выполнение следующих работ:

1. Определить ответственных должностных лиц за внедрение и проведение опытной эксплуатации.
2. Обеспечить присутствие пользователей на обучении работе с программой.
3. Обеспечить соответствие рабочих мест требованиям, изложенным в разделе 4.3 настоящего ТЗ.
4. Получить и активировать лицензию на геометрическое ядро C3D Toolkit.
5. Совместно с Исполнителем подготовить план установки программы на технических средствах Заказчика.
6. Провести опытную эксплуатацию программы на реальных инженерных задачах.

8 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ

Состав программной документации:

- Техническое задание;
- Руководство пользователя.

1 НАЗНАЧЕНИЕ И УСЛОВИЯ ПРИМЕНЕНИЯ

1.1 Наименование программы

Полное наименование программы: «Программное обеспечение для расчёта и проектирования спирального теплообменного аппарата».

Краткое наименование: мини-САПР «Спиральный теплообменный аппарат».

1.2 Назначение программы

Программное обеспечение предназначено для автоматизированного расчёта и параметрического проектирования спиральных теплообменных аппаратов. Приложение позволяет инженеру ввести исходные технологические данные, выполнить тепловой и гидравлический расчёт, определить геометрические параметры аппарата и автоматически сгенерировать его трёхмерную твердотельную модель с использованием геометрического ядра C3D Toolkit.

Программа ориентирована на автоматизацию рутинных расчётных и конструкторских задач, сокращение времени проектирования и снижение вероятности ошибок ручного моделирования.

1.3 Функции, выполняемые программой

Мини-САПР «Спиральный теплообменный аппарат» выполняет следующие функции:

- тепловой расчёт теплообменного аппарата (определение тепловой нагрузки, температурного напора, коэффициентов теплоотдачи и теплопередачи, площади теплообмена);

- гидравлический расчёт теплообменного аппарата (определение потерь давления в каналах, подбор параметров потоков);

- определение геометрических параметров аппарата (шаг навивки, диаметр, ширина каналов, количество витков);
- создание параметрической трёхмерной твердотельной модели теплообменника средствами API C3D Toolkit;
- интерактивный просмотр построенной 3D-модели непосредственно в окне приложения;
- сохранение твердотельной модели в файл для последующего использования в САПР.

1.4 Область применения

Программное обеспечение может применяться в учебном процессе инженерных специальностей, а также на предприятиях химической, нефтехимической, пищевой и энергетической промышленности при конструировании теплообменного оборудования.

1.5 Условия применения

Для корректной работы мини-САПР необходимо соблюдение требований к техническому и программному обеспечению, изложенных в разделе 2.2 настоящего руководства.

2 ПОДГОТОВКА К РАБОТЕ

2.1 Состав и содержание дистрибутива

Дистрибутив программного обеспечения содержит следующие компоненты:

- исполняемый файл приложения SpiralHX.exe;
- библиотеки геометрического ядра C3D Toolkit (c3d.dll и сопутствующие файлы);
- файл лицензии C3D Toolkit (c3d_license.lic);
- файл зависимостей Qt (Qt6Core.dll, Qt6Widgets.dll, Qt6Gui.dll и др.);
- настоящее Руководство пользователя.

2.2 Требования к техническому и программному обеспечению

Для работы с приложением необходим персональный компьютер под управлением операционной системы Microsoft Windows 10/11 (64-разрядная версия).

Минимальные требования к оборудованию:

- центральный процессор с тактовой частотой 2,0 ГГц и выше;
- оперативная память от 4 ГБ;
- место на жёстком диске — не менее 500 МБ;
- графический адаптер с поддержкой OpenGL 2.0;
- разрешение экрана не менее 1280×768 пикселей.

Рекомендуемые требования к оборудованию:

- центральный процессор Intel Core i5 / AMD Ryzen 5 и выше;
- оперативная память 8 ГБ и более;
- разрешение экрана 1920×1080 пикселей и выше;
- выделенный графический адаптер.

2.3 Установка программного обеспечения

Для установки программного обеспечения необходимо выполнить следующие действия:

1. Скопировать дистрибутив в выбранную папку на жёстком диске.
 2. Убедиться, что файл лицензии C3D Toolkit (c3d_license.lic) находится в той же папке, что и исполняемый файл SpiralHX.exe.
 3. При необходимости установить библиотеки Visual C++ Redistributable for Visual Studio 2019/2022 (входят в состав дистрибутива).
 4. Запустить файл SpiralHeatExchanger.exe. При необходимости разрешить запуск в диалоге «Контроль учётных записей пользователей», нажав кнопку «Да».
 5. Для удобства создать ярлык на рабочем столе.
- Удаление программы выполняется путём удаления папки с файлами приложения. Записи в реестре не создаются.

2.4 Проверка работоспособности

Для проверки работоспособности приложения необходимо:

1. Запустить SpiralHX.exe.
2. Ввести тестовые исходные данные в поля раздела «Исходные данные».
3. Нажать кнопку «Рассчитать».
4. Убедиться, что в разделе «Результаты» отображаются расчётные значения, а в области визуализации — трёхмерная модель аппарата.

Успешное отображение результатов расчёта и 3D-модели подтверждает корректную работу программы.

3 ОПИСАНИЕ ОПЕРАЦИЙ

3.1 Запуск программного обеспечения

Для запуска приложения необходимо дважды нажать на ярлык мини-САПР на рабочем столе или запустить файл SpiralHX.exe из папки установки. После загрузки откроется главное окно приложения, включающее панель ввода исходных данных, вкладки расчёта и область визуализации 3D-модели (Рисунок 1).

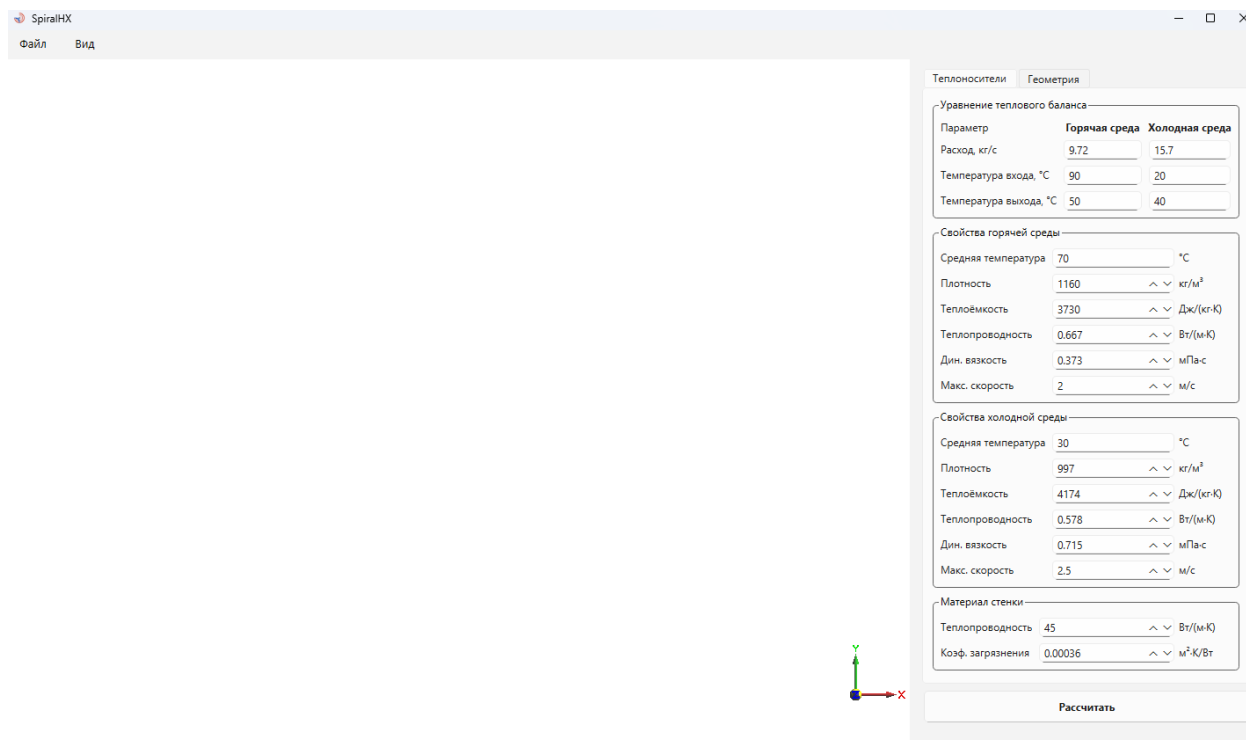


Рисунок 1 – Главное окно приложения

3.2 Ввод исходных данных

На вкладке «Исходные данные» пользователь задаёт параметры теплоносителей и режима работы аппарата (Рисунок 2):

- горячий теплоноситель: тип среды, температура на входе и выходе ($^{\circ}\text{C}$), массовый расход (кг/с), теплофизические свойства;
- холодный теплоноситель: тип среды, температура на входе ($^{\circ}\text{C}$), теплофизические свойства;
- конструктивные параметры: ширина канала (мм), толщина листа (мм), допустимые потери давления (Па).

Теплоносители

Геометрия

Уравнение теплового баланса

Параметр	Горячая среда	Холодная среда
Расход, кг/с	9.72	15.7
Температура входа, °C	90	20
Температура выхода, °C	50	40

Свойства горячей среды

Средняя температура	70	°C
Плотность	1160	кг/м³
Теплоёмкость	3730	Дж/(кг·K)
Теплопроводность	0.667	Вт/(м·K)
Дин. вязкость	0.373	мПа·с
Макс. скорость	2	м/с

Свойства холодной среды

Средняя температура	30	°C
Плотность	997	кг/м³
Теплоёмкость	4174	Дж/(кг·K)
Теплопроводность	0.578	Вт/(м·K)
Дин. вязкость	0.715	мПа·с
Макс. скорость	2.5	м/с

Материал стенки

Теплопроводность	45	Вт/(м·K)
Козф. загрязнения	0.00036	м²·K/Вт

Рисунок 2 – Вкладка «Исходные данные»

Значения вводятся с клавиатуры в соответствующие числовые поля. Все поля обязательны для заполнения. Допустимые диапазоны значений указаны во всплывающих подсказках при наведении курсора на поле ввода.

3.3 Выполнение расчёта

3.3.1 Тепловой и гидравлический расчёт

После заполнения всех полей исходных данных необходимо нажать кнопку «Рассчитать». Программа последовательно выполнит:

- определение тепловой нагрузки и материального баланса;
- расчёт среднего температурного напора;
- определение коэффициентов теплоотдачи и теплопередачи;
- геометрический расчёт аппарата (площадь теплообмена, шаг навивки, диаметр, число витков);
- гидравлический расчёт (потери давления в каналах).

По завершении расчёта результаты отобразятся на вкладке «Результаты расчёта» (Рисунок 3).

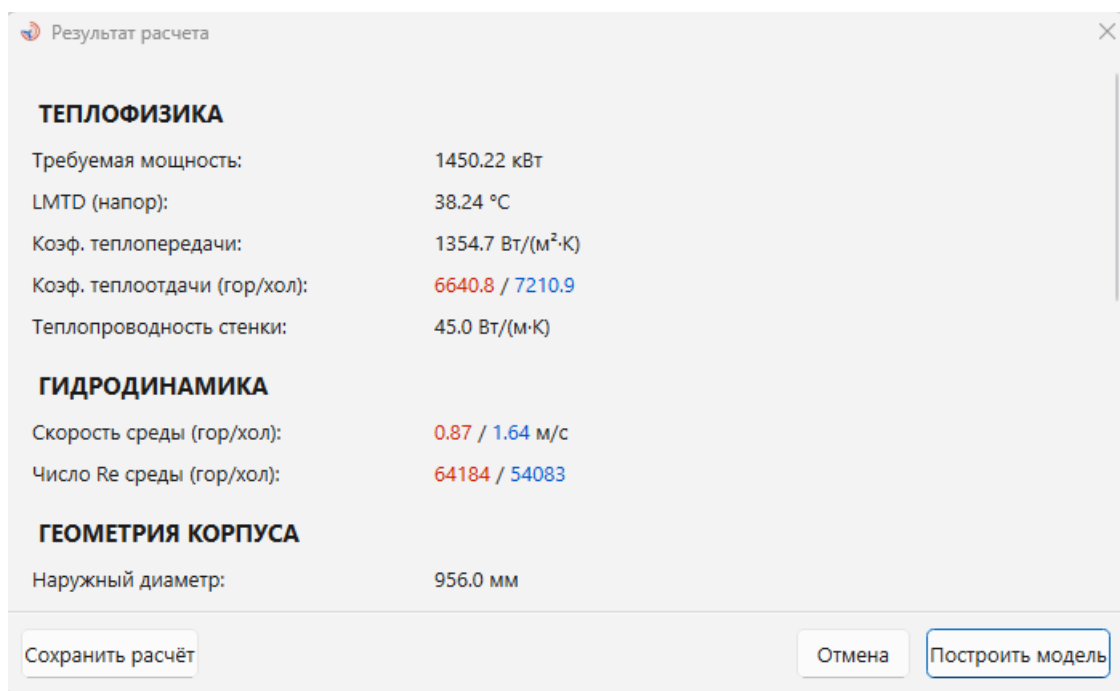


Рисунок 3 – Вкладка «Результаты расчёта»

3.3.2 Алгоритм выбора ширины канала

В случае если введенные параметры не позволяют найти конструктивно приемлемое решение, программа автоматически запустит итерационный алгоритм подбора оптимальной ширины канала. В строке состояния будет отображён прогресс подбора. По завершении автоматического подбора выбранное значение ширины будет подставлено в поле «Ширина канала» и выделено цветом.

3.4 Построение трёхмерной модели

Для построения 3D-модели аппарата необходимо нажать кнопку «Построить модель». Построение выполняется средствами API C3D Toolkit и включает следующие этапы (Рисунок 4):

- моделирование спиральных каналов по рассчитанным геометрическим параметрам;
- моделирование обечайки;

- моделирование патрубков входа и выхода теплоносителей;
- моделирование прокладок и уплотнений;
- моделирование крышек (днищ);
- сборка всех компонентов в единую модель.

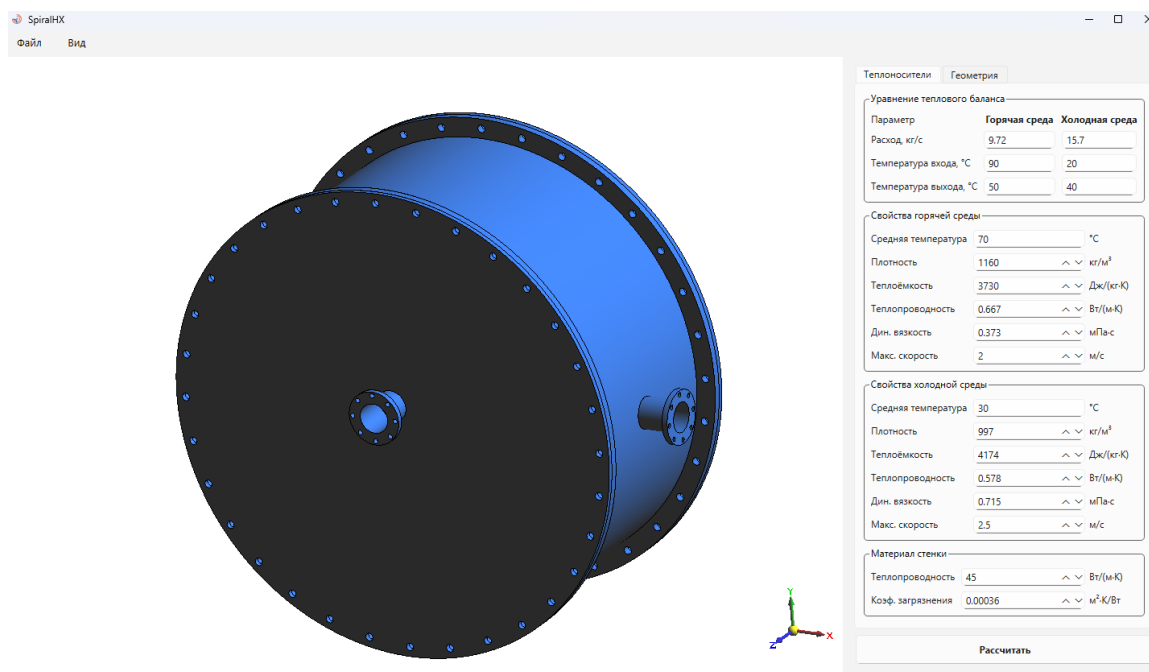


Рисунок 4 – Построение 3D-модели аппарата

После завершения построения трёхмерная модель отображается в области визуализации. Модель можно вращать, масштабировать и перемещать с помощью мыши (Таблица 1).

Таблица 1 – Управление трёхмерной моделью

Действие	Управление
Вращение модели	Удерживать левую кнопку мыши и перемещать курсор
Масштабирование	Колесо прокрутки мыши
Перемещение модели	Удерживать правую кнопку мыши и перемещать курсор
Сброс вида	Двойной щелчок левой кнопкой мыши в области модели

3.5 Сохранение модели

Для сохранения построенной трёхмерной модели необходимо нажать кнопку «Сохранить модель». Откроется диалоговое окно сохранения файла, в котором следует указать имя и путь для сохранения. Модель сохраняется в

формате C3D (расширение .c3d), пригодном для последующего открытия в системах, поддерживающих данный формат.

Для сохранения введённых параметров и результатов расчёта необходимо воспользоваться пунктом меню «Файл → Сохранить расчёт». Сохранённый файл имеет расширение .txt и может быть загружен при следующем запуске приложения.

3.6 Загрузка ранее сохранённого расчёта

Для загрузки ранее сохранённого расчёта необходимо воспользоваться пунктом меню «Файл → Открыть расчёт». В открывшемся диалоге выбрать файл с расширением .txt. После загрузки все поля исходных данных будут заполнены сохранёнными значениями, а результаты расчёта восстановлены.

3.7 Завершение работы

Для завершения работы приложения необходимо нажать кнопку закрытия окна.

4 АВАРИЙНЫЕ СИТУАЦИИ

В ходе работы приложения могут возникать ошибки. В Таблице 2 приведены возможные ошибки, их причины и рекомендуемые действия пользователя.

Таблица 2 – Возможные ошибки и способы их устранения

Класс ошибки	Ошибка	Описание ошибки	Требуемые действия пользователя
Ошибка ввода данных	Некорректные значения параметров	Введены данные вне допустимого диапазона (например, отрицательная температура, нулевой расход)	Проверьте правильность введённых значений и убедитесь, что они соответствуют физически допустимым диапазонам
Ошибка загрузки	Файл не открывается	Файл сохранения повреждён или создан в несовместимой версии программы	Убедитесь, что файл не повреждён. Попробуйте загрузить резервную копию или начните новый расчёт
Ошибка геометрии	Модель не построена	Параметры аппарата приводят к геометрически некорректной модели (например, шаг навивки меньше суммы толщин листов)	Скорректируйте геометрические параметры: проверьте шаг навивки, толщину листов и ширину канала
Ошибка лицензии	C3D Toolkit недоступен	Не найдена лицензия геометрического ядра C3D Toolkit или истёк срок её действия	Убедитесь, что файл лицензии расположен в корректной директории и не истёк срок её действия. При необходимости обратитесь к администратору

Если ошибка сохраняется после выполнения рекомендованных действий, следует перезапустить приложение. При повторном возникновении ошибки необходимо обратиться к разработчику программного обеспечения.

5 РЕКОМЕНДАЦИИ ПО ОСВОЕНИЮ

Для освоения данной программы и комфортного пользования необходимо: получить навыки работы с операционной системой Windows, а также ознакомиться с данным Руководством пользователя.

Код файла SpiralHeatExchanger.h

```

#pragma once

#include <string>
#include <vector>

struct Fluid
{
    double G;           // массовый расход [кг/с]
    double rho;          // плотность [кг/м3]
    double cp;           // теплоёмкость [Дж/(кг·К)]
    double lambda;       // теплопроводность [Вт/(м·К)]
    double viscosity;    // динамическая вязкость [Па·с]

    double t_in;         // входная температура
    double t_out;         // выходная температура

    double max_velocity; // допустимая скорость
};

struct Wall
{
    double lambda;
    double thickness;
    double fouling_factor;
};

struct Geometry
{
    double width_hot;
    double width_cold;

    double d_inner;
    double shell_thickness;
    double cover_thickness;

    bool auto_width = true;
    double width_min = 0.0;
    double width_max = 9999.0;
};

struct Flange {
    std::string name;
    double inner_d;
    double outer_d;
    double bolt_circle_d;
    double flange_thickness;
    double bolt_d;
    int bolt_count;
};

struct CalculationResult
{
    std::string error_message;

    double Q_required;
    double LMTD_required;
};

```



```

double velocity_hot;
double velocity_cold;

double Re_hot;
double Re_cold;

double alpha_hot;
double alpha_cold;
double K;

double Q_real;
double t_hot_out_real;
double t_cold_out_real;
double LMTD_real;

double power_hot;
double power_cold;

double A;
double channel_length;
double channel_width;
double turns;
double D;

Geometry geometry;
Wall wall;
Flange selected_flange;
Flange shell_flange;
double inlet_D;
double gasket_thickness = 0.003;
double nozzle_shell_thickness = 4.0;
double nozzle_shell_length = 150.0;
};

class SpiralHeatExchanger
{
public:
    SpiralHeatExchanger();
    SpiralHeatExchanger(const Fluid& hot, const Fluid& cold, const Wall&
wallProperties, const Geometry& geom);

    void setHotFluid(const Fluid& hot) { hotFluid = hot; };
    void setColdFluid(const Fluid& cold) { coldFluid = cold; };
    void setWall(const Wall& wall) { wallProperties = wall; };
    void setGeometry(const Geometry& geom) { geo = geom; };

    CalculationResult calculate();

private:
    std::string validateInputs();

    double calculateLMTD(double t_hot_in, double t_hot_out, double t_cold_in, double
t_cold_out) const;
    double calculateAlpha(const Fluid& f, double velocity, double d_eq, double Dc)
const;
    double calculateAlphaCoof(double de, double Dc) const;

    Fluid hotFluid;

```

```

    Fluid coldFluid;
    Wall wallProperties;
    Geometry geo;

    const std::vector<double> standard_width = { 0.2, 0.32, 0.5, 0.8, 1.0, 1.1, 1.25,
1.5 };
    const double ReConst1 = 10000.0;
    const double ReConst2 = 2300.0;
    const double Eps = 1e-6;
    const double g = 9.81;
    const double n = 0.7;
    const double PI = 3.14159265358979323846;
};

```

Код файла SpiralHeatExchanger.cpp

```

#include "SpiralHeatExchanger.h"
#include <math.h>

using namespace std;

SpiralHeatExchanger::SpiralHeatExchanger()
{
};

SpiralHeatExchanger::SpiralHeatExchanger(const Fluid& hot, const Fluid& cold, const
Wall& wallProperties, const Geometry& geom)
: hotFluid(hot), coldFluid(cold), wallProperties(wallProperties), geo(geom)
{
};

CalculationResult SpiralHeatExchanger::calculate()
{
    CalculationResult result;

    string err = validateInputs();
    if (!err.empty()) {
        result.error_message = err;
        return result;
    }

    result.Q_required = hotFluid.G * hotFluid.cp * std::abs(hotFluid.t_in -
hotFluid.t_out);
    result.LMTD_required = calculateLMTD(hotFluid.t_in, hotFluid.t_out,
coldFluid.t_in, coldFluid.t_out);

    double f_hot = hotFluid.G / (hotFluid.rho * hotFluid.max_velocity);
    double f_cold = coldFluid.G / (coldFluid.rho * coldFluid.max_velocity);

    double be_hot = f_hot / geo.width_hot;
    double be_cold = f_cold / geo.width_cold;

    double be_min = std::max(be_hot, be_cold);

    auto it = std::find_if(standard_width.begin(), standard_width.end(),
    [&](double h) {
        if (h < be_min) return false;
        if (!geo.auto_width) {

```

```

        if (h < geo.width_min || h > geo.width_max) return false;
    }
    return true;
});

if (it == standard_width.end()) {
    if (geo.auto_width) {
        result.error_message = "Требуемая ширина канала превышает максимальный
стандартный размер";
    }
    else {
        int be_min_mm = static_cast<int>(round(be_min * 1000));
        result.error_message = "В заданном диапазоне ширины нет подходящего
стандартного значения"
            " (минимально необходимая: "
            + std::to_string(be_min_mm) + " мм)";
    }
    return result;
}
double be = *it;

double d_eq_cold = 2.0 * geo.width_cold * be / (geo.width_cold + be);
double d_eq_hot = 2.0 * geo.width_hot * be / (geo.width_hot + be);

result.velocity_cold = coldFluid.G / (coldFluid.rho * geo.width_cold * be);
result.Re_cold = result.velocity_cold * d_eq_cold * coldFluid.rho /
coldFluid.viscosity;

result.velocity_hot = hotFluid.G / (hotFluid.rho * geo.width_hot * be);
result.Re_hot = result.velocity_hot * d_eq_hot * hotFluid.rho /
hotFluid.viscosity;

double k = 0.0;
double k_old = -1.0;
double Dc = 0.0;

const int MAX_ITER = 500;
int iter = 0;

while (iter++ < MAX_ITER)
{
    double alpha_cold = calculateAlpha(coldFluid, result.velocity_cold,
d_eq_cold, Dc);
    double alpha_hot = calculateAlpha(hotFluid, result.velocity_hot, d_eq_hot,
Dc);

    if (alpha_cold <= 0.0 || alpha_hot <= 0.0) {
        result.error_message = "Коэффициент теплоотдачи обратился в ноль –
проверьте входные данные";
        return result;
    }

    k = 1.0 / (
        1.0 / alpha_hot +
        wallProperties.thickness / wallProperties.lambda +
        1.0 / alpha_cold +
        wallProperties.fouling_factor
    );

    double A = result.Q_required / (k * result.LMTD_required);
    double L = A / (2.0 * be);

```

```

        double t = wallProperties.thickness + std::max(geo.width_cold,
geo.width_hot);
        double x = ((geo.d_inner / t) - 1.0) / 2.0;

        int N = static_cast<int>(ceil(sqrt(2.0 * L / PI / t + x * x) - x));
        if (N % 2 == 1)
            N += 1;

        Dc = geo.d_inner + t + 2.0 * N * t;

        if (k_old > 0.0 && std::abs(k - k_old) / k_old < Eps)
        {
            result.alpha_cold = alpha_cold;
            result.alpha_hot = alpha_hot;
            result.channel_width = be;
            result.A = A;
            result.channel_length = L;
            result.turns = N;
            result.K = k;
            result.D = Dc;
            break;
        }

        k_old = k;

        if (iter == MAX_ITER) {
            result.error_message = "Итерационный расчёт не сошёлся за " +
std::to_string(MAX_ITER) + " итераций";
            return result;
        }
    }

    result.Q_real = result.K * result.LMTD_required * (2.0 * result.channel_width *
result.channel_length);
    result.t_hot_out_real = hotFluid.t_in - result.Q_real / (hotFluid.G *
hotFluid.cp);
    result.t_cold_out_real = coldFluid.t_in + result.Q_real / (coldFluid.G *
coldFluid.cp);
    result.LMTD_real = calculateLMTD(hotFluid.t_in, result.t_hot_out_real,
coldFluid.t_in, result.t_cold_out_real);

    double inlet_area_hot = hotFluid.G / (hotFluid.rho * result.velocity_hot);
    double inlet_area_cold = coldFluid.G / (coldFluid.rho * result.velocity_cold);
    double inlet_area_max = std::max(inlet_area_hot, inlet_area_cold);
    result.inlet_D = sqrt(inlet_area_max / PI) * 2.0;

    double lambda_cold = 0.3164 / pow(result.Re_cold, 0.25);
    double lambda_hot = 0.3164 / pow(result.Re_hot, 0.25);

    double d_eq_cold_final = 2.0 * geo.width_cold * result.channel_width /
(geo.width_cold + result.channel_width);
    double d_eq_hot_final = 2.0 * geo.width_hot * result.channel_width /
(geo.width_hot + result.channel_width);

    double dp_cold = lambda_cold * (result.channel_length / d_eq_cold_final)
        * (coldFluid.rho * result.velocity_cold * result.velocity_cold / 2.0);
    double dp_hot = lambda_hot * (result.channel_length / d_eq_hot_final)
        * (hotFluid.rho * result.velocity_hot * result.velocity_hot / 2.0);

    result.power_cold = coldFluid.G * dp_cold / (coldFluid.rho * n);

```

```

        result.power_hot = hotFluid.G * dp_hot / (hotFluid.rho * n);

        result.geometry = geo;
        result.wall = wallProperties;

        return result;
    }

    double SpiralHeatExchanger::calculateLMTD(double t_hot_in, double t_hot_out,
        double t_cold_in, double t_cold_out) const
    {
        double dt1 = std::abs(t_hot_in - t_cold_out);
        double dt2 = std::abs(t_hot_out - t_cold_in);

        if (std::abs(dt1 - dt2) < 1e-9)
            return dt1;

        double larger = std::max(dt1, dt2);
        double smaller = std::min(dt1, dt2);
        return (larger - smaller) / log(larger / smaller);
    }

    double SpiralHeatExchanger::calculateAlpha(const Fluid& f, double velocity,
        double d_eq, double Dc) const
    {
        if (velocity <= 0.0)
            return 0.0;

        double Re = velocity * d_eq * f.rho / f.viscosity;
        if (Re <= ReConst1)
            return 0.0;

        double Pr = f.cp * f.viscosity / f.lambda;
        double a_coof = calculateAlphaCoof(d_eq, Dc);
        double Nu = 0.023 * pow(Re, 0.8) * pow(Pr, 0.4) * a_coof;

        return Nu * f.lambda / d_eq;
    }

    double SpiralHeatExchanger::calculateAlphaCoof(double de, double Dc) const
    {
        if (Dc <= 0.0)
            return 1.1;

        return 1.0 + 3.54 * de / Dc;
    }

    std::string SpiralHeatExchanger::validateInputs()
    {
        if (hotFluid.G <= 0 || coldFluid.G <= 0)
            return "Массовый расход должен быть положительным";
        if (hotFluid.t_in <= hotFluid.t_out)
            return "Температура входа горячей среды должна быть выше выхода";
        if (coldFluid.t_in >= coldFluid.t_out)
            return "Температура входа холодной среды должна быть ниже выхода";

        return "";
    }
}

```

Код файла ModelBuilder.h

```

#pragma once

#include "solid.h"
#include "assembly.h"
#include "SpiralHeatExchanger.h"
#include <vector>
#include <functional>
#include <QString>

class ModelBuilder {
public:
    using ProgressCallback = std::function<void(int, int, const QString&)>;

    void setProgressCallback(ProgressCallback cb) { m_progressCallback = cb; }

    SPtr<MbAssembly> BuildModel(const CalculationResult& res);

private:
    ProgressCallback m_progressCallback;

    void reportProgress(int step, int total, const QString& description) {
        if (m_progressCallback)
            m_progressCallback(step, total, description);
    }

private:
    MbSolid* CreateSpiral(double d_inner, double turns, double thickness,
        double gap_1, double gap_2, double channel_width, bool is_second_spiral);

    MbSolid* CreateSpiralDivider(double d_inner, double thickness,
        double channel_width, double gap_1, double gap_2, bool is_second);

    MbSolid* CreateShell(double d_in_shell, double shell_thickness,
        double channel_width, double gasket_length, double hole_diameter);

    MbSolid* CreateNozzle(const MbPlacement3D& placement,
        double d_in, double wall_thickness, double pipe_len,
        double flange_d, double flange_thickness,
        double hole_pcd, int num_holes, double hole_d, double f);

    MbSolid* CreateShellFlange(double d_in_shell, double shell_thickness,
        double channel_width, double gasket_length,
        int hole_count, double d_hole, double flange_width, double flange_height);

    MbSolid* CreateGasket(double d_in_shell, double r_start,
        double gasket_length, double nozzle_d_in);

    MbSolid* CreateCover(double d_in_shell, double r_in,
        double shell_thickness, double cap_thickness,
        double flange_height,
        int hole_count, double d_hole,
        double nozzle_d_in, double nozzle_wall, double nozzle_len,
        double nozzle_flange_d, double nozzle_flange_t,
        double nozzle_hole_pcd, int nozzle_num_holes, double nozzle_hole_d,
        double nozzle_chamfer);

    SPtr<MbAssembly> AssembleHeatExchanger(

```

```

    MbSolid* resultBody,
    MbSolid* gasket1, MbSolid* gasket2,
    MbSolid* cover1, MbSolid* cover2,
    double d_in_shell, double s_tck,
    double channel_width, double gasket_length,
    double flange_height);
};

```

Код файла ModelBuilder.cpp

```

#include "ModelBuilder.h"

#include <action_general.h>
#include <action_solid.h>

#include <templ_s_array.h>
#include <templ_rp_array.h>
#include <templ_sptr.h>

#include <mb_cart_point.h>
#include <mb_cart_point3d.h>
#include <mb_matrix3d.h>
#include <mb_placement3d.h>
#include <mb_axis3d.h>
#include <surf_cylinder_surface.h>

#include <cur_line_segment.h>
#include <cur_nurbs.h>
#include <cur_cubic_spline.h>
#include <cur_contour.h>
#include <cur_arc.h>

#include <surface.h>
#include <surf_plane.h>

#include <name_item.h>
#include <creator.h>
#include <solid.h>

#include "assembly.h"
#include <constraint.h>
#include "gcm_types.h"
#include <attr_product.h>

#include <last.h>

using namespace std;
using namespace c3d;

SPtr<MbAssembly> ModelBuilder::BuildModel(const CalculationResult& res)
{
    SPtr<MbAssembly> result;

    if (res.geometry.d_inner <= 0 || res.turns <= 0 || res.A <= 0)
        return result;

    double t = res.turns;
    double din = res.geometry.d_inner * 1000.0;
    double tck = res.wall.thickness * 1000.0;
    double w = res.channel_width * 1000.0;
    double gap_1 = res.geometry.width_hot * 1000.0;

```

```

double gap_2 = res.geometry.width_cold * 1000.0;
double s_tck = res.geometry.shell_thickness * 1000.0;

double r_in = din / 2.0;
double step_per_turn = (tck * 2.0) + gap_1 + gap_2;
double r_end_axis = r_in + (tck / 2.0) + (step_per_turn * t);
double r_outer_metal = r_end_axis + (tck / 2.0) + step_per_turn / 4.0;
double d_in_shell = r_outer_metal * 2.0;

double nozzle_d_in = res.selected_flange.inner_d;
double nozzle_flange_d = res.selected_flange.outer_d;
double nozzle_flange_t = res.selected_flange.flange_thickness;
double nozzle_hole_pcd = res.selected_flange.bolt_circle_d;
double nozzle_hole_d = res.selected_flange.bolt_d;
int nozzle_num_holes = res.selected_flange.bolt_count;
double nozzle_chamfer = 8.0;

double nozzle_wall = res.nozzle_shell_thickness;
double nozzle_len = res.nozzle_shell_length;

const int sf_hole_count = res.shell_flange.bolt_count;
const double sf_d_hole = res.shell_flange.bolt_d;
const double sf_flange_width = res.shell_flange.flange_thickness;
const double sf_flange_height = (res.shell_flange.outer_d -
res.shell_flange.inner_d) / 2.0;

const double gasket_length = res.gasket_thickness * 1000.0;

const int totalSteps = 12;
int step = 0;

reportProgress(step++, totalSteps, "Создание первой спирали...");
MbSolid* spiral1 = CreateSpiral(din, t, tck, gap_1, gap_2, w, false);

reportProgress(step++, totalSteps, "Создание второй спирали...");
MbSolid* spiral2 = CreateSpiral(din, t, tck, gap_1, gap_2, w, true);

reportProgress(step++, totalSteps, "Создание центральных перегородок...");
MbSolid* kern1 = CreateSpiralDivider(din, tck, w, gap_1, gap_2, false);
MbSolid* kern2 = CreateSpiralDivider(din, tck, w, gap_1, gap_2, true);

reportProgress(step++, totalSteps, "Создание корпуса...");
MbSolid* shell = CreateShell(d_in_shell, s_tck, w, gasket_length, nozzle_d_in);

reportProgress(step++, totalSteps, "Создание фланцев корпуса...");
MbSolid* flange1 = CreateShellFlange(d_in_shell, s_tck, w, gasket_length,
sf_hole_count, sf_d_hole, sf_flange_width, sf_flange_height);
MbSolid* gasket1 = CreateGasket(d_in_shell, r_in, gasket_length, nozzle_d_in);
MbSolid* gasket2 = CreateGasket(d_in_shell, r_in, gasket_length, nozzle_d_in);

MbSNameMaker symNames(ct_SymmetrySolid, MbSNameMaker::i_SideNone);
MbPlacement3D plSym;
SolidSPtr flange2Result;
::SymmetrySolid(*flange1, cm_Copy,
MbSymmetrySolidParams(plSym, symNames, ts_neutral),
flange2Result);
MbSolid* flange2 = flange2Result.detach();

reportProgress(step++, totalSteps, "Создание патрубков...");
double R_ext = r_outer_metal + s_tck;
double overlap = 2.0;

```



```

MbPlacement3D posForward(
    MbVector3D(0, 1, 0), MbVector3D(0, 0, 1),
    MbCartPoint3D(R_ext - overlap, 0, 0));
MbSolid* nozzle1 = CreateNozzle(posForward,
    nozzle_d_in, nozzle_wall, nozzle_len,
    nozzle_flange_d, nozzle_flange_t,
    nozzle_hole_pcd, nozzle_num_holes, nozzle_hole_d, nozzle_chamfer);

MbPlacement3D posBackward(
    MbVector3D(0, -1, 0), MbVector3D(0, 0, 1),
    MbCartPoint3D(-(R_ext - overlap), 0, 0));
MbSolid* nozzle2 = CreateNozzle(posBackward,
    nozzle_d_in, nozzle_wall, nozzle_len,
    nozzle_flange_d, nozzle_flange_t,
    nozzle_hole_pcd, nozzle_num_holes, nozzle_hole_d, nozzle_chamfer);

reportProgress(step++, totalSteps, "Объединение тел корпуса...");
RPArray<MbSolid> parts;
if (spiral1) parts.Add(spiral1);
if (spiral2) parts.Add(spiral2);
if (kern1) parts.Add(kern1);
if (kern2) parts.Add(kern2);
if (shell) parts.Add(shell);
if (flange1) parts.Add(flange1);
if (flange2) parts.Add(flange2);
if (nozzle1) parts.Add(nozzle1);
if (nozzle2) parts.Add(nozzle2);

MbSolid* bodyPtr = nullptr;
MbSNameMaker names(ct_BooleanSolid, MbSNameMaker::i_SideNone, 0);
::UnionSolid(parts, cm_Same, true, names, false, bodyPtr);

reportProgress(step++, totalSteps, "Создание крышек...");
double cap_thickness = res.geometry.cover_thickness * 1000.0;

MbSolid* cap1 = CreateCover(d_in_shell, r_in, s_tck, cap_thickness,
    sf_flange_height, sf_hole_count, sf_d_hole,
    nozzle_d_in, nozzle_wall, nozzle_len,
    nozzle_flange_d, nozzle_flange_t,
    nozzle_hole_pcd, nozzle_num_holes, nozzle_hole_d, nozzle_chamfer);

reportProgress(step++, totalSteps, "Создание крышек...");
MbSolid* cap2 = CreateCover(d_in_shell, r_in, s_tck, cap_thickness,
    sf_flange_height, sf_hole_count, sf_d_hole,
    nozzle_d_in, nozzle_wall, nozzle_len,
    nozzle_flange_d, nozzle_flange_t,
    nozzle_hole_pcd, nozzle_num_holes, nozzle_hole_d, nozzle_chamfer);

reportProgress(step++, totalSteps, "Сборка теплообменника...");
result = AssembleHeatExchanger(
    bodyPtr, gasket1, gasket2, cap1, cap2,
    d_in_shell, s_tck, w, gasket_length, sf_flange_height);

reportProgress(totalSteps, totalSteps, "Готово");
return result;
}

MbSolid* ModelBuilder::CreateSpiral(double d_inner, double turns, double thickness,
    double gap_1, double gap_2, double channel_width, bool is_second_spiral)
{

```

```

double r_start = d_inner / 2.0;
double step_per_turn = (thickness * 2.0) + gap_1 + gap_2;
double radial_correction = (gap_2 - gap_1) / 2.0;
double current_r_start = is_second_spiral ? (r_start + radial_correction) :
r_start;
double phase_shift = is_second_spiral ? M_PI : 0.0;

size_t points_per_turn = 64;
size_t total_points = static_cast<size_t>(points_per_turn * turns) + 1;

SArray<MbCartPoint> points(total_points, 1);
double delta_phi = M_PI2 / (double)points_per_turn;

for (size_t i = 0; i < total_points; ++i) {
    double phi = i * delta_phi;
    double r = current_r_start + (thickness / 2.0) + (step_per_turn * phi /
M_PI2);
    points.push_back(MbCartPoint(r * cos(phi + phase_shift), r * sin(phi +
phase_shift)));
}

MbPlacement3D place;
SPtr<MbCurve> spiral(MbCubicSpline::Create(points, false));
SPtr<MbContour> contour(new MbContour);
contour->AddSegment(spiral);

double t_end = spiral->GetTMax();
MbCartPoint p_end;
MbVector tangent;
spiral->GetLimitPoint(2, p_end);
spiral->Tangent(t_end, tangent);
tangent.Normalize();

double r_target = r_start + (step_per_turn * (turns + 0.5)) + thickness;
double b = 2.0 * (p_end.x * tangent.x + p_end.y * tangent.y);
double c = (p_end.x * p_end.x + p_end.y * p_end.y) - (r_target * r_target);
double D = b * b - 4.0 * c;
double tail_length = (D >= 0) ? (-b + sqrt(D)) / 2.0 : (thickness + gap_1);

contour->AddSegment(new MbLineSegment(p_end,
    MbCartPoint(p_end.x + tangent.x * tail_length, p_end.y + tangent.y *
tail_length)));

RPAArray<MbContour> contours(1, 1);
contours.push_back(contour);
SPtr<MbSurface> surface(new MbPlane(place));
MbSweptData curvesData(*surface, contours);

ExtrusionValues extrudeParams;
extrudeParams.side1.scalarValue = channel_width / 2;
extrudeParams.side1.way = sw_scalarValue;
extrudeParams.side2.scalarValue = channel_width / 2;
extrudeParams.side2.way = sw_scalarValue;
extrudeParams.thickness1 = thickness / 2.0;
extrudeParams.thickness2 = thickness / 2.0;

MbSNameMaker snMakerExtrude(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
PArray<MbSNameMaker> cNames(0, 1, false);

MbSolid* surfaceSolid = nullptr;
::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,

```

```

        extrudeParams, snMakerExtrude, cNames, surfaceSolid);

    return surfaceSolid;
}

MbSolid* ModelBuilder::CreateSpiralDivider(double d_inner, double thickness,
double channel_width, double gap_1, double gap_2, bool is_second)
{
    double radial_correction = (gap_2 - gap_1) / 2.0;

    SArray<MbCartPoint> rectPoints(4, 1);

    if (is_second) {
        double x_start = d_inner / 2.0 + thickness;
        double x_end = -(d_inner / 2.0 + radial_correction) - thickness / 2;
        rectPoints.push_back(MbCartPoint(x_start, 0));
        rectPoints.push_back(MbCartPoint(x_start, -thickness / 2.0));
        rectPoints.push_back(MbCartPoint(x_end, -thickness / 2.0));
        rectPoints.push_back(MbCartPoint(x_end, 0));
    }
    else {
        double x_start = d_inner / 2.0 + thickness / 2;
        double x_end = -(d_inner / 2.0 + radial_correction) - thickness;
        rectPoints.push_back(MbCartPoint(x_start, 0));
        rectPoints.push_back(MbCartPoint(x_start, thickness / 2.0));
        rectPoints.push_back(MbCartPoint(x_end, thickness / 2.0));
        rectPoints.push_back(MbCartPoint(x_end, 0));
    }

    SPtr<MbContour> contour(new MbContour());
    for (size_t i = 0; i < 4; ++i)
        contour->AddSegment(new MbLineSegment(rectPoints[i], rectPoints[(i + 1) %
4]));

    MbPlacement3D place;
    RPAArray<MbContour> contours(1, 1);
    contours.push_back(contour);
    SPtr<MbSurface> surface(new MbPlane(place));
    MbSweptData curvesData(*surface, contours);

    ExtrusionValues extrudeParams;
    extrudeParams.side1.scalarValue = channel_width / 2;
    extrudeParams.side1.way = sw_scalarValue;
    extrudeParams.side2.scalarValue = channel_width / 2;
    extrudeParams.side2.way = sw_scalarValue;

    MbSolid* dividerSolid = nullptr;
    MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
    PArray<MbSNameMaker> cNames(0, 1, false);
    ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
        extrudeParams, snMaker, cNames, dividerSolid);

    return dividerSolid;
}

MbSolid* ModelBuilder::CreateShell(double d_in_shell, double shell_thickness,
double channel_width, double gasket_length, double hole_diameter)
{
    double r_mid = d_in_shell / 2.0 + shell_thickness / 2.0;

    SPtr<MbArc> circle(new MbArc(r_mid));

```

```

SPtr<MbContour> contour(new MbContour());
contour->AddSegment(circle);

RPArray<MbContour> contours(1, 1);
contours.push_back(contour);

MbPlacement3D place;
SPtr<MbSurface> surface(new MbPlane(place));
MbSweptData curvesData(*surface, contours);

ExtrusionValues extrudeParams;
extrudeParams.side1.scalarValue = channel_width / 2 + gasket_length;
extrudeParams.side1.way = sw_scalarValue;
extrudeParams.side2.scalarValue = channel_width / 2 + gasket_length;
extrudeParams.side2.way = sw_scalarValue;
extrudeParams.thickness1 = shell_thickness / 2.0;
extrudeParams.thickness2 = shell_thickness / 2.0;

MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
PArray<MbSNameMaker> cNames(0, 1, false);

MbSolid* shellSolid = nullptr;
::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
    extrudeParams, snMaker, cNames, shellSolid);

if (shellSolid == nullptr || hole_diameter <= 0)
    return shellSolid;

MbPlacement3D holePlace;
holePlace.Rotate(MbAxis3D::yAxis, M_PI_2);

MbSNameMaker snHole(ct_ElementarySolid, MbSNameMaker::i_SideNone, 1);
MbElementarySolidParams cylParams(snHole);

double halfLength = r_mid + shell_thickness / 2.0 + 10.0;
cylParams.InitCylinder(holePlace, hole_diameter / 2.0, halfLength, halfLength);

MbElementarySolidResults results;
if (::ElementarySolid(cylParams, results) != rt_Success)
    return shellSolid;

SolidSPtr holeTool(results.DetachSolid());
if (!holeTool)
    return shellSolid;

MbSNameMaker boolSnMaker(ct_BooleanSolid, MbSNameMaker::i_SideNone, 0);
MbBooleanOperationParams pDiff(bo_Difference, true, boolSnMaker);
SolidSPtr shellPtr(shellSolid);
SolidSPtr boolResult;

if (::BooleanResult(shellPtr, cm_Same, holeTool, cm_Copy, pDiff, boolResult) ==
rt_Success && boolResult) {
    shellPtr = nullptr;
    shellSolid = boolResult.detach();
}
else {
    shellSolid = shellPtr.detach();
}

return shellSolid;
}

```

```

MbSolid* ModelBuilder::CreateNozzle(const MbPlacement3D& placement,
    double d_in, double wall_thickness, double pipe_len,
    double flange_d, double flange_thickness,
    double hole_pcd, int num_holes, double hole_d, double f)
{
    double r_pipe_mid = (d_in + wall_thickness) / 2.0;
    SPtr<MbArc> pipeCircle(new MbArc(MbCartPoint(0, 0), r_pipe_mid));

    SPtr<MbContour> pipeContour(new MbContour());
    pipeContour->AddSegment(pipeCircle);

    RPArray<MbContour> pipeContours(1, 1);
    pipeContours.push_back(pipeContour);

    SPtr<MbSurface> pipeSurface(new MbPlane(placement));
    MbSweptData pipeData(*pipeSurface, pipeContours);

    ExtrusionValues pipeParams;
    pipeParams.side1.scalarValue = pipe_len;
    pipeParams.side1.way = sw_scalarValue;
    pipeParams.thickness1 = wall_thickness / 2.0;
    pipeParams.thickness2 = wall_thickness / 2.0;

    MbSNameMaker snPipe(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 1);
    MbSolid* pipeSolid = nullptr;
    ::ExtrusionSolid(pipeData, placement.Normal(), nullptr, nullptr, false,
        pipeParams, snPipe, PArray<MbSNameMaker>(0, 1, false), pipeSolid);

    MbPlacement3D flangePlace(placement);
    MbVector3D offset = flangePlace.Normal();
    offset *= (pipe_len - flange_thickness);
    flangePlace.Move(offset);

    double flange_width = (flange_d - d_in) / 2.0;
    double r_flange_mid = (d_in + flange_width) / 2.0;

    SPtr<MbArc> flangeCircle(new MbArc(MbCartPoint(0, 0), r_flange_mid));
    SPtr<MbContour> flangeContour(new MbContour());
    flangeContour->AddSegment(flangeCircle);

    RPArray<MbContour> flangeContours(1, 1);
    flangeContours.push_back(flangeContour);

    SPtr<MbSurface> flangeSurface(new MbPlane(flangePlace));
    MbSweptData flangeData(*flangeSurface, flangeContours);

    ExtrusionValues flangeParams;
    flangeParams.side1.scalarValue = flange_thickness;
    flangeParams.side1.way = sw_scalarValue;
    flangeParams.thickness1 = flange_width / 2.0;
    flangeParams.thickness2 = flange_width / 2.0;

    MbSNameMaker snFlange(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 2);
    MbSolid* flangeSolid = nullptr;
    ::ExtrusionSolid(flangeData, flangePlace.Normal(), nullptr, nullptr, false,
        flangeParams, snFlange, PArray<MbSNameMaker>(0, 1, false), flangeSolid);

    MbSolid* nozzleBody = nullptr;
    RPArray<MbSolid> parts(2, 1);
    parts.Add(pipeSolid);
}

```

```

parts.Add(flangeSolid);
MbSNameMaker snUnion(ct_BooleanSolid, MbSNameMaker::i_SideNone, 10);
::UnionSolid(parts, cm_Same, true, snUnion, false, nozzleBody);

if (nozzleBody && num_holes > 0) {
    RPAarray<MbContour> holeContours(num_holes, 1);
    double angle_step = M_PI2 / (double)num_holes;

    for (int i = 0; i < num_holes; ++i) {
        double angle = i * angle_step;
        double x = (hole_pcd / 2.0) * cos(angle);
        double y = (hole_pcd / 2.0) * sin(angle);

        MbArc* holeCircle = new MbArc(MbCartPoint(x, y), hole_d / 2.0);
        MbContour* cnt = new MbContour();
        cnt->AddSegment(holeCircle);
        holeContours.push_back(cnt);
    }

    MbPlacement3D holePlace(flangePlace);
    holePlace.Move(placement.Normal() * -1.0);

    SPtr<MbSurface> holeSurf(new MbPlane(holePlace));
    MbSweptData holeData(*holeSurf, holeContours);

    ExtrusionValues holeParams;
    holeParams.side1.scalarValue = flange_thickness + 2.0;
    holeParams.side1.way = sw_scalarValue;

    MbSNameMaker snHoles(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 3);
    MbSolid* holesTool = nullptr;
    ::ExtrusionSolid(holeData, holePlace.Normal(), nullptr, nullptr, false,
        holeParams, snHoles, PArray<MbSNameMaker>(0, 1, false), holesTool);

    if (holesTool) {
        MbSNameMaker bNames(ct_BooleanSolid, MbSNameMaker::i_SideNone, 20);
        MbBooleanOperationParams bParams(bo_Difference, true, bNames);
        SolidSPtr solid1(nozzleBody);
        SolidSPtr solid2(holesTool);
        SolidSPtr resultBody;
        if (::BooleanResult(solid1, cm_Same, solid2, cm_Same, bParams,
resultBody) == rt_Success)
            if (resultBody != nullptr)
                nozzleBody = resultBody.detach();
    }
}

return nozzleBody;
}

MbSolid* ModelBuilder::CreateShellFlange(double d_in_shell, double shell_thickness,
double channel_width, double gasket_length,
int hole_count, double d_hole, double flange_width, double flange_height)
{
    double r_outer = d_in_shell / 2.0 + shell_thickness;
    double r_flange_mid = r_outer + flange_height / 2;

    SPtr<MbArc> circle1(new MbArc(r_flange_mid));
    SPtr<MbContour> contour(new MbContour());
    contour->AddSegment(circle1);

```

```

RPArray<MbContour> contours(1, 1);
contours.push_back(contour);

MbPlacement3D place;
SPtr<MbSurface> surface(new MbPlane(place));
MbSweptData curvesData(*surface, contours);

ExtrusionValues extrudeParams;
extrudeParams.side1.scalarValue = channel_width / 2 + gasket_length;
extrudeParams.side1.way = sw_scalarValue;
extrudeParams.side2.scalarValue = -channel_width / 2 - gasket_length +
flange_width;
extrudeParams.side2.way = sw_scalarValue;
extrudeParams.thickness1 = flange_height / 2;
extrudeParams.thickness2 = flange_height / 2;

MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 0);
PArray<MbSNameMaker> cNames(0, 1, false);

MbSolid* flangeSolid = nullptr;
::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
    extrudeParams, snMaker, cNames, flangeSolid);

if (flangeSolid && hole_count > 0) {
    double angle_step = M_PI2 / (double)hole_count;
    double hole_pcd = r_flange_mid;

    RPArray<MbContour> holeContours(hole_count, 1);
    for (int i = 0; i < hole_count; ++i) {
        double angle = i * angle_step;
        MbArc* hc = new MbArc(MbCartPoint(hole_pcd * cos(angle), hole_pcd *
sin(angle)), d_hole / 2.0);
        MbContour* cnt = new MbContour();
        cnt->AddSegment(hc);
        holeContours.push_back(cnt);
    }

    MbPlacement3D holePlace(place);
    holePlace.Move(place.Normal() * (channel_width / 2 + gasket_length));

    SPtr<MbSurface> holeSurf(new MbPlane(holePlace));
    MbSweptData holeData(*holeSurf, holeContours);

    ExtrusionValues holeParams;
    holeParams.side1.way = sw_scalarValue;
    holeParams.side1.scalarValue = flange_width;

    MbSNameMaker snHoles(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 3);
    MbSolid* holesTool = nullptr;
    ::ExtrusionSolid(holeData, holePlace.Normal() * -1.0, nullptr, nullptr,
false,
        holeParams, snHoles, PArray<MbSNameMaker>(0, 1, false), holesTool);

    if (holesTool) {
        SolidSPtr resultBody;
        MbSNameMaker bNames(ct_BooleanSolid, MbSNameMaker::i_SideNone, 20);
        MbBooleanOperationParams bParams(bo_Difference, true, bNames);
        SolidSPtr solid1(flangeSolid);
        SolidSPtr solid2(holesTool);
        if (::BooleanResult(solid1, cm_Same, solid2, cm_Same, bParams,
resultBody) == rt_Success)

```



```

        flangeSolid = resultBody.detach();
    }
}

return flangeSolid;
}

MbSolid* ModelBuilder::CreateGasket(double d_in_shell, double r_start,
double gasket_length, double nozzle_d_in)
{
    double r_outer_metal = d_in_shell / 2.0;

    MbPlacement3D place;

    SPtr<MbArc> circle(new MbArc(r_outer_metal));
    SPtr<MbContour> contour(new MbContour());
    contour->AddSegment(circle);

    RPArray<MbContour> contours(1, 1);
    contours.push_back(contour);

    SPtr<MbSurface> surface(new MbPlane(place));
    MbSweptData curvesData(*surface, contours);

    ExtrusionValues extrudeParams;
    extrudeParams.side1.scalarValue = gasket_length;
    extrudeParams.side1.way = sw_scalarValue;
    extrudeParams.side2.scalarValue = 0.0;
    extrudeParams.side2.way = sw_scalarValue;

    MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone,
0);
    PArray<MbSNameMaker> cNames(0, 1, false);

    MbSolid* gasketSolid = nullptr;
    MbResultType res = ::ExtrusionSolid(curvesData, place.Normal(),
        nullptr, nullptr, false,
        extrudeParams, snMaker, cNames,
        gasketSolid);
    if (!gasketSolid)
        return nullptr;

    if (nozzle_d_in > 0.0)
    {
        double r_hole = r_start / 2.0;
        double z_mid = gasket_length / 2.0;
        double halfLen = gasket_length / 2.0 + 1.0;

        MbPlacement3D cylPlace;
        cylPlace.Move(MbVector3D(0.0, r_hole, z_mid));

        MbSNameMaker snCyl(ct_ElementarySolid, MbSNameMaker::i_SideNone, 1);
        MbElementarySolidParams cylParams(snCyl);
        cylParams.InitCylinder(cylPlace, nozzle_d_in / 2.0, halfLen, halfLen);

        MbElementarySolidResults cylResults;
        if (::ElementarySolid(cylParams, cylResults) == rt_Success)
        {
            MbSolid* holeTool = cylResults.GetResultSolid();

            SolidSPtr resultBody;

```



```

        MbSNameMaker bNames(ct_BooleanSolid, MbSNameMaker::i_SideNone, 20);
        MbBooleanOperationParams bParams(bo_Difference, true, bNames);
        SolidSPtr solid1(gasketSolid);
        SolidSPtr solid2(holeTool);
        if (::BooleanResult(solid1, cm_Same, solid2, cm_Same,
            bParams, resultBody) == rt_Success)
            gasketSolid = resultBody.detach();
    }
}

return gasketSolid;
}

MbSolid* ModelBuilder::CreateCover(double d_in_shell, double r_in,
    double shell_thickness, double cap_thickness,
    double flange_height,
    int hole_count, double d_hole,
    double nozzle_d_in, double nozzle_wall, double nozzle_len,
    double nozzle_flange_d, double nozzle_flange_t,
    double nozzle_hole_pcd, int nozzle_num_holes, double nozzle_hole_d,
    double nozzle_chamfer)
{
    double r_outer = d_in_shell / 2.0 + shell_thickness;
    double r_cap = r_outer + flange_height;

    MbPlacement3D place;

    SPtr<MbArc> circle(new MbArc(r_cap));
    SPtr<MbContour> contour(new MbContour());
    contour->AddSegment(circle);

    RPArray<MbContour> contours(1, 1);
    contours.push_back(contour);

    SPtr<MbSurface> surface(new MbPlane(place));
    MbSweptData curvesData(*surface, contours);

    ExtrusionValues extrudeParams;
    extrudeParams.side1.scalarValue = cap_thickness;
    extrudeParams.side1.way = sw_scalarValue;
    extrudeParams.side2.scalarValue = 0.0;
    extrudeParams.side2.way = sw_scalarValue;

    MbSNameMaker snMaker(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone,
0);
    PArray<MbSNameMaker> cNames(0, 1, false);

    MbSolid* capSolid = nullptr;
    ::ExtrusionSolid(curvesData, place.Normal(), nullptr, nullptr, false,
        extrudeParams, snMaker, cNames, capSolid);

    if (!capSolid) return nullptr;

    if (hole_count > 0)
    {
        double angle_step = M_PI2 / (double)hole_count;
        RPArray<MbContour> holeContours(hole_count, 1);

        for (int i = 0; i < hole_count; ++i)
        {
            double angle = i * angle_step;

```

```

        double hole_r = r_outer + flange_height / 2.0;
        MbArc* hc = new MbArc(MbCartPoint(hole_r * cos(angle), hole_r *
sin(angle)), d_hole / 2.0);
        MbContour* cnt = new MbContour();
        cnt->AddSegment(hc);
        holeContours.push_back(cnt);
    }

    MbPlacement3D holePlace(place);
    holePlace.Move(place.Normal() * -1.0);

    SPtr<MbSurface> holeSurf(new MbPlane(holePlace));
    MbSweptData holeData(*holeSurf, holeContours);

    ExtrusionValues holeParams;
    holeParams.side1.scalarValue = cap_thickness + 2.0;
    holeParams.side1.way = sw_scalarValue;

    MbSNameMaker snHoles(ct_CurveExtrusionSolid, MbSNameMaker::i_SideNone, 1);
    MbSolid* holesTool = nullptr;
    ::ExtrusionSolid(holeData, holePlace.Normal(), nullptr, nullptr, false,
        holeParams, snHoles, PArray<MbSNameMaker>(0, 1, false), holesTool);

    if (holesTool)
    {
        MbSNameMaker          bNames(ct_BooleanSolid,
MbSNameMaker::i_SideNone, 20);
        MbBooleanOperationParams bParams(bo_Difference, true, bNames);
        SolidSPtr solid1(capSolid);
        SolidSPtr solid2(holesTool);
        SolidSPtr resultBody;
        if (::BooleanResult(solid1, cm_Same, solid2, cm_Same, bParams,
resultBody) == rt_Success && resultBody)
        {
            solid1 = nullptr;
            capSolid = resultBody.detach();
        }
        else
        {
            capSolid = solid1.detach();
        }
    }
}

if (nozzle_d_in > 0.0)
{
    double r_hole = r_in / 2.0;
    double z_mid = cap_thickness / 2.0;
    double halfLen = cap_thickness / 2.0 + 1.0;

    MbPlacement3D cylPlace;
    cylPlace.Move(MbVector3D(0.0, r_hole, z_mid));

    MbSNameMaker          snCyl(ct_ElementarySolid, MbSNameMaker::i_SideNone,
2);
    MbElementarySolidParams cylParams(snCyl);
    cylParams.InitCylinder(cylPlace, nozzle_d_in / 2.0, halfLen, halfLen);

    MbElementarySolidResults cylResults;
    if (::ElementarySolid(cylParams, cylResults) == rt_Success)
    {

```

```

        SolidSPtr holeTool(cylResults.DetachSolid());
        if (holeTool)
        {
            MbSNameMaker          bNames(ct_BooleanSolid,
MbSNameMaker::i_SideNone, 21);
            MbBooleanOperationParams bParams(bo_Difference, true, bNames);
            SolidSPtr solid1(capSolid);
            SolidSPtr resultBody;
            if (::BooleanResult(solid1, cm_Same, holeTool, cm_Copy, bParams,
resultBody) == rt_Success && resultBody)
            {
                solid1 = nullptr;
                capSolid = resultBody.detach();
            }
            else
            {
                capSolid = solid1.detach();
            }
        }
    }

    MbVector3D    axisX(1.0, 0.0, 0.0);
    MbVector3D    axisY(0.0, 1.0, 0.0);
    MbCartPoint3D nozzleOrigin(0.0, r_hole, 0);

    MbPlacement3D nozzlePlace(axisX, axisY, nozzleOrigin);
    MbSolid* nozzle = CreateNozzle(nozzlePlace,
        nozzle_d_in, nozzle_wall, nozzle_len,
        nozzle_flange_d, nozzle_flange_t,
        nozzle_hole_pcd, nozzle_num_holes, nozzle_hole_d,
        nozzle_chamfer);

    if (nozzle)
    {
        RPArray<MbSolid> parts(2, 1);
        parts.Add(capSolid);
        parts.Add(nozzle);
        MbSolid* united = nullptr;
        MbSNameMaker snUnion(ct_BooleanSolid, MbSNameMaker::i_SideNone, 30);
        ::UnionSolid(parts, cm_Same, true, snUnion, false, united);
        if (united) capSolid = united;
    }
}

return capSolid;
}

static MbFace* FindCylindricalFace(MbSolid* solid, double radius, double tol = 1.0)
{
    for (size_t i = 0; i < solid->GetFacesCount(); ++i)
    {
        MbFace* face = solid->GetFace(i);
        if (!face) continue;
        const MbSurface& surf = face->GetSurface().GetSurface();
        if (surf.IsA() == st_CylinderSurface)
        {
            const MbCylinderSurface& cyl = static_cast<const
MbCylinderSurface&>(surf);
            if (fabs(cyl.GetRadius() - radius) < tol)
                return face;
        }
    }
}

```

```

    }
    return nullptr;
}

static MbFace* FindPlanarFaceAtZ(MbSolid* solid, double z, double tol = 1.0)
{
    for (size_t i = 0; i < solid->GetFacesCount(); ++i)
    {
        MbFace* face = solid->GetFace(i);
        if (!face) continue;
        const MbSurface& surf = face->GetSurface().GetSurface();
        if (surf.IsA() == st_Plane)
        {
            const MbPlane& plane = static_cast<const MbPlane&>(surf);

            MbVector3D normal;
            plane._Normal(0.0, 0.0, normal);
            if (fabs(fabs(normal.z) - 1.0) > 0.01)
                continue;

            MbCartPoint3D origin;
            plane._PointOn(0.0, 0.0, origin);
            if (fabs(origin.z - z) < tol)
                return face;
        }
    }
    return nullptr;
}

SPtr<MbAssembly> ModelBuilder::AssembleHeatExchanger(
    MbSolid* resultBody,
    MbSolid* gasket1,
    MbSolid* gasket2,
    MbSolid* cover1,
    MbSolid* cover2,
    double d_in_shell,
    double s_tck,
    double channel_width,
    double gasket_length,
    double flange_height)
{
    const double L = channel_width / 2.0 + gasket_length;
    const MbVector3D axX(1, 0, 0);
    const MbVector3D axYp(0, 1, 0);
    const MbVector3D axYm(0, -1, 0);
    SPtr<MbInstance> compBody(
        new MbInstance(*resultBody, MbPlacement3D()));
    SPtr<MbInstance> compGasket1(
        new MbInstance(*gasket1,
            MbPlacement3D(axX, axYp, MbCartPoint3D(0, 0, channel_width / 2))));
    SPtr<MbInstance> compGasket2(
        new MbInstance(*gasket2,
            MbPlacement3D(axX, axYm, MbCartPoint3D(0, 0, -channel_width / 2))));
    SPtr<MbInstance> compCover1(
        new MbInstance(*cover1,
            MbPlacement3D(axX, axYp, MbCartPoint3D(0, 0, L))));
    SPtr<MbInstance> compCover2(
        new MbInstance(*cover2,
            MbPlacement3D(axX, axYm, MbCartPoint3D(0, 0, -L))));
    std::vector<SPtr<MbInstance>> parts;
    parts.push_back(compBody);

```

```

parts.push_back(compGasket1);
parts.push_back(compGasket2);
parts.push_back(compCover1);
parts.push_back(compCover2);

SPtr<MbAssembly> assm(new MbAssembly(parts));

{
    const double r_shell_outer = d_in_shell / 2.0 + s_tck;
    const double r_gasket_outer = d_in_shell / 2.0;
    const double r_cover_outer = d_in_shell / 2.0 + s_tck + flange_height;

    MbFace* fBodyCyl = FindCylindricalFace(resultBody, r_shell_outer);

    MbFace* fBodyFlatTop = FindPlanarFaceAtZ(resultBody, L);
    MbFace* fBodyFlatBot = FindPlanarFaceAtZ(resultBody, -L);

    MbFace* fG1CylNear = FindCylindricalFace(gasket1, r_gasket_outer);
    MbFace* fG1FlatNear = FindPlanarFaceAtZ(gasket1, gasket_length);
    MbFace* fG1FlatFar = FindPlanarFaceAtZ(gasket1, 0);

    MbFace* fG2CylNear = FindCylindricalFace(gasket2, r_gasket_outer);
    MbFace* fG2FlatNear = FindPlanarFaceAtZ(gasket2, gasket_length);
    MbFace* fG2FlatFar = FindPlanarFaceAtZ(gasket2, 0);

    MbFace* fCov1Cyl = FindCylindricalFace(cover1, r_cover_outer);
    MbFace* fCov1Flat = FindPlanarFaceAtZ(cover1, 0.0);

    MbFace* fCov2Cyl = FindCylindricalFace(cover2, r_cover_outer);
    MbFace* fCov2Flat = FindPlanarFaceAtZ(cover2, 0.0);

    if (fBodyCyl && fG1CylNear) {
        MtGeomArgument argA(fBodyCyl, compBody);
        MtGeomArgument argB(fG1CylNear, compGasket1);
        assm->AddConstraint(GCM_CONCENTRIC, argA, argB);
    }
    if (fBodyFlatTop && fG1FlatNear) {
        MtGeomArgument argA(fBodyFlatTop, compBody);
        MtGeomArgument argB(fG1FlatNear, compGasket1);
        assm->AddConstraint(GCM_DISTANCE, argA, argB, 0);
    }

    if (fBodyCyl && fG2CylNear) {
        MtGeomArgument argA(fBodyCyl, compBody);
        MtGeomArgument argB(fG2CylNear, compGasket2);
        assm->AddConstraint(GCM_CONCENTRIC, argA, argB);
    }
    if (fBodyFlatBot && fG2FlatNear) {
        MtGeomArgument argA(fBodyFlatBot, compBody);
        MtGeomArgument argB(fG2FlatNear, compGasket2);
        assm->AddConstraint(GCM_DISTANCE, argA, argB, 0.0);
    }

    if (fG1FlatFar && fCov1Cyl) {
        MtGeomArgument argA(fG1CylNear, compGasket1);
        MtGeomArgument argB(fCov1Cyl, compCover1);
        assm->AddConstraint(GCM_CONCENTRIC, argA, argB);
    }
    if (fG1FlatFar && fCov1Flat) {
        MtGeomArgument argA(fG1FlatNear, compGasket1);
        MtGeomArgument argB(fCov1Flat, compCover1);
    }
}

```

```

        assm->AddConstraint(GCM_DISTANCE, argA, argB, 0.0);
    }

    if (fG2FlatFar && fCov2Cyl) {
        MtGeomArgument argA(fG2CylNear, compGasket2);
        MtGeomArgument argB(fCov2Cyl, compCover2);
        assm->AddConstraint(GCM_CONCENTRIC, argA, argB);
    }
    if (fG2FlatNear && fCov2Flat) {
        MtGeomArgument argA(fG2FlatNear, compGasket2);
        MtGeomArgument argB(fCov2Flat, compCover2);
        assm->AddConstraint(GCM_DISTANCE, argA, argB, 0.0);
    }
}

assm->EvaluateConstraints();
compBody->AddAttribute(MbProductInfo(false, " ", "Корпус", " "));
compGasket1->AddAttribute(MbProductInfo(false, " ", "Прокладка", " "));
compGasket2->AddAttribute(MbProductInfo(false, " ", "Прокладка", " "));
compCover1->AddAttribute(MbProductInfo(false, " ", "Крышка", " "));
compCover2->AddAttribute(MbProductInfo(false, " ", "Крышка", " "));
assm->AddAttribute(MbProductInfo(false, " ", "Спиральный теплообменник", " "));

return assm;
}

```

Код файла main.cpp

```

#include <QApplication>
#include <QWindow>
#include <QStyleFactory>
#include <qstyle.h>
#include <QPixmap>
#include <QPainter>
#include <QSvgRenderer>
#include "AppWindow.h"
#include <vsn_application.h>

int main(int argc, char* argv[])
{
    qputenv("QT_QPA_PLATFORM", "windows:darkmode=0");

    QApplication app(argc, argv);
    VSN::Application vapp;

    QSvgRenderer renderer(QString(":/logo.svg"));
    if (renderer.isValid()) {
        QPixmap pixmap(256, 256);
        pixmap.fill(Qt::transparent);
        QPainter painter(&pixmap);
        renderer.render(&painter);
        app.setWindowIcon(QIcon(pixmap));
    }

    int result = 0;

    {
        AppWindow win;
        win.showMaximized();

        result = app.exec();
    }

    return result;
}

```

Код файла AppWindow.h

```

#pragma once

#include <QMainWindow>
#include "LicenseManager.h"
#include "solid.h"

class CalcSceneWidget;
class CalcManagerWidget;
class QSplitter;

class AppWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit AppWindow(QWidget* parent = nullptr);

private:
    void checkLicense();
    void setupUi();
}

```

```

    void onSaveModel();
    void onOpenCalc();

protected:
    void showEvent(QShowEvent* event) override;

private:
    LicenseManager m_licenseManager;
    QSplitter* m_splitter = nullptr;
    CalcSceneWidget* m_viewport = nullptr;
    CalcManagerWidget* m_sidePanel = nullptr;
};

```

Код файла AppWindow.cpp

```

#include "AppWindow.h"
#include "LicenseManager.h"
#include "LicenseDialog.h"
#include "CalcSceneWidget.h"
#include "CalcManagerWidget.h"
#include "CalcResultSerializer.h"
#include "ResultDialog.h"

#include <QApplication>
#include <QWindow>
#include <QMessageBox>
#include <QSplitter>
#include <QMenuBar>
#include <QHBoxLayout>
#include <QTimer>
#include <QFileDialog>
#include <QFileInfo>
#include <QActionGroup>
#include <QIcon>
#include <QPixmap>
#include <QPainter>
#include <QSvgRenderer>

#include <system_atomic.h>
#include <action_solid.h>
#include <action_shell.h>
#include <assembly.h>
#include <topology_iterator.h>
#include <instance_item.h>
#include <space_instance.h>
#include <plane_instance.h>
#include <reference_item.h>
#include <surface.h>
#include <solid.h>
#include <wire_frame.h>
#include <point_frame.h>
#include <cur_polyline.h>
#include <conv_model_exchange.h>
#include <conv_exchange_settings.h>
#include <conv_model_document.h>
#include <conv_requestor.h>
#include <attr_color.h>
#include <attr_common_attribute.h>
#include <check_geometry.h>
#include <alg_draw.h>
#include <list>

```



```

#include <sstream>
#include <stdio.h>
#include <last.h>

AppWindow::AppWindow(QWidget* parent)
    : QMainWindow(parent)
{
    setWindowTitle("SpiralHX");

    QSvgRenderer renderer(QString(":/logo.svg"));
    if (renderer.isValid()) {
        QPixmap pixmap(256, 256);
        pixmap.fill(Qt::transparent);
        QPainter painter(&pixmap);
        renderer.render(&painter);
        setWindowIcon(QIcon(pixmap));
    }

    checkLicense();

    setupUi();
}

void AppWindow::checkLicense()
{
    if (m_licenseManager.activateFromFile())
    {
        return;
    }

    while (true)
    {
        LicenseDialog dlg(this);

        if (dlg.exec() != QDialog::Accepted)
        {
            QApplication::quit();
            return;
        }

        auto result = m_licenseManager.activate(
            dlg.key(),
            dlg.signature()
        );

        if (result == LicenseManager::Result::Success)
            return;

        QMessageBox::warning(this, "Активация лицензии", "Ошибка активации
лицензии");
    }
}

void AppWindow::setupUi()
{
    auto fileMenu = menuBar()->addMenu("Файл");
    auto openCalcAction = fileMenu->addAction("Открыть расчёт...");
    auto saveStlAction = fileMenu->addAction("Сохранить модель");

    connect(openCalcAction, &QAction::triggered,

```

```

        this, &AppWindow::onOpenCalc);
connect(saveStlAction, &QAction::triggered,
        this, &AppWindow::onSaveModel);

auto viewMenu = menuBar()->addMenu("Вид");
auto planeMenu = viewMenu->addMenu("Сечение модели");

auto actX = planeMenu->addAction("По оси X");
auto actY = planeMenu->addAction("По оси Y");
auto actZ = planeMenu->addAction("По оси Z");

QList<QAction*> actions = { actX, actY, actZ };

for (auto act : actions) {
    act->setCheckable(true);

    connect(act, &QAction::triggered, this, [this, act, actions]() {
        if (!act->isChecked()) {
            m_viewport->togglePlaneCutting(false);
        }
        else {
            for (auto other : actions) {
                if (other != act) {
                    other->blockSignals(true);
                    other->setChecked(false);
                    other->blockSignals(false);
                }
            }

            CuttingPlane plane = CuttingPlane::Y;
            if (act->text().contains("X")) plane = CuttingPlane::X;
            else if (act->text().contains("Z")) plane = CuttingPlane::Z;

            m_viewport->togglePlaneCutting(true, plane);
        }
    });
}

auto central = new QWidget(this);
setCentralWidget(central);

m_splitter = new QSplitter(Qt::Horizontal, central);

m_viewport = new CalcSceneWidget(m_splitter);
m_sidePanel = new CalcManagerWidget(m_splitter);
m_sidePanel->setViewport(m_viewport);

auto layout = new QHBoxLayout(central);
layout->setContentsMargins(0, 0, 0, 0);
layout->addWidget(m_splitter);

m_splitter->setStretchFactor(0, 4);
m_splitter->setStretchFactor(1, 1);

QTimer::singleShot(0, this, [this]()
{
    int w = this->width();
    if (w <= 0) w = 1200;

    m_splitter->setSizes({ int(w * 0.75), int(w * 0.25) });
});

```

```

    });

    resize(1400, 900);
    setMinimumSize(1100, 700);
}

void AppWindow::showEvent(QShowEvent* event) {
    windowHandle()->setSurfaceType(QWindow::OpenGLSurface);
}

void AppWindow::onSaveModel()
{
    SPtr<MbAssembly> assembly = m_viewport->getAssembly();

    if (!assembly) {
        QMessageBox::warning(this, "Сохранение", "Нет модели для сохранения.");
        return;
    }

    QStringList filters;
    filters
        << "File C3D (*.c3d)"
        << "File OBJ (*.obj)"
        << "File ACISC (*.sat)"
        << "File STEP (*.step *.stp)"
        << "File PARASOLID (*.x_b)"
        << "File JT (*.jt *.JT)"
        << "File STL (*.stl *.STL)";

    QString fileName = QFileDialog::getSaveFileName(
        this,
        "Сохранить модель как...",
        "",
        filters.join(";;")
    );

    if (fileName.isEmpty())
        return;

    QApplication::setOverrideCursor(Qt::WaitCursor);

    MbModel model;
    model.AddItem(*assembly);

    MbeConvResType result = c3d::ExportIntoFile(
        model,
        c3d::WToPathstring(fileName.toStdWString())
    );

    QApplication::restoreOverrideCursor();

    if (result == cnv_Success) {
        QMessageBox::information(this, "Успех", "Файл успешно сохранен.");
    }
    else {
        QMessageBox::critical(
            this,
            "Ошибка",
            QString("Не удалось сохранить файл. Код ошибки C3D: %1")
                .arg(static_cast<int>(result))
        );
    }
}

```

```

    );
}
}

void AppWindow::onOpenCalc()
{
    QString path = QFileDialog::getOpenFileName(
        this,
        "Открыть расчёт",
        {},
        "Файл расчёта SpiralHX (*.txt)"
    );
    if (path.isEmpty())
        return;

    QString err;
    CalculationResult res = CalcResultSerializer::load(path, err);
    if (!err.isEmpty()) {
        QMessageBox::critical(this, "Ошибка открытия", err);
        return;
    }

    auto* dlg = new ResultDialog(res, this);
    dlg->setWindowTitle("Результат расчёта – " + QFileInfo(path).fileName());
    dlg->setAttribute(Qt::WA_DeleteOnClose);
    dlg->show();

    connect(dlg, &ResultDialog::selectedResultConfirmed,
        this, [this](const CalculationResult& r) {
            m_sidePanel->buildModelFromResult(r);
        });
}

```

Код файла LicenseManager.h

```

#pragma once

class QString;

class LicenseManager
{
public:
    enum class Result
    {
        Success,
        Invalid,
    };

    Result activate(const QString& key, const QString& signature);
    bool activateFromFile();
    bool isActive() const;
    void clear();

private:
    QString filePath() const;
    bool loadFromFile(QString& key, QString& signature);
    bool saveToFile(const QString& key, const QString& signature);
    void removeFile();

private:
    bool m_active = false;
};

```

Код файла LicenseManager.cpp

```
#include "LicenseManager.h"

#include <tool_enabler.h>

#include <QFile>
#include <QTextStream>
#include <QStandardPaths>

QString LicenseManager::filePath() const
{
    QString home = QStandardPaths::writableLocation(QStandardPaths::HomeLocation);
    return home + "/c3d.lic";
}

bool LicenseManager::loadFromFile(QString& key, QString& signature)
{
    QFile file(filePath());
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text))
        return false;

    QTextStream in(&file);

    key = in.readLine().trimmed();
    signature = in.readLine().trimmed();

    return !key.isEmpty() && !signature.isEmpty();
}

bool LicenseManager::saveToFile(const QString& key, const QString& signature)
{
    QFile file(filePath());
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
        return false;

    QTextStream out(&file);
    out << key << "\n";
    out << signature << "\n";

    return true;
}

void LicenseManager::removeFile()
{
    QFile::remove(filePath());
}

LicenseManager::Result LicenseManager::activate(const QString& key, const QString& signature)
{
    QByteArray keyB = key.toUtf8();
    QByteArray sigB = signature.toUtf8();

    EnableMathModules(
        keyB.constData(),
        keyB.size(),
        sigB.constData(),
        sigB.size()
    );
}
```

```

        m_active = IsMathVisionEnable();

        if (m_active)
            saveToFile(key, signature);

        return m_active
            ? Result::Success
            : Result::Invalid;
    }

bool LicenseManager::activateFromFile()
{
    QString key;
    QString signature;

    if (!loadFromFile(key, signature))
        return false;

    QByteArray keyB = key.toUtf8();
    QByteArray sigB = signature.toUtf8();

    EnableMathModules(
        keyB.constData(),
        keyB.size(),
        sigB.constData(),
        sigB.size()
    );

    m_active = IsMathVisionEnable();

    return m_active;
}

void LicenseManager::clear()
{
    DisableMathModules();
    removeFile();

    m_active = false;
}

bool LicenseManager::isActive() const
{
    return m_active && IsMathVisionEnable();
}

```

Код файла CalcResultSerializer.h

```

#pragma once

#include "SpiralHeatExchanger.h"
#include <QString>

class CalcResultSerializer
{
public:
    static QString save(const CalculationResult& res, const QString& filePath);
    static CalculationResult load(const QString& filePath, QString& outError);
    static QString savePdf(const CalculationResult& res, const QString& filePath);

```

```
private:
    static QString buildHtmlReport(const CalculationResult& res);
};
```

Код файла CalcResultSerializer.cpp

```
#include "CalcResultSerializer.h"

#include <QFile>
#include <QTextStream>
#include <QMap>
#include <QTextDocument>
#include <QPrinter>
#include <QPageSize>
#include <QPageLayout>
#include <QMarginsF>

static void w(QTextStream& out, const QString& key, double val)
{
    out << key << "=" << QString::number(val, 'g', 15) << "\n";
}

static void w(QTextStream& out, const QString& key, int val)
{
    out << key << "=" << val << "\n";
}

static void w(QTextStream& out, const QString& key, const std::string& val)
{
    out << key << "=" << QString::fromStdString(val) << "\n";
}

static double rd(const QMap<QString, QString>& m, const QString& key, double def = 0.0)
{
    auto it = m.find(key);
    return (it != m.end()) ? it->toDouble() : def;
}

static int ri(const QMap<QString, QString>& m, const QString& key, int def = 0)
{
    auto it = m.find(key);
    return (it != m.end()) ? it->toInt() : def;
}

static std::string rs(const QMap<QString, QString>& m, const QString& key)
{
    auto it = m.find(key);
    return (it != m.end()) ? it->toStdString() : std::string{};
}

QString CalcResultSerializer::save(const CalculationResult& r, const QString&
filePath)
{
    QFile file(filePath);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
        return QString("Не удалось открыть файл для записи: %1").arg(filePath);

    QTextStream out(&file);
```

```

out.setEncoding(QStringConverter::Utf8);

out << "[SpiralHX_Calculation]\n";

w(out, "Q_required", r.Q_required);
w(out, "LMTD_required", r.LMTD_required);
w(out, "Q_real", r.Q_real);
w(out, "LMTD_real", r.LMTD_real);
w(out, "K", r.K);
w(out, "alpha_hot", r.alpha_hot);
w(out, "alpha_cold", r.alpha_cold);

w(out, "velocity_hot", r.velocity_hot);
w(out, "velocity_cold", r.velocity_cold);
w(out, "Re_hot", r.Re_hot);
w(out, "Re_cold", r.Re_cold);
w(out, "t_hot_out_real", r.t_hot_out_real);
w(out, "t_cold_out_real", r.t_cold_out_real);
w(out, "power_hot", r.power_hot);
w(out, "power_cold", r.power_cold);

w(out, "A", r.A);
w(out, "channel_length", r.channel_length);
w(out, "channel_width", r.channel_width);
w(out, "turns", r.turns);
w(out, "D", r.D);
w(out, "inlet_D", r.inlet_D);

w(out, "geom.width_hot", r.geometry.width_hot);
w(out, "geom.width_cold", r.geometry.width_cold);
w(out, "geom.d_inner", r.geometry.d_inner);
w(out, "geom.shell_thickness", r.geometry.shell_thickness);
w(out, "geom.cover_thickness", r.geometry.cover_thickness);

w(out, "wall.lambda", r.wall.lambda);
w(out, "wall.thickness", r.wall.thickness);
w(out, "wall.fouling_factor", r.wall.fouling_factor);

w(out, "gasket_thickness", r.gasket_thickness);
w(out, "nozzle_shell_thickness", r.nozzle_shell_thickness);
w(out, "nozzle_shell_length", r.nozzle_shell_length);

w(out, "flange.name", r.selected_flange.name);
w(out, "flange.inner_d", r.selected_flange.inner_d);
w(out, "flange.outer_d", r.selected_flange.outer_d);
w(out, "flange.bolt_circle_d", r.selected_flange.bolt_circle_d);
w(out, "flange.flange_thickness", r.selected_flange.flange_thickness);
w(out, "flange.bolt_d", r.selected_flange.bolt_d);
w(out, "flange.bolt_count", r.selected_flange.bolt_count);

w(out, "sf.name", r.shell_flange.name);
w(out, "sf.inner_d", r.shell_flange.inner_d);
w(out, "sf.outer_d", r.shell_flange.outer_d);
w(out, "sf.bolt_circle_d", r.shell_flange.bolt_circle_d);
w(out, "sf.flange_thickness", r.shell_flange.flange_thickness);
w(out, "sf.bolt_d", r.shell_flange.bolt_d);
w(out, "sf.bolt_count", r.shell_flange.bolt_count);

return {};
}

```



```

CalculationResult CalcResultSerializer::load(const QString& filePath, QString&
outError)
{
    CalculationResult r{};
    outError.clear();

    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        outError = QString("Не удалось открыть файл: %1").arg(filePath);
        return r;
    }

    QTextStream in(&file);
    in.setEncoding(QStringConverter::Utf8);

    QString firstLine = in.readLine().trimmed();
    if (firstLine != "[SpiralHX_Calculation]") {
        outError = "Файл не является файлом расчёта SpiralHX.";
        return r;
    }

    QMap<QString, QString> m;
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (line.isEmpty() || line.startsWith('#')) continue;
        int eq = line.indexOf('=');
        if (eq < 0) continue;
        m[line.left(eq).trimmed()] = line.mid(eq + 1).trimmed();
    }

    r.Q_required = rd(m, "Q_required");
    r.LMTD_required = rd(m, "LMTD_required");
    r.Q_real = rd(m, "Q_real");
    r.LMTD_real = rd(m, "LMTD_real");
    r.K = rd(m, "K");
    r.alpha_hot = rd(m, "alpha_hot");
    r.alpha_cold = rd(m, "alpha_cold");

    r.velocity_hot = rd(m, "velocity_hot");
    r.velocity_cold = rd(m, "velocity_cold");
    r.Re_hot = rd(m, "Re_hot");
    r.Re_cold = rd(m, "Re_cold");
    r.t_hot_out_real = rd(m, "t_hot_out_real");
    r.t_cold_out_real = rd(m, "t_cold_out_real");
    r.power_hot = rd(m, "power_hot");
    r.power_cold = rd(m, "power_cold");

    r.A = rd(m, "A");
    r.channel_length = rd(m, "channel_length");
    r.channel_width = rd(m, "channel_width");
    r.turns = rd(m, "turns");
    r.D = rd(m, "D");
    r.inlet_D = rd(m, "inlet_D");

    r.geometry.width_hot = rd(m, "geom.width_hot");
    r.geometry.width_cold = rd(m, "geom.width_cold");
    r.geometry.d_inner = rd(m, "geom.d_inner");
    r.geometry.shell_thickness = rd(m, "geom.shell_thickness");
    r.geometry.cover_thickness = rd(m, "geom.cover_thickness");

    r.wall.lambda = rd(m, "wall.lambda");

```

```

r.wall.thickness = rd(m, "wall.thickness");
r.wall.fouling_factor = rd(m, "wall.fouling_factor");

r.gasket_thickness = rd(m, "gasket_thickness", 0.003);
r.nozzle_shell_thickness = rd(m, "nozzle_shell_thickness", 4.0);
r.nozzle_shell_length = rd(m, "nozzle_shell_length", 150.0);

r.selected_flange.name = rs(m, "flange.name");
r.selected_flange.inner_d = rd(m, "flange.inner_d");
r.selected_flange.outer_d = rd(m, "flange.outer_d");
r.selected_flange.bolt_circle_d = rd(m, "flange.bolt_circle_d");
r.selected_flange.flange_thickness = rd(m, "flange.flange_thickness");
r.selected_flange.bolt_d = rd(m, "flange.bolt_d");
r.selected_flange.bolt_count = ri(m, "flange.bolt_count");

r.shell_flange.name = rs(m, "sf.name");
r.shell_flange.inner_d = rd(m, "sf.inner_d");
r.shell_flange.outer_d = rd(m, "sf.outer_d");
r.shell_flange.bolt_circle_d = rd(m, "sf.bolt_circle_d");
r.shell_flange.flange_thickness = rd(m, "sf.flange_thickness");
r.shell_flange.bolt_d = rd(m, "sf.bolt_d");
r.shell_flange.bolt_count = ri(m, "sf.bolt_count");

return r;
}

static QString cell(const QString& label, const QString& value,
const QString& color = {})
{
    QString valStyle = color.isEmpty()
        ? "font-weight:600;"
        : QString("font-weight:600;color:%1;").arg(color);

    return QString(
        "<tr>"
        "<td style='padding:4px 12px 4px 4px;color:#444;'>%1</td>"
        "<td style='padding:4px 4px 4px 4px;%2'>%3</td>"
        "</tr>")
        .arg(label, valStyle, value);
}

static QString hotColdCell(const QString& label,
const QString& hot, const QString& cold,
const QString& unit = {})
{
    QString val = QString(
        "<span style='color:#cc2200;font-weight:600;'>%1</span>"
        " / "
        "<span style='color:#0055cc;font-weight:600;'>%2</span>%3")
        .arg(hot, cold,
            unit.isEmpty() ? QString() : QString("&nbsp;%1").arg(unit));
    return cell(label, val);
}

static QString sectionHeader(const QString& title)
{
    return QString(
        "<tr><td colspan='2' style='"
        "padding:10px 4px 4px 4px;"
        "font-size:11pt;"
        "font-weight:bold;"

```

```

        "color:#1a3a5c;"
        "border-bottom:2px solid #1a3a5c;"
        "'>%1</td></tr>")
        .arg(title);
    }

QString CalcResultSerializer::buildHtmlReport(const CalculationResult& r)
{
    auto n = [](double v, int dec = 2) { return QString::number(v, 'f', dec); };

    QString rows;

    rows += sectionHeader("ТЕПЛОФИЗИКА");
    rows += cell("Требуемая тепловая мощность:",
        n(r.Q_required / 1000.0, 2) + " кВт");
    rows += cell("LMTD (средний температурный напор):",
        n(r.LMTD_real, 2) + " °C");
    rows += cell("Коэффициент теплопередачи K:",
        n(r.K, 1) + " Вт/(м2·K)");
    rows += hotColdCell("Коэффициент теплоотдачи (гор / хол):",
        n(r.alpha_hot, 1), n(r.alpha_cold, 1),
        "Вт/(м2·K)");
    rows += cell("Теплопроводность стенки:",
        n(r.wall.lambda, 1) + " Вт/(м·K)");
    rows += cell("Фактическая тепловая мощность:",
        n(r.Q_real / 1000.0, 2) + " кВт");
    rows += hotColdCell("Температура на выходе (гор / хол):",
        n(r.t_hot_out_real, 1), n(r.t_cold_out_real, 1),
        "&deg;C");

    rows += sectionHeader("ГИДРОДИНАМИКА");
    rows += hotColdCell("Скорость среды (гор / хол):",
        n(r.velocity_hot, 2), n(r.velocity_cold, 2), "м/с");
    rows += hotColdCell("Число Рейнольдса Re (гор / хол):",
        QString::number(static_cast<int>(r.Re_hot)),
        QString::number(static_cast<int>(r.Re_cold)));
    rows += hotColdCell("Мощность насоса (гор / хол):",
        n(r.power_hot, 2), n(r.power_cold, 2), "Вт");

    rows += sectionHeader("ГЕОМЕТРИЯ ТЕПЛООБМЕННИКА");
    rows += cell("Наружный диаметр корпуса:",
        n(r.D * 1000.0, 1) + " мм");
    rows += cell("Поверхность теплообмена:",
        n(r.A, 3) + " м2");
    rows += cell("Количество витков:",
        n(r.turns, 0));
    rows += cell("Длина канала:",
        n(r.channel_length, 2) + " м");
    rows += cell("Ширина листа (b):",
        n(r.channel_width * 1000.0, 1) + " мм");
    rows += hotColdCell("Ширина канала (гор / хол):",
        n(r.geometry.width_hot * 1000.0, 1),
        n(r.geometry.width_cold * 1000.0, 1), "мм");
    rows += cell("Внутренний диаметр спирали:",
        n(r.geometry.d_inner * 1000.0, 1) + " мм");
    rows += cell("Толщина стенки разделителя:",
        n(r.wall.thickness * 1000.0, 1) + " мм");
    rows += cell("Толщина обечайки:",
        n(r.geometry.shell_thickness * 1000.0, 1) + " мм");
    rows += cell("Толщина крышки:",
        n(r.geometry.cover_thickness * 1000.0, 1) + " мм");
}

```

```

rows += cell("Толщина прокладки:",
    n(r.gasket_thickness * 1000.0, 1) + "&nbsp;мм");

rows += sectionHeader("ПАТРУБКИ И ФЛАНЦЫ");
rows += cell("Расчётный диаметр входного отверстия:",
    n(r.inlet_D * 1000.0, 1) + "&nbsp;мм");
rows += cell("Толщина стенки патрубка:",
    n(r.nozzle_shell_thickness, 1) + "&nbsp;мм");
rows += cell("Длина патрубка:",
    n(r.nozzle_shell_length, 1) + "&nbsp;мм");

if (!r.selected_flange.name.empty()) {
    const auto& f = r.selected_flange;
    rows += cell("Фланец патрубка:",
        QString::fromStdString(f.name));
    rows += cell("&nbsp;&nbsp;&nbsp;Внутренний / наружный диаметр:",
        n(f.inner_d, 1) + " / " + n(f.outer_d, 1) + "&nbsp;мм");
    rows += cell("&nbsp;&nbsp;&nbsp;Диаметр болтовой окружности:",
        n(f.bolt_circle_d, 1) + "&nbsp;мм");
    rows += cell("&nbsp;&nbsp;&nbsp;Толщина фланца:",
        n(f.flange_thickness, 1) + "&nbsp;мм");
    rows += cell("&nbsp;&nbsp;&nbsp;Болты:",
        "М" + n(f.bolt_d, 0) + " × " +
        QString::number(f.bolt_count) + "&nbsp;шт.");
}

if (!r.shell_flange.name.empty()) {
    const auto& sf = r.shell_flange;
    rows += cell("Фланец обечайки:",
        QString::fromStdString(sf.name));
    rows += cell("&nbsp;&nbsp;&nbsp;Внутренний / наружный диаметр:",
        n(sf.inner_d, 1) + " / " + n(sf.outer_d, 1) + "&nbsp;мм");
    rows += cell("&nbsp;&nbsp;&nbsp;Диаметр болтовой окружности:",
        n(sf.bolt_circle_d, 1) + "&nbsp;мм");
    rows += cell("&nbsp;&nbsp;&nbsp;Толщина фланца:",
        n(sf.flange_thickness, 1) + "&nbsp;мм");
    rows += cell("&nbsp;&nbsp;&nbsp;Болты:",
        "М" + n(sf.bolt_d, 0) + " × " +
        QString::number(sf.bolt_count) + "&nbsp;шт.");
}

return QString(R"(
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8"/>
<style>
    body { font-family: Arial, sans-serif; font-size: 10pt;
        color: #222; margin: 0; padding: 0; }
    .header { background:#1a3a5c; color:white; padding:16px 20px 12px 20px;
        margin-bottom:12px; }
    .header h1 { margin:0; font-size:15pt; letter-spacing:1px; }
    .header p { margin:4px 0 0; font-size:9pt; color:#b0c8e8; }
    table { width:100%; border-collapse:collapse; }
    td { vertical-align:top; }
    .footer { margin-top:20px; font-size:8pt; color:#888;
        border-top:1px solid #ccc; padding-top:6px; }
</style>
</head>
<body>
<div class="header">

```

```

        <h1>Спиральный теплообменник – результаты расчёта</h1>
        <p>SpiralHX</p>
    </div>
    <table>
    %1
    </table>
    <div class="footer">Документ сформирован автоматически программой SpiralHX</div>
    </body>
    </html>
    )").arg(rows);
    }

QString CalcResultSerializer::savePdf(const CalculationResult& r,
    const QString& filePath)
{
    QPrinter printer(QPrinter::HighResolution);
    printer.setOutputFormat(QPrinter::PdfFormat);
    printer.setOutputFileName(filePath);
    printer.setPageSize(QPageSize(QPageSize::A4));
    printer.setPageOrientation(QPageLayout::Portrait);
    printer.setPageMargins(QMarginsF(15.0, 15.0, 15.0, 15.0),
        QPageLayout::Millimeter);

    QTextDocument doc;
    doc.setDefaultFont(QFont("Arial", 10));
    doc.setHtml(buildHtmlReport(r));

    doc.print(&printer);
    return {};
}

```

Код файла CalcManagerWidget.h

```

#pragma once

#include <QWidget>
#include <QCheckBox>
#include <QThread>
#include <QProgressDialog>
#include "SpiralHeatExchanger.h"
#include "ModelBuilder.h"
#include "solid.h"
#include "assembly.h"

class QTabWidget;
class QPushButton;
class QDoubleSpinBox;
class QComboBox;
class QFormLayout;
class QGroupBox;
class CleanDoubleSpinBox;
class CalcSceneWidget;

class ModelBuildWorker : public QObject
{
    Q_OBJECT
public:
    explicit ModelBuildWorker(const CalculationResult& result, QObject* parent =
        nullptr)
        : QObject(parent), m_result(result) {
    }
}

```

```

signals:
    void progress(int step, int total, const QString& description);
    void finished(SPtr<MbAssembly> assembly);

public slots:
    void run() {
        ModelBuilder builder;
        builder.setProgressCallback([this](int step, int total, const QString& desc)
{
            emit progress(step, total, desc);
        });
        auto assembly = builder.BuildModel(m_result);
        emit finished(assembly);
    }

private:
    CalculationResult m_result;
};

struct FlangeRecord {
    double d_outer;
    double d_inner;
    double bcd;
    double plate_thk;
    double bolt_d;
    int    bolt_n;
    double shell_thk;
    double shell_len;
};

class CalcManagerWidget : public QWidget
{
    Q_OBJECT

public:
    explicit CalcManagerWidget(QWidget* parent = nullptr);
    void setViewport(CalcSceneWidget* vp) { m_viewport = vp; }
    void buildModelFromResult(const CalculationResult& result);

private slots:
    void onCalculate();

private:
    struct FluidUI {
        CleanDoubleSpinBox* G;
        CleanDoubleSpinBox* t_in;
        CleanDoubleSpinBox* t_out;
        CleanDoubleSpinBox* rho;
        CleanDoubleSpinBox* cp;
        CleanDoubleSpinBox* lambda;
        CleanDoubleSpinBox* viscosity;
        CleanDoubleSpinBox* max_velocity;
        CleanDoubleSpinBox* t_avg;
    };

    struct WallUI {
        CleanDoubleSpinBox* lambda;
        CleanDoubleSpinBox* fouling;
    };
};

```

```

struct GeometryUI {
    CleanDoubleSpinBox* width_hot;
    CleanDoubleSpinBox* width_cold;
    CleanDoubleSpinBox* d_inner;
    CleanDoubleSpinBox* thickness_mm;

    QCheckBox* autoWidthCheck;
    CleanDoubleSpinBox* widthMin;
    CleanDoubleSpinBox* widthMax;
};

struct FlangeUI {
    QComboBox* d_outer;
    QComboBox* d_inner;
    QComboBox* bcd;
    QComboBox* plate_thk;
    QComboBox* bolt_d;
    QComboBox* bolt_n;
    QComboBox* shell_thk;
    QComboBox* shell_len;
};

QTabWidget* tabs;
QPushButton* calcButton;

QList<FlangeRecord> flanges;

FluidUI hot;
FluidUI cold;
WallUI wall;
GeometryUI geom;
FlangeUI flange;
CalcSceneWidget* m_viewport = nullptr;

private:
    QWidget* createFluidsTab();
    void setupAutoCalc();
    void onBalanceEdited();
    void solveBalance();
    void updateAverageTemps();
    QWidget* createGeometryTab();
    void addRowWithUnit(QFormLayout* form, const QString& name, QDoubleSpinBox* spin,
const QString& unit);

    Fluid toFluid(const FluidUI& ui);
    Wall toWall(const WallUI& ui);
    Geometry toGeometry(const GeometryUI& ui);

    void buildModel(const CalculationResult& result);
    void setDefaults();
};

```

Код файла CalcManagerWidget.cpp

```

#include "CalcManagerWidget.h"
#include "SpiralHeatExchanger.h"
#include "ResultDialog.h"
#include "ModelBuilder.h"
#include "CalcSceneWidget.h"

#include <QVBoxLayout>
#include <QTabWidget>

```

```

#include <QGroupBox>
#include <QFormLayout>
#include <QDoubleSpinBox>
#include <QPushButton>
#include <QLabel>
#include <QLineEdit>
#include <QTimer>
#include <QToolTip>
#include <QKeyEvent>
#include <QThread>
#include <QProgressDialog>

class CleanDoubleSpinBox : public QDoubleSpinBox
{
public:
    using QDoubleSpinBox::QDoubleSpinBox;

    void setRangeWithWarning(double min, double max, const QString& msg = "") {
        setRange(min, max);
        m_warningText = msg.isEmpty()
            ? QString("Значение должно быть в диапазоне от %1 до %2").arg(min).arg(max)
            : msg;
    }

protected:
    QString m_warningText;

    void validateValue() {
        if (value() >= maximum() || value() <= minimum()) {
            QToolTip::showText(mapToGlobal(QPoint(width() / 2, 0)), m_warningText,
this);
            this->setStyleSheet("border: 1px solid #FF4444; background-color:
#FFF0F0;");
            QTimer::singleShot(1500, this, [this]() { this->setStyleSheet(""); });
        }
    }

    QString textFromValue(double value) const override {
        if (!hasFocus() && value == 0.0)
            return "";
        QString s = QString::number(value, 'f', decimals());
        if (s.contains('.')) {
            while (s.endsWith('0')) s.chop(1);
            if (s.endsWith('.')) s.chop(1);
        }
        return s;
    }

    void focusInEvent(QFocusEvent* event) override {
        QDoubleSpinBox::focusInEvent(event);
        QTimer::singleShot(0, this, [this]() {
            if (lineEdit()) lineEdit()->selectAll();
        });
    }

    void focusOutEvent(QFocusEvent* event) override {
        validateValue();
        QDoubleSpinBox::focusOutEvent(event);
    }

    void keyPressEvent(QKeyEvent* event) override {
        QDoubleSpinBox::keyPressEvent(event);
        if (event->key() == Qt::Key_Enter || event->key() == Qt::Key_Return) {
            validateValue();
        }
    }
};

```



```

CalcManagerWidget::CalcManagerWidget(QWidget* parent) : QWidget(parent)
{
    auto layout = new QVBoxLayout(this);

    tabs = new QTabWidget(this);
    tabs->addTab(createFluidsTab(), "Теплоносители");
    tabs->addTab(createGeometryTab(), "Геометрия");

    layout->addWidget(tabs);

    calcButton = new QPushButton("Рассчитать", this);
    calcButton->setFixedHeight(40);
    calcButton->setStyleSheet("font-weight: bold;");
    layout->addWidget(calcButton);
    layout->addStretch();

    connect(calcButton, &QPushButton::clicked, this, &CalcManagerWidget::onCalculate);

    setDefaults();
}

QWidget* CalcManagerWidget::createFluidsTab()
{
    auto w = new QWidget;
    auto layout = new QVBoxLayout(w);

    auto makeFlowSB = []() {
        auto s = new CleanDoubleSpinBox;
        s->setDecimals(3);
        s->setButtonSymbols(QAbstractSpinBox::NoButtons);
        s->setRangeWithWarning(0.0, 1000.0, "Расход должен быть в пределах 0 - 1000 кг/с  
(0 для авторасчета)");
        return s;
    };

    auto makeTempSB = []() {
        auto s = new CleanDoubleSpinBox;
        s->setDecimals(2);
        s->setButtonSymbols(QAbstractSpinBox::NoButtons);
        s->setRangeWithWarning(-100.0, 600.0, "Температура вне промышленного диапазона  
(-100...600°C)");
        return s;
    };

    auto makeRhoSB = []() {
        auto s = new CleanDoubleSpinBox;
        s->setDecimals(2);
        s->setRangeWithWarning(0.5, 5000.0, "Плотность должна быть от 0.5 (газы) до 5000  
(тяжелые среды) кг/м³");
        return s;
    };

    auto makeCpSB = []() {
        auto s = new CleanDoubleSpinBox;
        s->setDecimals(1);
        s->setRangeWithWarning(100.0, 10000.0, "Удельная теплоемкость обычно находится в  
пределах 100...10000 Дж/(кг·К)");
        return s;
    };

    auto makeLambdaSB = []() {
        auto s = new CleanDoubleSpinBox;
        s->setDecimals(4);
        s->setRangeWithWarning(0.01, 5.0, "Теплопроводность сред обычно варьируется от  
0.01 до 5 Вт/(м·К)");
        return s;
    };
}

```

```

auto makeViscSB = []() {
    auto s = new CleanDoubleSpinBox;
    s->setDecimals(4);
    s->setRangeWithWarning(0.001, 5000.0, "Динамическая вязкость должна быть в
пределах 0.001 - 5000 мПа·с");
    return s;
};

auto makeVeloSB = []() {
    auto s = new CleanDoubleSpinBox;
    s->setDecimals(2);
    s->setRangeWithWarning(0.1, 15.0, "Скорость выше 10-15 м/с приводит к
критическому износу стенок");
    return s;
};

auto balanceBox = new QGroupBox("Уравнение теплового баланса");
auto grid = new QGridLayout(balanceBox);

grid->addWidget(new QLabel("Параметр"), 0, 0);
auto hLabel = new QLabel("Горячая среда");
hLabel->setStyleSheet("font-weight: bold;");
hLabel->setAlignment(Qt::AlignCenter);
grid->addWidget(hLabel, 0, 1);

auto cLabel = new QLabel("Холодная среда");
cLabel->setStyleSheet("font-weight: bold;");
cLabel->setAlignment(Qt::AlignCenter);
grid->addWidget(cLabel, 0, 2);

hot.G = makeFlowSB();
hot.t_in = makeTempSB();
hot.t_out = makeTempSB();

cold.G = makeFlowSB();
cold.t_in = makeTempSB();
cold.t_out = makeTempSB();

grid->addWidget(new QLabel("Расход, кг/с"), 1, 0);
grid->addWidget(hot.G, 1, 1);
grid->addWidget(cold.G, 1, 2);

grid->addWidget(new QLabel("Температура входа, °C"), 2, 0);
grid->addWidget(hot.t_in, 2, 1);
grid->addWidget(cold.t_in, 2, 2);

grid->addWidget(new QLabel("Температура выхода, °C"), 3, 0);
grid->addWidget(hot.t_out, 3, 1);
grid->addWidget(cold.t_out, 3, 2);

auto hotBox = new QGroupBox("Свойства горячей среды");
auto hotForm = new QFormLayout(hotBox);

hot.t_avg = makeTempSB();
hot.t_avg->setReadOnly(true);
hot.t_avg->setFocusPolicy(Qt::NoFocus);
hot.rho = makeRhoSB();
hot.cp = makeCpSB();
hot.lambda = makeLambdaSB();
hot.viscosity = makeViscSB();
hot.max_velocity = makeVeloSB();

addRowWithUnit(hotForm, "Средняя температура", hot.t_avg, "°C");
addRowWithUnit(hotForm, "Плотность", hot.rho, "кг/м³");
addRowWithUnit(hotForm, "Теплоёмкость", hot.cp, "Дж/(кг·К)");
addRowWithUnit(hotForm, "Теплопроводность", hot.lambda, "Вт/(м·К)");
addRowWithUnit(hotForm, "Дин. вязкость", hot.viscosity, "мПа·с");

```

```

        addRowWithUnit(hotForm, "Макс. скорость", hot.max_velocity, "м/с");

        auto coldBox = new QGroupBox("Свойства холодной среды");
        auto coldForm = new QFormLayout(coldBox);

        cold.t_avg = makeTempSB();
        cold.t_avg->setReadOnly(true);
        cold.t_avg->setFocusPolicy(Qt::NoFocus);
        cold.rho = makeRhoSB();
        cold.cp = makeCpSB();
        cold.lambda = makeLambdaSB();
        cold.viscosity = makeViscSB();
        cold.max_velocity = makeVeloSB();

        addRowWithUnit(coldForm, "Средняя температура", cold.t_avg, "°C");
        addRowWithUnit(coldForm, "Плотность", cold.rho, "кг/м³");
        addRowWithUnit(coldForm, "Теплоёмкость", cold.cp, "Дж/(кг·К)");
        addRowWithUnit(coldForm, "Теплопроводность", cold.lambda, "Вт/(м·К)");
        addRowWithUnit(coldForm, "Дин. вязкость", cold.viscosity, "мПа·с");
        addRowWithUnit(coldForm, "Макс. скорость", cold.max_velocity, "м/с");

        auto wallBox = new QGroupBox("Материал стенки");
        auto wallForm = new QFormLayout(wallBox);

        wall.lambda = new CleanDoubleSpinBox;
        wall.lambda->setRangeWithWarning(10.0, 500.0, "Сталь: ~15-50, Медь: ~380 Вт/(м·К)");

        wall.fouling = new CleanDoubleSpinBox;
        wall.fouling->setDecimals(6);
        wall.fouling->setRangeWithWarning(0.0, 0.05, "Коэффициент загрязнения обычно крайне мал");

        addRowWithUnit(wallForm, "Теплопроводность", wall.lambda, "Вт/(м·К)");
        addRowWithUnit(wallForm, "Коэф. загрязнения", wall.fouling, "м²·К/Вт");

        layout->addWidget(balanceBox);
        layout->addWidget(hotBox);
        layout->addWidget(coldBox);
        layout->addWidget(wallBox);
        layout->addStretch();

        setupAutoCalc();
        updateAverageTemps();

        return w;
    }

    void CalcManagerWidget::setupAutoCalc()
    {
        auto hook = [this](QDoubleSpinBox* s) {
            connect(s, QOverload<double>::of(&QDoubleSpinBox::valueChanged),
                    this, &CalcManagerWidget::onBalanceEdited);
        };

        hook(hot.G); hook(hot.t_in); hook(hot.t_out); hook(hot.cp);
        hook(cold.G); hook(cold.t_in); hook(cold.t_out); hook(cold.cp);
    }

    void CalcManagerWidget::onBalanceEdited()
    {
        solveBalance();
        updateAverageTemps();
    }

    void CalcManagerWidget::solveBalance()
    {
        double G1 = hot.G->value();
        double T1i = hot.t_in->value();
    }

```

```

double T1o = hot.t_out->value();
double cp1 = hot.cp->value();

double G2 = cold.G->value();
double T2i = cold.t_in->value();
double T2o = cold.t_out->value();
double cp2 = cold.cp->value();

int unknowns = 0;
if (G1 <= 0) unknowns++;
if (T1i == 0) unknowns++;
if (T1o == 0) unknowns++;
if (G2 <= 0) unknowns++;
if (T2i == 0) unknowns++;
if (T2o == 0) unknowns++;

if (unknowns != 1) return;

if (G1 > 0 && cp1 > 0 && T1i != 0 && T1o != 0) {
    double Q = G1 * cp1 * qAbs(T1i - T1o);

    if (T2o == 0 && G2 > 0 && cp2 > 0) {
        cold.t_out->blockSignals(true);
        cold.t_out->setValue(T2i + Q / (G2 * cp2));
        cold.t_out->blockSignals(false);
    }
    else if (T2i == 0 && G2 > 0 && cp2 > 0) {
        cold.t_in->blockSignals(true);
        cold.t_in->setValue(T2o - Q / (G2 * cp2));
        cold.t_in->blockSignals(false);
    }
    else if (G2 <= 0 && cp2 > 0 && qAbs(T2o - T2i) > 0.01) {
        cold.G->blockSignals(true);
        cold.G->setValue(Q / (cp2 * qAbs(T2o - T2i)));
        cold.G->blockSignals(false);
    }
}
else if (G2 > 0 && cp2 > 0 && T2i != 0 && T2o != 0) {
    double Q = G2 * cp2 * qAbs(T2o - T2i);

    if (T1o == 0 && G1 > 0 && cp1 > 0) {
        hot.t_out->blockSignals(true);
        hot.t_out->setValue(T1i - Q / (G1 * cp1));
        hot.t_out->blockSignals(false);
    }
    else if (T1i == 0 && G1 > 0 && cp1 > 0) {
        hot.t_in->blockSignals(true);
        hot.t_in->setValue(T1o + Q / (G1 * cp1));
        hot.t_in->blockSignals(false);
    }
    else if (G1 <= 0 && cp1 > 0 && qAbs(T1i - T1o) > 0.01) {
        hot.G->blockSignals(true);
        hot.G->setValue(Q / (cp1 * qAbs(T1i - T1o)));
        hot.G->blockSignals(false);
    }
}
}

void CalcManagerWidget::updateAverageTemps()
{
    hot.t_avg->setValue((hot.t_in->value() + hot.t_out->value()) / 2.0);
    cold.t_avg->setValue((cold.t_in->value() + cold.t_out->value()) / 2.0);
}

QWidget* CalcManagerWidget::createGeometryTab()
{
    auto w = new QWidget;
    auto layout = new QVBoxLayout(w);

```

```

auto box = new QGroupBox("Геометрия");
auto form = new QFormLayout(box);

auto makeChannelSB = []() {
    auto s = new CleanDoubleSpinBox;
    s->setRangeWithWarning(8.0, 25.0, "Ширина канала обычно от 8 до 25 мм");
    s->setDecimals(1);
    return s;
};

auto makeCoreSB = []() {
    auto s = new CleanDoubleSpinBox;
    s->setRangeWithWarning(150.0, 1000.0, "Внутренний диаметр обычно от 150 до 1000
мм");
    s->setDecimals(1);
    return s;
};

auto makeWallThickSB = []() {
    auto s = new CleanDoubleSpinBox;
    s->setRangeWithWarning(0.4, 30.0, "Толщина листа спирали обычно от 0.4 до 30
мм");
    s->setDecimals(1);
    return s;
};

geom.width_hot = makeChannelSB();
geom.width_cold = makeChannelSB();
geom.d_inner = makeCoreSB();
geom.thickness_mm = makeWallThickSB();

addRowWithUnit(form, "Ширина канала горячей среды", geom.width_hot, "мм");
addRowWithUnit(form, "Ширина канала холодной среды", geom.width_cold, "мм");
addRowWithUnit(form, "Внутренний диаметр спирали", geom.d_inner, "мм");
addRowWithUnit(form, "Толщина стенки листа", geom.thickness_mm, "мм");

auto widthBox = new QGroupBox("Ширина теплообменника");
auto widthLayout = new QVBoxLayout(widthBox);

geom.autoWidthCheck = new QCheckBox("Минимальная подходящая");
geom.autoWidthCheck->setChecked(true);
widthLayout->addWidget(geom.autoWidthCheck);

auto makeWidthSB = []() {
    auto s = new CleanDoubleSpinBox;
    s->setRangeWithWarning(100.0, 1500.0,
        "Стандартные ширины: 400, 500, 700, 1000, 1100, 1250 мм");
    s->setDecimals(0);
    s->setSingleStep(50);
    s->setButtonSymbols(QAbstractSpinBox::NoButtons);
    return s;
};

geom.widthMin = makeWidthSB();
geom.widthMax = makeWidthSB();
geom.widthMin->setValue(400);
geom.widthMax->setValue(1250);

auto rangeWidget = new QWidget;
auto rangeLayout = new QHBoxLayout(rangeWidget);
rangeLayout->setContentsMargins(0, 0, 0, 0);
rangeLayout->setSpacing(6);
rangeLayout->addWidget(new QLabel("от"));
rangeLayout->addWidget(geom.widthMin);
rangeLayout->addWidget(new QLabel("мм до"));
rangeLayout->addWidget(geom.widthMax);
rangeLayout->addWidget(new QLabel("мм"));
rangeLayout->addStretch();

```

```

widthLayout->addWidget(rangeWidget);

rangeWidget->setEnabled(false);
connect(geom.autoWidthCheck, &QCheckBox::toggled, this, [=](bool checked) {
    rangeWidget->setEnabled(!checked);
});

layout->addWidget(box);
layout->addWidget(widthBox);
layout->addStretch();
return w;
}

void CalcManagerWidget::addRowWithUnit(QFormLayout* form, const QString& name,
QDoubleSpinBox* spin, const QString& unit)
{
    auto container = new QWidget;
    auto hLayout = new QHBoxLayout(container);
    hLayout->addWidget(spin);
    auto l = new QLabel(unit);
    l->setFixedWidth(60);
    hLayout->addWidget(l);
    hLayout->setContentsMargins(0, 0, 0, 0);
    hLayout->setSpacing(4);
    form->addRow(name, container);
}

void CalcManagerWidget::setDefaults()
{
    this->blockSignals(true);

    hot.G->setValue(9.72);
    hot.t_in->setValue(90.0);
    hot.t_out->setValue(50.0);
    hot.rho->setValue(1160.0);
    hot.cp->setValue(3730.0);
    hot.lambda->setValue(0.667);
    hot.viscosity->setValue(0.373);
    hot.max_velocity->setValue(2.0);

    cold.G->setValue(15.7);
    cold.t_in->setValue(20.0);
    cold.t_out->setValue(40.0);
    cold.rho->setValue(997.0);
    cold.cp->setValue(4174.0);
    cold.lambda->setValue(0.578);
    cold.viscosity->setValue(0.715);
    cold.max_velocity->setValue(2.5);

    wall.lambda->setValue(45.0);
    wall.fouling->setValue(0.00036);

    geom.width_hot->setValue(12.0);
    geom.width_cold->setValue(12.0);
    geom.d_inner->setValue(300.0);
    geom.thickness_mm->setValue(4.0);
    geom.autoWidthCheck->setChecked(true);
    geom.widthMin->setValue(400);
    geom.widthMax->setValue(1250);

    this->blockSignals(false);
    updateAverageTemps();
}

Fluid CalcManagerWidget::toFluid(const FluidUI& ui)
{
    Fluid f;
    f.G = ui.G->value();

```

```

        f.t_in = ui.t_in->value();
        f.t_out = ui.t_out->value();
        f.rho = ui.rho->value();
        f.cp = ui.cp->value();
        f.lambda = ui.lambda->value();
        f.viscosity = ui.viscosity->value() / 1000.0;
        f.max_velocity = ui.max_velocity->value();
        return f;
    }

    Wall CalcManagerWidget::toWall(const WallUI& ui)
    {
        Wall w;
        w.lambda = wall.lambda->value();
        w.thickness = geom.thickness_mm->value() / 1000.0;
        w.fouling_factor = wall.fouling->value();
        return w;
    }

    Geometry CalcManagerWidget::toGeometry(const GeometryUI& ui)
    {
        Geometry g;
        g.width_hot = geom.width_hot->value() / 1000;
        g.width_cold = geom.width_cold->value() / 1000;
        g.d_inner = geom.d_inner->value() / 1000;
        g.cover_thickness = 0.020;
        g.shell_thickness = 0.040;

        g.auto_width = geom.autoWidthCheck->isChecked();
        g.width_min = geom.widthMin->value() / 1000.0;
        g.width_max = geom.widthMax->value() / 1000.0;

        return g;
    }

    void CalcManagerWidget::onCalculate()
    {
        Fluid hotF = toFluid(hot);
        Fluid coldF = toFluid(cold);
        Wall wallM = toWall(wall);
        Geometry geomM = toGeometry(geom);

        SpiralHeatExchanger hx(hotF, coldF, wallM, geomM);

        CalculationResult result = hx.calculate();
        if (!result.error_message.empty()) {
            qDebug() << QString::fromStdString(result.error_message);
            return;
        }

        auto w = new ResultDialog({ result }, this);
        w->setWindowTitle("Результат расчета");
        w->resize(700, 400);
        w->show();

        connect(w, &ResultDialog::selectedResultConfirmed, this, [=](const
CalculationResult& r) {
            buildModel(r);
            w->close();
        });
    }

    void CalcManagerWidget::buildModelFromResult(const CalculationResult& result) {
        buildModel(result);
    }

    void CalcManagerWidget::buildModel(const CalculationResult& result) {

```

```

    auto* progress = new QProgressDialog("Построение модели...", QString(), 0, 12,
this);
    progress->setWindowTitle("Создание 3D-модели");
    progress->setWindowModality(Qt::WindowModal);
    progress->setMinimumDuration(0);
    progress->setAutoClose(false);
    progress->setAutoReset(false);
    progress->setCancelButton(nullptr);
    progress->setValue(0);
    progress->show();

    auto* thread = new QThread(this);
    auto* worker = new ModelBuildWorker(result);
    worker->moveToThread(thread);

    connect(worker, &ModelBuildWorker::progress, this,
        [progress](int step, int, const QString& desc) {
            progress->setValue(step);
            progress->setLabelText(desc);
        }, Qt::QueuedConnection);

    connect(worker, &ModelBuildWorker::finished, this,
        [this, progress, thread](SPtr<MbAssembly> assembly) {
            progress->setValue(progress->maximum());
            progress->close();
            progress->deleteLater();

            if (m_viewport)
                m_viewport->displayModel(assembly);

            thread->quit();
        }, Qt::QueuedConnection);

    connect(thread, &QThread::started, worker, &ModelBuildWorker::run);
    connect(thread, &QThread::finished, worker, &QObject::deleteLater);
    connect(thread, &QThread::finished, thread, &QObject::deleteLater);

    thread->start();
}

```

Код файла CalcSceneWidget.h

```

#pragma once

#include <QWidget>
#include <solid.h>
#include "assembly.h"
#include <vs_n_cuttingtool.h>

namespace VSN {
    namespace QtVision {
        class QtOpenGLSceneWidget;
    }

    class SceneSegment;
}

enum class CuttingPlane {
    X,
    Y,
    Z
};

class CalcSceneWidget : public QWidget
{
    Q_OBJECT

```



```

public:
    explicit CalcSceneWidget(QWidget* parent = nullptr);

    SPtr<MbAssembly> getAssembly() const { return m_assembly; }

    void togglePlaneCutting(bool enable, CuttingPlane plane = CuttingPlane::Y);

    void displaySolid(const c3d::SolidSPtr& solid);

    void clearScene();

    void displayModel(const SPtr<MbAssembly>& assembly);

    VSN::QtVision::QtOpenGLSceneWidget* scene() const
    {
        return m_scene;
    }

protected:
    void mousePressEvent(QMouseEvent* event) override;
    void mouseMoveEvent(QMouseEvent* event) override;
    void mouseReleaseEvent(QMouseEvent* event) override;
    void wheelEvent(QWheelEvent* event) override;
    void showEvent(QShowEvent* event) override;

private:
    void initScene();

    c3d::SolidSPtr createTestSolid();

private:
    VSN::QtVision::QtOpenGLSceneWidget* m_scene = nullptr;

    VSN::SceneSegment* m_currentSegment = nullptr;

    SPtr<MbAssembly> m_assembly;

    uint32_t m_curIdPlane = 0;

    QPoint m_lastMousePos;

    bool m_firstMouse = true;

    float rotate_sensitivity = 10.0f;
    float move_sensitivity = 0.001f;
    float scale_sensitivity = 0.002f;

    QPointF start_position;
};

```

Код файла CalcSceneWidget.cpp

```

#include "CalcSceneWidget.h"

#include <QVBoxLayout>
#include <QMouseEvent>
#include <QWheelEvent>
#include <QTimer>

```

```

#include <vsn_namespace.h>
#include <vsn_prcameraorbit.h>
#include <vsn_prcamerarotate.h>
#include <vsn_geometryfactory.h>
#include <vsn_scenecontent.h>
#include <vsn_scenesegment.h>
#include <vsn_material.h>

#include <widget.h>
#include <solid.h>
#include <action_solid.h>
#include <vector>

using namespace std;
using namespace VSN;
using namespace c3d;

c3d::SolidSPtr CalcSceneWidget::createTestSolid()
{
    SpacePointsVector points;
    points.push_back(MbCartPoint3D(5, 0, -5));
    points.push_back(MbCartPoint3D(-5, -5, 0));
    points.push_back(MbCartPoint3D(0, 5, 5));

    MbSNameMaker names(ct_ElementarySolid, MbSNameMaker::i_SideNone, 0);
    MbElementarySolidResults results;

    MbResultType r = ::ElementarySolid(
        MbElementarySolidParams(et_Cylinder, points, names),
        results
    );

    if (r != rt_Success) return nullptr;

    c3d::SolidSPtr solid = results.GetResultSolid();

    return solid;
}

CalcSceneWidget::CalcSceneWidget(QWidget* parent)
: QWidget(parent)
{
    auto layout = new QVBoxLayout(this);
    layout->setContentsMargins(0, 0, 0, 0);

    initScene();
}

void CalcSceneWidget::initScene()
{
    setMouseTracking(true);

    m_scene = new VSN::QtVision::QtOpenGLSceneWidget(this);
    m_scene->setMouseTracking(true);
    m_scene->setFocusPolicy(Qt::StrongFocus);

    m_scene->viewport()->SetVisibleSceneWidgets(true);
    m_scene->viewport()->SetVisibleViewWidgets(true);

    layout()->addWidget(m_scene);
}

```

```

        //displaySolid(createTestSolid());
    }

void CalcSceneWidget::displaySolid(const c3d::SolidSPtr& solid)
{
    if (!m_scene || !solid) return;

    auto root = m_scene->sceneContent()->GetRootSegment();
    auto rep = VSN::GeometryFactory::Instance()->CreateMathRep(solid,
VSN::CommandType::Synchronous, 0.05);

    if (!rep) return;

    m_currentSegment = new VSN::SceneSegment(rep, root);
    m_currentSegment->CreateRenderObject();
    m_currentSegment->SetColorPresentationMaterial(VSN::Color(30, 100, 220));
    m_scene->update();
}

void CalcSceneWidget::clearScene() {
    if (!m_scene) return;

    m_assembly = nullptr;

    auto content = m_scene->sceneContent();
    if (content) {
        auto root = content->GetRootSegment();
        if (root) {
            root->RemoveChildren();
        }
    }

    m_scene->blockSignals(false);
    m_scene->update();
}

void CalcSceneWidget::displayModel(const SPtr<MbAssembly>& assembly)
{
    if (!m_scene || !assembly)
        return;

    clearScene();

    m_assembly = assembly;

    auto root = m_scene->sceneContent()->GetRootSegment();

    RArray<MbItem> items;
    assembly->GetItems(items);

    for (size_t i = 0; i < items.Count(); ++i)
    {
        MbItem* item = items[i];

        if (!item || item->IsA() != st_Instance)
            continue;

        MbInstance* instance = static_cast<MbInstance*>(item);
        const MbItem* instanceItem = instance->GetItem();
    }
}

```

```

        if (!instanceItem || instanceItem->IsA() != st_Solid)
            continue;

        const MbSolid* solid = static_cast<const MbSolid*>(instanceItem);

        MbSolid* solidRaw = const_cast<MbSolid*>(solid);
        solidRaw->AddRef();
        c3d::SolidSPtr solidPtr(solidRaw);

        auto rep = VSN::GeometryFactory::Instance()->CreateMathRep(
            solidPtr,
            VSN::CommandType::Synchronous,
            0.05
        );

        if (!rep)
            continue;

        auto segment = new VSN::SceneSegment(rep, root);
        segment->CreateRenderObject();

        segment->SetColorPresentationMaterial(VSN::Color(30, 100, 220));

        MbPlacement3D placement;
        instance->GetPlacement(placement);

        MbMatrix3D mx;
        placement.GetMatrixFrom(mx);

        VSN::Matrix3DF vsnMatrix(
            (float)mx.El(0, 0), (float)mx.El(0, 1), (float)mx.El(0, 2),
            (float)mx.El(0, 3),
            (float)mx.El(1, 0), (float)mx.El(1, 1), (float)mx.El(1, 2),
            (float)mx.El(1, 3),
            (float)mx.El(2, 0), (float)mx.El(2, 1), (float)mx.El(2, 2),
            (float)mx.El(2, 3),
            (float)mx.El(3, 0), (float)mx.El(3, 1), (float)mx.El(3, 2),
            (float)mx.El(3, 3)
        );

        segment->GetTransform().SetMatrix(vsnMatrix);
        segment->RefreshAbsoluteMatrix();
    }

    m_scene->update();
}

void CalcSceneWidget::mousePressEvent(QMouseEvent* event)
{
    m_lastMousePos = event->pos();
    m_firstMouse = false;

    if (event->button() == Qt::RightButton) {
        QPointF p = event->position();
        start_position = QPointF(p.x() / width() - 0.5, p.y() / height() - 0.5);
    }
    m_scene->update();
}

void CalcSceneWidget::mouseMoveEvent(QMouseEvent* event)
{

```

```

    if (!m_scene || !(event->buttons() & (Qt::RightButton | Qt::MiddleButton))) {
        m_firstMouse = true;
        return;
    }

    auto cam = m_scene->camera();
    if (!cam) return;

    if (m_firstMouse) {
        m_lastMousePos = event->pos();
        m_firstMouse = false;
        return;
    }

    float dx = event->pos().x() - m_lastMousePos.x();
    float dy = m_lastMousePos.y() - event->pos().y();

    m_lastMousePos = event->pos();

    if (event->buttons() & Qt::RightButton) {
        dx = dx / width() * rotate_sensitivity;
        dy = dy / height() * rotate_sensitivity;

        cam->RotateAboutTarget(cam->GetUpVector(), -dx);
        cam->RotateAboutTarget(cam->GetSideVector(), dy);
    }

    if (event->buttons() & Qt::MiddleButton) {
        float dist = cam->DistanceCameraToTarget();
        cam->Pan(Vector3DF(-dx * move_sensitivity * dist,
                        -dy * move_sensitivity * dist,
                        0));
    }

    m_scene->update();
}

void CalcSceneWidget::mouseReleaseEvent(QMouseEvent* event)
{
    m_firstMouse = true;
    m_lastMousePos = event->pos();

    m_scene->update();
}

void CalcSceneWidget::showEvent(QShowEvent* event)
{
    QWidget::showEvent(event);

    QTimer::singleShot(0, this, [this]() {
        m_scene->viewport()->SetGradientBackgroundColour(
            VSN::Color(255, 255, 255), VSN::Color(255, 255, 255));
        m_scene->viewport()->SetBackgroundColour(VSN::Color(255, 255, 255));
        m_scene->update();
    });
}

void CalcSceneWidget::wheelEvent(QWheelEvent* event)
{
    if (!m_scene) return;
    auto cam = m_scene->camera();

```

```

    if (!cam) return;

    float scale = std::pow(2.0f, event->angleDelta().y()) * scale_sensitivity);
    float dist = cam->DistanceCameraToTarget();

    cam->SetZoom(scale);
    cam->Pan(Vector3DF((event->position().x() / width() - 0.5f) * (scale - 1.0f) *
dist,
    -(event->position().y() / height() - 0.5f) * (scale - 1.0f) * dist,
    0));

    m_scene->update();
}

void CalcSceneWidget::togglePlaneCutting(bool enable, CuttingPlane planeType)
{
    if (!m_scene) return;
    auto tool = m_scene->graphicsScene()->GetCuttingTool();

    if (m_curIdPlane != 0) {
        tool->DeleteSectionPlane(m_curIdPlane);
        m_curIdPlane = 0;
    }

    if (!enable) {
        m_scene->update();
        return;
    }

    MbCube box = m_scene->sceneContent()->GetBoundingBox();
    MbCartPoint3D center;
    box.GetCenter(center);

    MbVector3D normal(0, 1, 0);
    if (planeType == CuttingPlane::X) normal.Init(1, 0, 0);
    if (planeType == CuttingPlane::Z) normal.Init(0, 0, 1);

    m_curIdPlane = tool->AddSectionPlane(MbPlacement3D(center, normal));

    tool->SetEnable(m_curIdPlane, true);
    tool->EnableInteractiveMode(m_curIdPlane, false);

    m_scene->update();
}

```

Код файла ResultDialog.h

```

#pragma once
#include <QDialog>
#include <QDoubleSpinBox>
#include <vector>
#include <optional>
#include "SpiralHeatExchanger.h"

class QFormLayout;
class QLabel;
class QPushButton;

class ResultDialog : public QDialog
{
    Q_OBJECT

```

```

public:
    explicit ResultDialog(const CalculationResult& res, QWidget* parent = nullptr);

signals:
    void selectedResultConfirmed(const CalculationResult& r);

private slots:
    void openFlangeEditor();
    void openShellFlangeEditor();
    void onSaveCalc();

private:
    void addRow(QFormLayout* layout, const QString& label,
               const QString& value, bool isHeader = false);

    void addRowHotCold(QFormLayout* layout, const QString& label,
                       const QString& hotValue, const QString& coldValue,
                       const QString& suffix = {});

    void initStandardFlanges();
    void autoSelectFlange();
    QString getFlangeSummary() const;

    void autoSelectShellFlange();
    QString getShellFlangeSummary() const;

    CalculationResult m_result;
    std::vector<Flange> m_standardFlanges;

    std::optional<Flange> m_currentSelectedFlange;
    std::optional<Flange> m_currentShellFlange;

    QLabel* m_flangeLabel = nullptr;
    QLabel* m_shellFlangeLabel = nullptr;
    QPushButton* m_confirmBtn = nullptr;
    QPushButton* m_saveCalcBtn = nullptr;
    QDoubleSpinBox* m_gasketThicknessSpin = nullptr;
    QDoubleSpinBox* m_shellThicknessSpin = nullptr;
    QDoubleSpinBox* m_coverThicknessSpin = nullptr;

    QStringList buildWarnings() const;
};

```

Код файла ResultDialog.cpp

```

#include "ResultDialog.h"
#include "FlangeConfigDialog.h"
#include "ShellFlangeConfigDialog.h"
#include "CalcResultSerializer.h"

#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QLabel>
#include <QPushButton>
#include <QScrollArea>
#include <QFrame>
#include <QFileDialog>
#include <QMessageBox>
#include <algorithm>
#include <cmath>

```

```

ResultDialog::ResultDialog(const CalculationResult& res, QWidget* parent)
: QDialog(parent), m_result(res)
{
    setWindowTitle("Подробные результаты расчета");
    this->setStyleSheet("QWidget { font-size: 10pt; }");

    initStandardFlanges();
    autoSelectFlange();
    autoSelectShellFlange();

    auto* mainLayout = new QVBoxLayout(this);
    mainLayout->setContentsMargins(0, 0, 0, 8);

    if (!m_result.error_message.empty()) {
        auto* err = new QLabel(QString::fromStdString(m_result.error_message));
        err->setStyleSheet("font-weight: bold; color: red;");
        err->setWordWrap(true);
        mainLayout->addWidget(err);
        auto* close = new QPushButton("Заккрыть");
        connect(close, &QPushButton::clicked, this, &QDialog::reject);
        mainLayout->addWidget(close);
        return;
    }

    auto* container = new QWidget();
    auto* form = new QFormLayout(container);
    form->setSpacing(8);
    form->setLabelAlignment(Qt::AlignLeft);

    addRow(form, "ТЕПЛОФИЗИКА", "", true);
    addRow(form, "Требуемая мощность:",
        QString::number(m_result.Q_required / 1000.0, 'f', 2) + " кВт");
    addRow(form, "LMTD (напор):",
        QString::number(m_result.LMTD_real, 'f', 2) + " °C");
    addRow(form, "Коэф. теплопередачи:",
        QString::number(m_result.K, 'f', 1) + " Вт/(м²·К)");
    addRowHotCold(form, "Коэф. теплоотдачи (гор/хол):",
        QString::number(m_result.alpha_hot, 'f', 1),
        QString::number(m_result.alpha_cold, 'f', 1));
    addRow(form, "Теплопроводность стенки:",
        QString::number(m_result.wall.lambda, 'f', 1) + " Вт/(м·К)");

    addRow(form, "ГИДРОДИНАМИКА", "", true);
    addRowHotCold(form, "Скорость среды (гор/хол):",
        QString::number(m_result.velocity_hot, 'f', 2),
        QString::number(m_result.velocity_cold, 'f', 2),
        "м/с");
    addRowHotCold(form, "Число Re среды (гор/хол):",
        QString::number((int)m_result.Re_hot),
        QString::number((int)m_result.Re_cold));

    addRow(form, "ГЕОМЕТРИЯ КОРПУСА", "", true);
    addRow(form, "Наружный диаметр:",
        QString::number(m_result.D * 1000.0, 'f', 1) + " мм");
    addRow(form, "Площадь поверхности:",
        QString::number(m_result.A, 'f', 2) + " м²");
    addRow(form, "Количество витков:",
        QString::number(m_result.turns, 'f', 1));
    addRowHotCold(form, "Ширина канала (гор/хол):",
        QString::number(m_result.geometry.width_hot * 1000.0, 'f', 1),

```



```

        QString::number(m_result.geometry.width_cold * 1000.0, 'f', 1),
        "мм");
addRow(form, "Длина канала:",
        QString::number(m_result.channel_length, 'f', 2) + " м");
addRow(form, "Ширина листа:",
        QString::number(m_result.channel_width * 1000.0, 'f', 1) + " мм");
addRow(form, "Вн. диаметр спирали:",
        QString::number(m_result.geometry.d_inner * 1000.0, 'f', 1) + " мм");

{
    m_coverThicknessSpin = new QDoubleSpinBox();
    m_coverThicknessSpin->setRange(5.0, 200.0);
    m_coverThicknessSpin->setDecimals(1);
    m_coverThicknessSpin->setSuffix(" мм");
    m_coverThicknessSpin->setSingleStep(1.0);
    m_coverThicknessSpin->setValue(m_result.geometry.cover_thickness * 1000.0);
    m_coverThicknessSpin->setButtonSymbols(QAbstractSpinBox::NoButtons);
    m_coverThicknessSpin->setFixedWidth(90);
    form->addRow("Толщина крышки:", m_coverThicknessSpin);
}

{
    m_shellThicknessSpin = new QDoubleSpinBox();
    m_shellThicknessSpin->setRange(5.0, 200.0);
    m_shellThicknessSpin->setDecimals(1);
    m_shellThicknessSpin->setSuffix(" мм");
    m_shellThicknessSpin->setSingleStep(1.0);
    m_shellThicknessSpin->setValue(m_result.geometry.shell_thickness * 1000.0);
    m_shellThicknessSpin->setButtonSymbols(QAbstractSpinBox::NoButtons);
    m_shellThicknessSpin->setFixedWidth(90);
    form->addRow("Толщина обечайки:", m_shellThicknessSpin);
}

{
    m_gasketThicknessSpin = new QDoubleSpinBox();
    m_gasketThicknessSpin->setRange(0.5, 50.0);
    m_gasketThicknessSpin->setDecimals(1);
    m_gasketThicknessSpin->setSuffix(" мм");
    m_gasketThicknessSpin->setSingleStep(0.5);
    m_gasketThicknessSpin->setValue(m_result.gasket_thickness * 1000.0);
    m_gasketThicknessSpin->setButtonSymbols(QAbstractSpinBox::NoButtons);
    m_gasketThicknessSpin->setFixedWidth(90);
    form->addRow("Толщина прокладки:", m_gasketThicknessSpin);
}

addRow(form, "Толщина стенки листа:",
        QString::number(m_result.wall.thickness * 1000.0, 'f', 1) + " мм");

addRow(form, "ПАТРУБКИ И ФЛАНЦЫ", "", true);
addRow(form, "Расчетный диаметр входного отверстия:",
        QString::number(m_result.inlet_D * 1000.0, 'f', 1) + " мм");

{
    auto* flangeLine = new QWidget();
    auto* flangeLayout = new QHBoxLayout(flangeLine);
    flangeLayout->setContentsMargins(0, 0, 0, 0);

    m_flangeLabel = new QLabel(getFlangeSummary());
    m_flangeLabel->setStyleSheet("font-weight: 500;");

    auto* editBtn = new QPushButton("✎");

```

```

        editBtn->setFixedSize(24, 24);
        connect(editBtn, &QPushButton::clicked, this,
&ResultDialog::openFlangeEditor);

        flangeLayout->addWidget(m_flangeLabel);
        flangeLayout->addWidget(editBtn);
        flangeLayout->addStretch();
        form->addRow("Фланец патрубкa:", flangeLine);
    }

    {
        auto* sfLine = new QWidget();
        auto* sfLayout = new QHBoxLayout(sfLine);
        sfLayout->setContentsMargins(0, 0, 0, 0);

        m_shellFlangeLabel = new QLabel(getShellFlangeSummary());
        m_shellFlangeLabel->setStyleSheet("font-weight: 500;");

        auto* editBtn = new QPushButton("⚙");
        editBtn->setFixedSize(24, 24);
        connect(editBtn, &QPushButton::clicked,
            this, &ResultDialog::openShellFlangeEditor);

        sfLayout->addWidget(m_shellFlangeLabel);
        sfLayout->addWidget(editBtn);
        sfLayout->addStretch();
        form->addRow("Фланец обечайки:", sfLine);
    }

    auto* scrollContent = new QWidget();
    auto* scrollLayout = new QVBoxLayout(scrollContent);
    scrollLayout->setContentsMargins(8, 8, 8, 8);
    scrollLayout->setSpacing(4);
    scrollLayout->addWidget(container);

    scrollLayout->addStretch();

    auto* scrollArea = new QScrollArea();
    scrollArea->setWidget(scrollContent);
    scrollArea->setWidgetResizable(true);
    scrollArea->setFrameShape(QFrame::NoFrame);
    scrollArea->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
    mainLayout->addWidget(scrollArea, 1);

    {
        QStringList warnings = buildWarnings();
        if (!warnings.isEmpty()) {
            auto* warnFrame = new QFrame();
            warnFrame->setFrameShape(QFrame::StyledPanel);
            warnFrame->setStyleSheet(
                "QFrame { background: #fff8e1; border: 0px solid #f9a825; border-
radius: 4px; }"
                "QLabel { color: #5d4037; font-size: 9.5pt; }");

            auto* warnLayout = new QVBoxLayout(warnFrame);
            warnLayout->setContentsMargins(8, 6, 8, 6);
            warnLayout->setSpacing(3);

            auto* title = new QLabel("Обнаружены несоответствия:");
            title->setStyleSheet("font-weight: bold; color: #e65100;");
            warnLayout->addWidget(title);
        }
    }

```

```

        for (const QString& w : warnings) {
            auto* lbl = new QLabel("• " + w);
            lbl->setWordWrap(true);
            warnLayout->addWidget(lbl);
        }

        mainLayout->addWidget(warnFrame);
    }
}

auto* separator = new QFrame();
separator->setFrameShape(QFrame::HLine);
separator->setFrameShadow(QFrame::Sunken);
mainLayout->addWidget(separator);

auto* bottom = new QHBoxLayout();
bottom->setContentsMargins(8, 0, 8, 0);
auto* cancel = new QPushButton("Отмена");
m_saveCalcBtn = new QPushButton("Сохранить расчёт");
m_confirmBtn = new QPushButton("Построить модель");
cancel->setMinimumHeight(35);
m_saveCalcBtn->setMinimumHeight(35);
m_confirmBtn->setMinimumHeight(35);
m_confirmBtn->setDefault(true);

bottom->addWidget(m_saveCalcBtn);
bottom->addStretch();
bottom->addWidget(cancel);
bottom->addWidget(m_confirmBtn);
mainLayout->addLayout(bottom);

resize(580, 700);

connect(cancel, &QPushButton::clicked, this, &QDialog::reject);
connect(m_saveCalcBtn, &QPushButton::clicked, this, &ResultDialog::onSaveCalc);
connect(m_confirmBtn, &QPushButton::clicked, this, [this]() {
    if (m_currentSelectedFlange)
        m_result.selected_flange = *m_currentSelectedFlange;
    if (m_currentShellFlange)
        m_result.shell_flange = *m_currentShellFlange;
    if (m_gasketThicknessSpin)
        m_result.gasket_thickness = m_gasketThicknessSpin->value() / 1000.0;
    if (m_coverThicknessSpin)
        m_result.geometry.cover_thickness = m_coverThicknessSpin->value() /
1000.0;
    if (m_shellThicknessSpin)
        m_result.geometry.shell_thickness = m_shellThicknessSpin->value() /
1000.0;

    emit selectedResultConfirmed(m_result);
    accept();
});
}

void ResultDialog::addRow(QFormLayout* layout, const QString& label,
    const QString& value, bool isHeader)
{
    if (isHeader) {
        auto* lbl = new QLabel(label);
        lbl->setStyleSheet(

```

```

        "font-weight: bold; font-size: 11pt; "
        "margin-top: 10px; border-bottom: 2px solid palette(mid);");
    layout->addRow(lbl);
}
else {
    auto* val = new QLabel(value);
    val->setStyleSheet("font-weight: 500;");
    layout->addRow(new QLabel(label), val);
}
}

void ResultDialog::addRowHotCold(QFormLayout* layout, const QString& label,
    const QString& hotValue, const QString& coldValue, const QString& suffix)
{
    QString html = QString(
        "<span style='color:#cc2200;font-weight:500'>%1</span>"
        " / "
        "<span style='color:#0055cc;font-weight:500'>%2</span>%3")
        .arg(hotValue)
        .arg(coldValue)
        .arg(suffix.isEmpty() ? QString() : " " + suffix);

    auto* val = new QLabel(html);
    val->setTextFormat(Qt::RichText);
    layout->addRow(new QLabel(label), val);
}

void ResultDialog::autoSelectFlange()
{
    double target = m_result.inlet_D * 1000.0;
    for (const auto& f : m_standardFlanges) {
        if (f.inner_d >= target) {
            m_currentSelectedFlange = f;
            return;
        }
    }
    if (!m_standardFlanges.empty())
        m_currentSelectedFlange = m_standardFlanges.front();
}

QString ResultDialog::getFlangeSummary() const
{
    if (!m_currentSelectedFlange) return "Не выбран";
    return QString("%1, s=%2 мм, L=%3 мм")
        .arg(QString::fromStdString(m_currentSelectedFlange->name))
        .arg(m_result.nozzle_shell_thickness, 0, 'f', 1)
        .arg(m_result.nozzle_shell_length, 0, 'f', 1);
}

void ResultDialog::openFlangeEditor()
{
    if (!m_currentSelectedFlange) return;
    double minD = m_result.inlet_D * 1000.0;

    FlangeConfigDialog dlg(
        *m_currentSelectedFlange,
        m_result.nozzle_shell_thickness,
        m_result.nozzle_shell_length,
        m_standardFlanges,
        minD,
        this);
}

```

```

        if (dlg.exec() == QDialog::Accepted) {
            m_currentSelectedFlange = dlg.getSelectedFlange();
            m_result.nozzle_shell_thickness = dlg.getShellThickness();
            m_result.nozzle_shell_length = dlg.getShellLength();
            m_flangeLabel->setText(getFlangeSummary());
        }
    }

void ResultDialog::autoSelectShellFlange()
{
    double D_outer_mm = m_result.D * 1000.0
        + 2.0 * m_result.geometry.shell_thickness * 1000.0;

    for (const auto& f : m_standardFlanges) {
        if (f.inner_d >= D_outer_mm) {
            m_currentShellFlange = f;
            return;
        }
    }
    if (!m_standardFlanges.empty())
        m_currentShellFlange = m_standardFlanges.back();
}

QString ResultDialog::getShellFlangeSummary() const
{
    if (!m_currentShellFlange) return "Не задан";
    const auto& f = *m_currentShellFlange;
    return QString("%1 D1=%2 мм, D=%3 мм, n=%4")
        .arg(QString::fromStdString(f.name))
        .arg(f.inner_d, 0, 'f', 1)
        .arg(f.outer_d, 0, 'f', 1)
        .arg(f.bolt_count);
}

void ResultDialog::openShellFlangeEditor()
{
    if (!m_currentShellFlange) return;

    double D_outer_mm = m_result.D * 1000.0
        + 2.0 * m_result.geometry.shell_thickness * 1000.0;

    ShellFlangeConfigDialog dlg(
        *m_currentShellFlange,
        m_standardFlanges,
        D_outer_mm,
        this);

    if (dlg.exec() == QDialog::Accepted) {
        m_currentShellFlange = dlg.getResult();
        m_shellFlangeLabel->setText(getShellFlangeSummary());
    }
}

QStringList ResultDialog::buildWarnings() const
{
    QStringList warnings;
    const auto& r = m_result;

    const double d_inner_mm = r.geometry.d_inner * 1000.0;

```

```

const double inlet_D_mm = r.inlet_D * 1000.0;
const double channel_w_mm = r.channel_width * 1000.0;
const double turns = r.turns;
const double Re_hot = r.Re_hot;
const double Re_cold = r.Re_cold;
const double shell_thk_mm = r.geometry.shell_thickness * 1000.0;
const double D_outer_mm = r.D * 1000.0;

if (d_inner_mm < inlet_D_mm * 2.0)
    warnings << QString(
        "Внутренний диаметр спирали (%1 мм) меньше удвоенного расчётного "
        "диаметра входного отверстия (%2 мм × 2 = %3 мм). "
        "Рекомендуется увеличить внутренний диаметр спирали.")
        .arg(d_inner_mm, 0, 'f', 1)
        .arg(inlet_D_mm, 0, 'f', 1)
        .arg(inlet_D_mm * 2.0, 0, 'f', 1);

if (channel_w_mm < inlet_D_mm)
    warnings << QString(
        "Ширина теплообменника (%1 мм) меньше расчётного диаметра "
        "входного отверстия (%2 мм). Патрубок не вписывается в торец.")
        .arg(channel_w_mm, 0, 'f', 1)
        .arg(inlet_D_mm, 0, 'f', 1);

if (Re_hot > 0 && Re_hot < 10000)
    warnings << QString(
        "Число Re горячей среды (%1) ниже 10 000 – "
        "расчёт коэффициента теплоотдачи по формуле Дитуса–Белтера
недостововерен.")
        .arg((int)Re_hot);

if (Re_cold > 0 && Re_cold < 10000)
    warnings << QString(
        "Число Re холодной среды (%1) ниже 10 000 – "
        "расчёт коэффициента теплоотдачи по формуле Дитуса–Белтера
недостововерен.")
        .arg((int)Re_cold);

if (turns > 0 && turns < 2)
    warnings << QString(
        "Количество витков спирали (%1) меньше 2 – "
        "конструкция теплообменника нецелесообразна.")
        .arg((int)turns);

if (D_outer_mm > 4000.0)
    warnings << QString(
        "Наружный диаметр теплообменника (%1 мм) превышает 4 000 мм – "
        "проверьте исходные данные или разбейте на несколько аппаратов.")
        .arg(D_outer_mm, 0, 'f', 0);

if (m_currentShellFlange) {
    double D_shell_outer_mm = D_outer_mm + 2.0 * shell_thk_mm;
    if (m_currentShellFlange->inner_d < D_shell_outer_mm)
        warnings << QString(
            "Внутренний диаметр фланца обечайки (%1 мм) меньше "
            "наружного диаметра корпуса (%2 мм).")
            .arg(m_currentShellFlange->inner_d, 0, 'f', 1)
            .arg(D_shell_outer_mm, 0, 'f', 1);
}

if (m_currentSelectedFlange) {

```

```

        if (m_currentSelectedFlange->inner_d < inlet_D_mm)
            warnings << QString(
                "Внутренний диаметр фланца патрубка (%1 мм) меньше "
                "расчётного диаметра входного отверстия (%2 мм).")
                .arg(m_currentSelectedFlange->inner_d, 0, 'f', 1)
                .arg(inlet_D_mm, 0, 'f', 1);
    }

    return warnings;
}

void ResultDialog::initStandardFlanges()
{
    m_standardFlanges = {
        { "DN10 PN1",      15.0,  75.0,  50.0, 10.0, 10.0, 4 },
        { "DN10 PN2.5",    15.0,  75.0,  50.0, 12.0, 10.0, 4 },
        { "DN10 PN6",      15.0,  75.0,  50.0, 12.0, 10.0, 4 },
        { "DN10 PN10",     18.0,  90.0,  60.0, 14.0, 12.0, 4 },
        { "DN10 PN16",     18.0,  90.0,  60.0, 14.0, 12.0, 4 },
        { "DN10 PN25",     18.0,  90.0,  60.0, 16.0, 12.0, 4 },

        { "DN15 PN1",      19.0,  80.0,  55.0, 10.0, 10.0, 4 },
        { "DN15 PN2.5",    19.0,  80.0,  55.0, 12.0, 10.0, 4 },
        { "DN15 PN6",      19.0,  80.0,  55.0, 12.0, 10.0, 4 },
        { "DN15 PN10",     22.0,  95.0,  65.0, 14.0, 12.0, 4 },
        { "DN15 PN16",     22.0,  95.0,  65.0, 14.0, 12.0, 4 },
        { "DN15 PN25",     22.0,  95.0,  65.0, 16.0, 12.0, 4 },

        { "DN20 PN1",      26.0,  90.0,  65.0, 12.0, 10.0, 4 },
        { "DN20 PN2.5",    26.0,  90.0,  65.0, 14.0, 10.0, 4 },
        { "DN20 PN6",      26.0,  90.0,  65.0, 14.0, 10.0, 4 },
        { "DN20 PN10",     27.5, 105.0,  75.0, 16.0, 12.0, 4 },
        { "DN20 PN16",     27.5, 105.0,  75.0, 16.0, 12.0, 4 },
        { "DN20 PN25",     27.5, 105.0,  75.0, 18.0, 12.0, 4 },

        { "DN25 PN1",      33.0, 100.0,  75.0, 12.0, 10.0, 4 },
        { "DN25 PN2.5",    33.0, 100.0,  75.0, 14.0, 10.0, 4 },
        { "DN25 PN6",      33.0, 100.0,  75.0, 14.0, 10.0, 4 },
        { "DN25 PN10",     34.5, 115.0,  85.0, 16.0, 12.0, 4 },
        { "DN25 PN16",     34.5, 115.0,  85.0, 16.0, 12.0, 4 },
        { "DN25 PN25",     34.5, 115.0,  85.0, 18.0, 12.0, 4 },

        { "DN32 PN1",      39.0, 120.0,  90.0, 12.0, 12.0, 4 },
        { "DN32 PN2.5",    39.0, 120.0,  90.0, 16.0, 12.0, 4 },
        { "DN32 PN6",      39.0, 120.0,  90.0, 15.0, 12.0, 4 },
        { "DN32 PN10",     43.5, 135.0, 100.0, 16.0, 16.0, 4 },
        { "DN32 PN16",     43.5, 140.0, 100.0, 18.0, 16.0, 4 },
        { "DN32 PN25",     43.5, 140.0, 100.0, 20.0, 16.0, 4 },

        { "DN40 PN1",      46.0, 130.0, 100.0, 13.0, 12.0, 4 },
        { "DN40 PN2.5",    46.0, 130.0, 100.0, 16.0, 12.0, 4 },
        { "DN40 PN6",      46.0, 130.0, 100.0, 16.0, 12.0, 4 },
        { "DN40 PN10",     49.5, 145.0, 110.0, 18.0, 16.0, 4 },
        { "DN40 PN16",     49.5, 150.0, 110.0, 18.0, 16.0, 4 },
        { "DN40 PN25",     49.5, 150.0, 110.0, 22.0, 16.0, 4 },

        { "DN50 PN1",      59.0, 140.0, 110.0, 13.0, 12.0, 4 },
        { "DN50 PN2.5",    59.0, 140.0, 110.0, 16.0, 12.0, 4 },
        { "DN50 PN6",      59.0, 140.0, 110.0, 16.0, 12.0, 4 },
        { "DN50 PN10",     61.5, 160.0, 125.0, 18.0, 16.0, 4 },
    }
}

```

```

{ "DN50 PN16", 61.5, 165.0, 125.0, 20.0, 16.0, 4 },
{ "DN50 PN25", 61.5, 165.0, 125.0, 24.0, 16.0, 4 },

{ "DN65 PN1", 78.0, 160.0, 130.0, 14.0, 12.0, 4 },
{ "DN65 PN2.5", 78.0, 160.0, 130.0, 16.0, 12.0, 4 },
{ "DN65 PN6", 78.0, 160.0, 130.0, 16.0, 12.0, 4 },
{ "DN65 PN10", 77.5, 180.0, 145.0, 20.0, 16.0, 4 },
{ "DN65 PN16", 77.5, 185.0, 145.0, 20.0, 16.0, 8 },
{ "DN65 PN25", 77.5, 185.0, 145.0, 22.0, 16.0, 8 },

{ "DN80 PN1", 91.0, 185.0, 150.0, 14.0, 18.0, 4 },
{ "DN80 PN2.5", 91.0, 185.0, 150.0, 14.0, 18.0, 4 },
{ "DN80 PN6", 91.0, 185.0, 150.0, 18.0, 18.0, 4 },
{ "DN80 PN10", 90.5, 190.0, 150.0, 20.0, 18.0, 4 },
{ "DN80 PN16", 90.5, 195.0, 160.0, 24.0, 18.0, 8 },
{ "DN80 PN25", 90.5, 200.0, 160.0, 24.0, 18.0, 8 },

{ "DN100 PN1", 110.0, 205.0, 170.0, 14.0, 18.0, 4 },
{ "DN100 PN2.5", 110.0, 205.0, 170.0, 18.0, 18.0, 4 },
{ "DN100 PN6", 100.0, 205.0, 170.0, 18.0, 18.0, 4 },
{ "DN100 PN10", 116.0, 210.0, 180.0, 22.0, 18.0, 8 },
{ "DN100 PN16", 116.0, 220.0, 180.0, 22.0, 18.0, 8 },
{ "DN100 PN25", 116.0, 235.0, 190.0, 26.0, 22.0, 8 },

{ "DN125 PN1", 135.0, 235.0, 200.0, 16.0, 18.0, 8 },
{ "DN125 PN2.5", 135.0, 235.0, 200.0, 16.0, 18.0, 8 },
{ "DN125 PN6", 135.0, 240.0, 200.0, 20.0, 18.0, 8 },
{ "DN125 PN10", 141.5, 245.0, 210.0, 24.0, 18.0, 8 },
{ "DN125 PN16", 141.5, 250.0, 210.0, 28.0, 18.0, 8 },
{ "DN125 PN25", 141.5, 270.0, 220.0, 22.0, 26.0, 8 },

{ "DN150 PN1", 154.0, 260.0, 225.0, 16.0, 18.0, 8 },
{ "DN150 PN2.5", 154.0, 260.0, 225.0, 16.0, 18.0, 8 },
{ "DN150 PN6", 154.0, 265.0, 225.0, 20.0, 18.0, 8 },
{ "DN150 PN10", 170.5, 280.0, 240.0, 24.0, 22.0, 8 },
{ "DN150 PN16", 170.5, 285.0, 240.0, 24.0, 22.0, 8 },
{ "DN150 PN25", 170.5, 300.0, 250.0, 30.0, 26.0, 8 },

{ "DN200 PN1", 222.0, 315.0, 280.0, 18.0, 18.0, 8 },
{ "DN200 PN2.5", 222.0, 315.0, 280.0, 22.0, 18.0, 8 },
{ "DN200 PN6", 222.0, 320.0, 280.0, 22.0, 18.0, 8 },
{ "DN200 PN10", 221.5, 335.0, 295.0, 24.0, 22.0, 8 },
{ "DN200 PN16", 221.5, 340.0, 295.0, 26.0, 22.0, 8 },
{ "DN200 PN25", 221.5, 360.0, 310.0, 32.0, 26.0, 12 },

{ "DN400 PN1", 426.0, 535.0, 495.0, 22.0, 22.0, 16 },
{ "DN400 PN2.5", 426.0, 540.0, 495.0, 28.0, 22.0, 16 },
{ "DN400 PN6", 426.0, 540.0, 495.0, 28.0, 22.0, 16 },
{ "DN400 PN10", 411.0, 565.0, 515.0, 32.0, 26.0, 16 },
{ "DN400 PN16", 411.0, 580.0, 525.0, 38.0, 30.0, 16 },
{ "DN400 PN25", 411.0, 620.0, 550.0, 48.0, 36.0, 16 },

{ "DN800 PN1", 820.0, 975.0, 920.0, 26.0, 30.0, 24 },
{ "DN800 PN2.5", 820.0, 975.0, 920.0, 44.0, 30.0, 24 },
{ "DN800 PN6", 820.0, 975.0, 920.0, 44.0, 30.0, 24 },
{ "DN800 PN10", 820.0, 1015.0, 950.0, 56.0, 33.0, 24 },
{ "DN800 PN16", 820.0, 1025.0, 950.0, 74.0, 39.0, 24 },
{ "DN800 PN25", 820.0, 1085.0, 990.0, 95.0, 48.0, 24 },

{ "DN900 PN1", 920.0, 1075.0, 1020.0, 28.0, 30.0, 24 },
{ "DN900 PN2.5", 920.0, 1075.0, 1020.0, 48.0, 30.0, 24 },

```



```

        { "DN900 PN6",      920.0, 1075.0, 1020.0, 48.0, 30.0, 24 },
        { "DN900 PN10",     920.0, 1115.0, 1050.0, 62.0, 33.0, 28 },
        { "DN900 PN16",     920.0, 1125.0, 1050.0, 82.0, 39.0, 28 },

        { "DN1000 PN1",     1020.0, 1175.0, 1120.0, 30.0, 30.0, 28 },
        { "DN1000 PN2.5",  1020.0, 1175.0, 1120.0, 52.0, 30.0, 28 },
        { "DN1000 PN6",     1020.0, 1175.0, 1120.0, 52.0, 30.0, 28 },
        { "DN1000 PN10",    1020.0, 1230.0, 1160.0, 70.0, 36.0, 28 },
        { "DN1000 PN16",    1020.0, 1255.0, 1170.0, 90.0, 42.0, 28 },

        { "DN1200 PN1",     1220.0, 1375.0, 1320.0, 30.0, 30.0, 32 },
        { "DN1200 PN2.5",  1220.0, 1375.0, 1320.0, 60.0, 30.0, 32 },
        { "DN1200 PN6",     1220.0, 1405.0, 1340.0, 60.0, 33.0, 32 },
        { "DN1200 PN10",    1220.0, 1455.0, 1380.0, 83.0, 39.0, 32 },
        { "DN1200 PN16",    1220.0, 1485.0, 1390.0, 76.0, 48.0, 32 },

        { "DN1400 PN1",     1420.0, 1575.0, 1520.0, 32.0, 30.0, 36 },
        { "DN1400 PN2.5",  1420.0, 1630.0, 1560.0, 72.0, 36.0, 36 },
        { "DN1400 PN10",    1420.0, 1675.0, 1590.0, 65.0, 42.0, 36 },

        { "DN1600 PN1",     1620.0, 1785.0, 1730.0, 32.0, 30.0, 40 },
        { "DN1600 PN2.5",  1620.0, 1785.0, 1730.0, 32.0, 30.0, 40 },
        { "DN1600 PN6",     1620.0, 1830.0, 1760.0, 80.0, 36.0, 40 },
        { "DN1600 PN10",    1620.0, 1915.0, 1820.0, 75.0, 48.0, 40 },

        { "DN1800 PN1",     1820.0, 1985.0, 1930.0, 35.0, 30.0, 44 },
        { "DN1800 PN2.5",  1820.0, 1985.0, 1930.0, 35.0, 30.0, 44 },
        { "DN1800 PN6",     1820.0, 2045.0, 1970.0, 88.0, 39.0, 44 },

        { "DN2000 PN1",     2020.0, 2190.0, 2130.0, 35.0, 30.0, 48 },
        { "DN2000 PN2.5",  2020.0, 2190.0, 2130.0, 35.0, 30.0, 48 },
        { "DN2000 PN6",     2020.0, 2265.0, 2180.0, 96.0, 42.0, 48 },

        { "DN2200 PN1",     2220.0, 2405.0, 2340.0, 42.0, 33.0, 52 },
        { "DN2200 PN2.5",  2220.0, 2405.0, 2340.0, 42.0, 33.0, 52 },

        { "DN2400 PN1",     2420.0, 2605.0, 2540.0, 47.0, 33.0, 56 },
    };
}

void ResultDialog::onSaveCalc()
{
    CalculationResult snapshot = m_result;
    if (m_currentSelectedFlange)
        snapshot.selected_flange = *m_currentSelectedFlange;
    if (m_currentShellFlange)
        snapshot.shell_flange = *m_currentShellFlange;
    if (m_gasketThicknessSpin)
        snapshot.gasket_thickness = m_gasketThicknessSpin->value() / 1000.0;
    if (m_coverThicknessSpin)
        snapshot.geometry.cover_thickness = m_coverThicknessSpin->value() / 1000.0;
    if (m_shellThicknessSpin)
        snapshot.geometry.shell_thickness = m_shellThicknessSpin->value() / 1000.0;

    QString selectedFilter;
    QString path = QFileDialog::getSaveFileName(
        this, "Сохранить расчёт", {},
        "Файл расчёта SpiralHX (*.txt);;"
        "PDF документ (*.pdf)",
        &selectedFilter);
    if (path.isEmpty()) return;
}

```

```

QString err;
if (selectedFilter.contains("*.pdf")) {
    if (!path.endsWith(".pdf", Qt::CaseInsensitive)) path += ".pdf";
    err = CalcResultSerializer::savePdf(snapshot, path);
}
else {
    if (!path.endsWith(".txt", Qt::CaseInsensitive)) path += ".txt";
    err = CalcResultSerializer::save(snapshot, path);
}

if (err.isEmpty())
    QMessageBox::information(this, "Сохранение", "Файл сохранён:\n" + path);
else
    QMessageBox::critical(this, "Ошибка", err);
}

```

Код файла LicenseDialog.h

```

#pragma once

#include <QDialog>

class QLineEdit;
class QPushButton;

class LicenseDialog : public QDialog
{
    Q_OBJECT

public:
    explicit LicenseDialog(QWidget* parent = nullptr);

    QString key() const;
    QString signature() const;

private:
    QLineEdit* m_keyEdit;
    QLineEdit* m_signatureEdit;
    QPushButton* m_okButton;
};

```

Код файла LicenseDialog.cpp

```

#include "LicenseDialog.h"

#include <QVBoxLayout>
#include <QLineEdit>
#include <QPushButton>

LicenseDialog::LicenseDialog(QWidget* parent)
    : QDialog(parent)
{
    setWindowTitle("License Activation");
    setModal(true);
    resize(300, 150);

    m_keyEdit = new QLineEdit(this);
    m_keyEdit->setPlaceholderText("License key");

    m_signatureEdit = new QLineEdit(this);
}

```

```

m_signatureEdit->setPlaceholderText("Signature");

m_okButton = new QPushButton("OK", this);

auto* layout = new QVBoxLayout(this);
layout->addWidget(m_keyEdit);
layout->addWidget(m_signatureEdit);
layout->addWidget(m_okButton);

connect(m_okButton, &QPushButton::clicked, this, [this]()
{
    if (m_keyEdit->text().isEmpty() || m_signatureEdit->text().isEmpty())
        return;

    accept();
});
}

QString LicenseDialog::key() const
{
    return m_keyEdit->text();
}

QString LicenseDialog::signature() const
{
    return m_signatureEdit->text();
}

```

Код файла FlangeConfigDialog.h

```

#pragma once
#include <QDialog>
#include <QComboBox>
#include <QDoubleSpinBox>
#include <QSpinBox>
#include <QCheckBox>
#include <QLabel>
#include <QStackedWidget>
#include <vector>
#include <string>
#include "SpiralHeatExchanger.h"

class FlangeConfigDialog : public QDialog {
    Q_OBJECT
public:
    FlangeConfigDialog(const Flange& current,
        double shellThicknessMm,
        double shellLengthMm,
        const std::vector<Flange>& all,
        double minInnerD,
        QWidget* parent = nullptr);

    Flange getSelectedFlange() const { return m_current; }
    double getShellThickness() const { return m_shellThickness; }
    double getShellLength() const { return m_shellLength; }

private slots:
    void updateCascade(int level);
    void onAccept();
    void onStandardToggled(bool checked);

```

```

private:
    void setupUi();
    void initCombos();
    Flange flangeFromManual() const;

    const std::vector<Flange> m_all;
    Flange m_current;
    double m_minInnerD;
    double m_shellThickness;
    double m_shellLength;

    QLabel* m_imageLabel = nullptr;
    QCheckBox* m_standardCheck = nullptr;
    QStackedWidget* m_stack = nullptr;

    QComboBox* m_combos[7];

    QDoubleSpinBox* m_mName = nullptr;
    QDoubleSpinBox* m_mInnerD = nullptr;
    QDoubleSpinBox* m_mOuterD = nullptr;
    QDoubleSpinBox* m_mBoltCircD = nullptr;
    QDoubleSpinBox* m_mFlangThk = nullptr;
    QDoubleSpinBox* m_mBoltD = nullptr;
    QSpinBox* m_mBoltN = nullptr;

    QDoubleSpinBox* m_shellThicknessEdit = nullptr;
    QDoubleSpinBox* m_shellLengthEdit = nullptr;
};

```

Код файла FlangeConfigDialog.cpp

```

#include "FlangeConfigDialog.h"
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QPushButton>
#include <QPixmap>
#include <QGroupBox>
#include <set>
#include <cmath>

FlangeConfigDialog::FlangeConfigDialog(const Flange& current,
    double shellThicknessMm,
    double shellLengthMm,
    const std::vector<Flange>& all,
    double minInnerD,
    QWidget* parent)
    : QDialog(parent)
    , m_all(all)
    , m_current(current)
    , m_minInnerD(minInnerD)
    , m_shellThickness(shellThicknessMm)
    , m_shellLength(shellLengthMm)
{
    setWindowTitle("Настройка параметров фланца патрубка");
    setupUi();
    initCombos();
}

void FlangeConfigDialog::setupUi()
{

```

```

auto* mainLayout = new QHBoxLayout(this);
mainLayout->setSizeConstraint(QLayout::SetFixedSize);

m_imageLabel = new QLabel();
m_imageLabel->setFixedSize(300, 300);
m_imageLabel->setAlignment(Qt::AlignCenter);
m_imageLabel->setStyleSheet(
    "border: 1px solid #bdc3c7; background: white; border-radius: 4px;");
QPixmap pix(":/images/flange_drawing.png");
if (!pix.isNull())
    m_imageLabel->setPixmap(
        pix.scaled(280, 280, Qt::KeepAspectRatio, Qt::SmoothTransformation));
else
    m_imageLabel->setText("Схема фланца");
mainLayout->addWidget(m_imageLabel);

auto* rightLayout = new QVBoxLayout();

m_standardCheck = new QCheckBox("Стандартные значения");
m_standardCheck->setChecked(true);
rightLayout->addWidget(m_standardCheck);

m_stack = new QStackedWidget();

auto* pageCombo = new QWidget();
auto* formCombo = new QFormLayout(pageCombo);
const QString labels[7] = {
    "Наименование:",
    "Внутренний диаметр (D1), мм:",
    "Наружный диаметр (D), мм:",
    "Болтовой круг (D2), мм:",
    "Толщина тарелки (b), мм:",
    "Диаметр отверстий (d), мм:",
    "Кол-во болтов (n), шт:"
};
for (int i = 0; i < 7; ++i) {
    m_combos[i] = new QComboBox();
    formCombo->addRow(labels[i], m_combos[i]);
    if (i == 1)
        connect(m_combos[i], QOverload<int>::of(&QComboBox::currentIndexChanged),
            [this](int) { updateCascade(1); });
}
m_stack->addWidget(pageCombo);
auto* pageManual = new QWidget();
auto* formManual = new QFormLayout(pageManual);

auto makeDSpin = [](double min, double max, int dec = 1) {
    auto* s = new QDoubleSpinBox();
    s->setRange(min, max);
    s->setDecimals(dec);
    s->setButtonSymbols(QAbstractSpinBox::NoButtons);
    s->setSuffix(" мм");
    return s;
};

m_mInnerD = makeDSpin(1, 5000);
m_mOuterD = makeDSpin(1, 5000);
m_mBoltCircD = makeDSpin(1, 5000);
m_mFlangThk = makeDSpin(1, 500);
m_mBoltD = makeDSpin(1, 200);
m_mBoltN = new QSpinBox();

```

```

m_mBoltN->setRange(1, 200);
m_mBoltN->setButtonSymbols(QAbstractSpinBox::NoButtons);
m_mInnerD->setValue(m_current.inner_d);
m_mOuterD->setValue(m_current.outer_d);
m_mBoltCircD->setValue(m_current.bolt_circle_d);
m_mFlangThk->setValue(m_current.flange_thickness);
m_mBoltD->setValue(m_current.bolt_d);
m_mBoltN->setValue(m_current.bolt_count);

formManual->addRow("Внутренний диаметр (D1), мм:", m_mInnerD);
formManual->addRow("Наружный диаметр (D), мм:", m_mOuterD);
formManual->addRow("Болтовой круг (D2), мм:", m_mBoltCircD);
formManual->addRow("Толщина тарелки (b), мм:", m_mFlangThk);
formManual->addRow("Диаметр отверстий (d), мм:", m_mBoltD);
formManual->addRow("Кол-во болтов (n), шт:", m_mBoltN);
m_stack->addWidget(pageManual);

rightLayout->addWidget(m_stack);
auto* formCommon = new QFormLayout();

m_shellThicknessEdit = new QDoubleSpinBox();
m_shellThicknessEdit->setRange(0.5, 100.0);
m_shellThicknessEdit->setDecimals(1);
m_shellThicknessEdit->setSuffix(" мм");
m_shellThicknessEdit->setValue(m_shellThickness);
m_shellThicknessEdit->setButtonSymbols(QAbstractSpinBox::NoButtons);
formCommon->addRow("Толщина обечайки (s), мм:", m_shellThicknessEdit);

m_shellLengthEdit = new QDoubleSpinBox();
m_shellLengthEdit->setRange(0.1, 10000.0);
m_shellLengthEdit->setDecimals(1);
m_shellLengthEdit->setSuffix(" мм");
m_shellLengthEdit->setValue(m_shellLength);
m_shellLengthEdit->setButtonSymbols(QAbstractSpinBox::NoButtons);
formCommon->addRow("Длина патрубка (L), мм:", m_shellLengthEdit);

rightLayout->addLayout(formCommon);
rightLayout->addStretch();

auto* buttons = new QHBoxLayout();
auto* okBtn = new QPushButton("Принять");
auto* cancelBtn = new QPushButton("Отмена");
connect(okBtn, &QPushButton::clicked, this, &FlangeConfigDialog::onAccept);
connect(cancelBtn, &QPushButton::clicked, this, &QDialog::reject);
buttons->addStretch();
buttons->addWidget(cancelBtn);
buttons->addWidget(okBtn);
rightLayout->addLayout(buttons);

mainLayout->addLayout(rightLayout);
okBtn->setFocus();

connect(m_standardCheck, &QCheckBox::toggled,
        this, &FlangeConfigDialog::onStandardToggled);
}

void FlangeConfigDialog::onStandardToggled(bool checked)
{
    m_stack->setCurrentIndex(checked ? 0 : 1);

    if (!checked && m_combos[1]->currentData().isValid()) {

```

```

        double d1 = m_combos[1]->currentData().toDouble();
        for (const auto& f : m_all) {
            if (std::abs(f.inner_d - d1) < 0.001) {
                m_mInnerD->setValue(f.inner_d);
                m_mOuterD->setValue(f.outer_d);
                m_mBoltCircD->setValue(f.bolt_circle_d);
                m_mFlangThk->setValue(f.flange_thickness);
                m_mBoltD->setValue(f.bolt_d);
                m_mBoltN->setValue(f.bolt_count);
                break;
            }
        }
    }
}

void FlangeConfigDialog::initCombos()
{
    for (int i = 0; i < 7; ++i) m_combos[i]->blockSignals(true);

    std::set<double> innerDiameters;
    for (const auto& f : m_all)
        if (f.inner_d >= m_minInnerD)
            innerDiameters.insert(f.inner_d);

    if (innerDiameters.empty() && !m_all.empty()) {
        double best = m_all[0].inner_d;
        for (const auto& f : m_all) if (f.inner_d > best) best = f.inner_d;
        innerDiameters.insert(best);
    }

    for (double d : innerDiameters)
        m_combos[1]->addItem(QString::number(d), d);

    int dIdx = m_combos[1]->findData(m_current.inner_d);
    m_combos[1]->setCurrentIndex(dIdx >= 0 ? dIdx : 0);

    for (int i = 0; i < 7; ++i) m_combos[i]->blockSignals(false);
    updateCascade(1);
}

void FlangeConfigDialog::updateCascade(int level)
{
    if (level != 1) return;

    double selectedD1 = m_combos[1]->currentData().toDouble();

    for (int i = 0; i < 7; ++i) if (i != 1) m_combos[i]->blockSignals(true);
    for (int i = 0; i < 7; ++i) if (i != 1) m_combos[i]->clear();

    std::set<std::string> names;
    std::set<double> d_out, bolt_c, thick, b_diam, b_count;

    for (const auto& f : m_all) {
        if (std::abs(f.inner_d - selectedD1) < 0.001) {
            names.insert(f.name);
            d_out.insert(f.outer_d);
            bolt_c.insert(f.bolt_circle_d);
            thick.insert(f.flange_thickness);
            b_diam.insert(f.bolt_d);
            b_count.insert(static_cast<double>(f.bolt_count));
        }
    }
}

```

```

    }

    auto fillStr = [&](int idx, const std::set<std::string>& vals) {
        for (const auto& v : vals) m_combos[idx]->addItem(QString::fromStdString(v));
    };
    auto fillNum = [&](int idx, const std::set<double>& vals) {
        for (double v : vals) m_combos[idx]->addItem(QString::number(v), v);
    };

    fillStr(0, names);
    fillNum(2, d_out);
    fillNum(3, bolt_c);
    fillNum(4, thick);
    fillNum(5, b_diam);
    fillNum(6, b_count);

    for (int i = 0; i < 7; ++i) m_combos[i]->blockSignals(false);
}

Flange FlangeConfigDialog::flangeFromManual() const
{
    Flange f;
    f.inner_d = m_mInnerD->value();
    f.outer_d = m_mOuterD->value();
    f.bolt_circle_d = m_mBoltCircD->value();
    f.flange_thickness = m_mFlangThk->value();
    f.bolt_d = m_mBoltD->value();
    f.bolt_count = m_mBoltN->value();
    f.name = "(пользоват.)";
    return f;
}

void FlangeConfigDialog::onAccept()
{
    if (m_standardCheck->isChecked()) {
        m_current.name = m_combos[0]->currentText().toStdString();
        m_current.inner_d = m_combos[1]->currentData().toDouble();
        m_current.outer_d = m_combos[2]->currentData().toDouble();
        m_current.bolt_circle_d = m_combos[3]->currentData().toDouble();
        m_current.flange_thickness = m_combos[4]->currentData().toDouble();
        m_current.bolt_d = m_combos[5]->currentData().toDouble();
        m_current.bolt_count = m_combos[6]->currentData().toInt();
    }
    else {
        m_current = flangeFromManual();
    }

    m_shellThickness = m_shellThicknessEdit->value();
    m_shellLength = m_shellLengthEdit->value();

    accept();
}

```

Код файла ShellFlangeConfigDialog.h

```

#pragma once
#include <QDialog>
#include <QComboBox>
#include <QDoubleSpinBox>
#include <QSpinBox>
#include <QCheckBox>

```



```

#include <QLabel>
#include <QStackedWidget>
#include <vector>
#include "SpiralHeatExchanger.h"

class ShellFlangeConfigDialog : public QDialog
{
    Q_OBJECT
public:
    ShellFlangeConfigDialog(const Flange& current,
        const std::vector<Flange>& all,
        double minInnerD,
        QWidget* parent = nullptr);

    Flange getResult() const { return m_current; }

private slots:
    void updateCascade(int level);
    void onAccept();
    void onStandardToggled(bool checked);

private:
    void setupUi();
    void initCombos();
    Flange flangeFromManual() const;

    const std::vector<Flange> m_all;
    Flange m_current;
    double m_minInnerD;

    QLabel* m_imageLabel = nullptr;
    QCheckBox* m_standardCheck = nullptr;
    QStackedWidget* m_stack = nullptr;

    QComboBox* m_combos[7];

    QDoubleSpinBox* m_mInnerD = nullptr;
    QDoubleSpinBox* m_mOuterD = nullptr;
    QDoubleSpinBox* m_mBoltCircD = nullptr;
    QDoubleSpinBox* m_mFlangThk = nullptr;
    QDoubleSpinBox* m_mBoltD = nullptr;
    QSpinBox* m_mBoltN = nullptr;
};

```

Код файла ShellFlangeConfigDialog.cpp

```

#include "ShellFlangeConfigDialog.h"
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QPushButton>
#include <QPixmap>
#include <set>
#include <cmath>

ShellFlangeConfigDialog::ShellFlangeConfigDialog(
    const Flange& current,
    const std::vector<Flange>& all,
    double minInnerD,
    QWidget* parent)
    : QDialog(parent)

```

```

        , m_all(all)
        , m_current(current)
        , m_minInnerD(minInnerD)
    {
        setWindowTitle("Настройка параметров фланца обечайки");
        setupUi();
        initCombos();
    }

void ShellFlangeConfigDialog::setupUi()
{
    auto* mainLayout = new QHBoxLayout(this);
    mainLayout->setSizeConstraint(QLayout::SetFixedSize);

    m_imageLabel = new QLabel();
    m_imageLabel->setFixedSize(300, 300);
    m_imageLabel->setAlignment(Qt::AlignCenter);
    m_imageLabel->setStyleSheet(
        "border: 1px solid #bdc3c7; background: white; border-radius: 4px;");
    QPixmap pix(":/images/flange_drawing.png");
    if (!pix.isNull())
        m_imageLabel->setPixmap(
            pix.scaled(280, 280, Qt::KeepAspectRatio, Qt::SmoothTransformation));
    else
        m_imageLabel->setText("Схема фланца");
    mainLayout->addWidget(m_imageLabel);

    auto* rightLayout = new QVBoxLayout();

    auto* infoLabel = new QLabel(
        QString("Наружный диаметр обечайки: <b>%1 мм</b>")
        .arg(m_minInnerD, 0, 'f', 1));
    infoLabel->setStyleSheet("color: #888; font-size: 9pt; margin-bottom: 6px;");
    rightLayout->addWidget(infoLabel);

    m_standardCheck = new QCheckBox("Стандартные значения");
    m_standardCheck->setChecked(true);
    rightLayout->addWidget(m_standardCheck);

    m_stack = new QStackedWidget();

    auto* pageCombo = new QWidget();
    auto* formCombo = new QFormLayout(pageCombo);
    const QString labels[7] = {
        "Наименование:",
        "Внутренний диаметр (D1), мм:",
        "Наружный диаметр (D), мм:",
        "Болтовой круг (D2), мм:",
        "Толщина тарелки (b), мм:",
        "Диаметр отверстий (d), мм:",
        "Кол-во болтов (n), шт:"
    };
    for (int i = 0; i < 7; ++i) {
        m_combos[i] = new QComboBox();
        formCombo->addRow(labels[i], m_combos[i]);
        if (i == 1)
            connect(m_combos[i], QOverload<int>::of(&QComboBox::currentIndexChanged),
                [this](int) { updateCascade(1); });
    }
    m_stack->addWidget(pageCombo);
}

```

```

auto* pageManual = new QWidget();
auto* formManual = new QFormLayout(pageManual);

auto makeDSpin = [](double min, double max, int dec = 1) {
    auto* s = new QDoubleSpinBox();
    s->setRange(min, max);
    s->setDecimals(dec);
    s->setButtonSymbols(QAbstractSpinBox::NoButtons);
    s->setSuffix(" мм");
    return s;
};

m_mInnerD = makeDSpin(1, 5000);
m_mOuterD = makeDSpin(1, 5000);
m_mBoltCircD = makeDSpin(1, 5000);
m_mFlangThk = makeDSpin(1, 500);
m_mBoltD = makeDSpin(1, 200);
m_mBoltN = new QSpinBox();
m_mBoltN->setRange(1, 200);
m_mBoltN->setButtonSymbols(QAbstractSpinBox::NoButtons);

m_mInnerD->setValue(m_current.inner_d);
m_mOuterD->setValue(m_current.outer_d);
m_mBoltCircD->setValue(m_current.bolt_circle_d);
m_mFlangThk->setValue(m_current.flange_thickness);
m_mBoltD->setValue(m_current.bolt_d);
m_mBoltN->setValue(m_current.bolt_count);

formManual->addRow("Внутренний диаметр (D1), мм:", m_mInnerD);
formManual->addRow("Наружный диаметр (D), мм:", m_mOuterD);
formManual->addRow("Болтовой круг (D2), мм:", m_mBoltCircD);
formManual->addRow("Толщина тарелки (b), мм:", m_mFlangThk);
formManual->addRow("Диаметр отверстий (d), мм:", m_mBoltD);
formManual->addRow("Кол-во болтов (n), шт:", m_mBoltN);
m_stack->addWidget(pageManual);

rightLayout->addWidget(m_stack);
rightLayout->addStretch();

auto* buttons = new QHBoxLayout();
auto* okBtn = new QPushButton("Принять");
auto* cancelBtn = new QPushButton("Отмена");
connect(okBtn, &QPushButton::clicked, this, &ShellFlangeConfigDialog::onAccept);
connect(cancelBtn, &QPushButton::clicked, this, &QDialog::reject);
buttons->addStretch();
buttons->addWidget(cancelBtn);
buttons->addWidget(okBtn);
rightLayout->addLayout(buttons);

mainLayout->addLayout(rightLayout);
okBtn->setFocus();

connect(m_standardCheck, &QCheckBox::toggled,
        this, &ShellFlangeConfigDialog::onStandardToggled);
}

void ShellFlangeConfigDialog::onStandardToggled(bool checked)
{
    m_stack->setCurrentIndex(checked ? 0 : 1);

    if (!checked && m_combos[1]->currentData().isValid()) {

```

```

        double d1 = m_combos[1]->currentData().toDouble();
        for (const auto& f : m_all) {
            if (std::abs(f.inner_d - d1) < 0.001) {
                m_mInnerD->setValue(f.inner_d);
                m_mOuterD->setValue(f.outer_d);
                m_mBoltCircD->setValue(f.bolt_circle_d);
                m_mFlangThk->setValue(f.flange_thickness);
                m_mBoltD->setValue(f.bolt_d);
                m_mBoltN->setValue(f.bolt_count);
                break;
            }
        }
    }
}

void ShellFlangeConfigDialog::initCombos()
{
    for (int i = 0; i < 7; ++i) m_combos[i]->blockSignals(true);

    std::set<double> innerDiameters;
    for (const auto& f : m_all)
        if (f.inner_d >= m_minInnerD)
            innerDiameters.insert(f.inner_d);

    if (innerDiameters.empty() && !m_all.empty()) {
        double best = m_all[0].inner_d;
        for (const auto& f : m_all) if (f.inner_d > best) best = f.inner_d;
        innerDiameters.insert(best);
    }

    for (double d : innerDiameters)
        m_combos[1]->addItem(QString::number(d), d);

    int dIdx = m_combos[1]->findData(m_current.inner_d);
    m_combos[1]->setCurrentIndex(dIdx >= 0 ? dIdx : 0);

    for (int i = 0; i < 7; ++i) m_combos[i]->blockSignals(false);
    updateCascade(1);
}

void ShellFlangeConfigDialog::updateCascade(int level)
{
    if (level != 1) return;

    double selectedD1 = m_combos[1]->currentData().toDouble();

    for (int i = 0; i < 7; ++i) if (i != 1) m_combos[i]->blockSignals(true);
    for (int i = 0; i < 7; ++i) if (i != 1) m_combos[i]->clear();

    std::set<std::string> names;
    std::set<double> d_out, bolt_c, thick, b_diam, b_count;

    for (const auto& f : m_all) {
        if (std::abs(f.inner_d - selectedD1) < 0.001) {
            names.insert(f.name);
            d_out.insert(f.outer_d);
            bolt_c.insert(f.bolt_circle_d);
            thick.insert(f.flange_thickness);
            b_diam.insert(f.bolt_d);
            b_count.insert(static_cast<double>(f.bolt_count));
        }
    }
}

```

```

    }

    auto fillStr = [&](int idx, const std::set<std::string>& vals) {
        for (const auto& v : vals) m_combos[idx]->addItem(QString::fromStdString(v));
    };
    auto fillNum = [&](int idx, const std::set<double>& vals) {
        for (double v : vals) m_combos[idx]->addItem(QString::number(v), v);
    };

    fillStr(0, names);
    fillNum(2, d_out);
    fillNum(3, bolt_c);
    fillNum(4, thick);
    fillNum(5, b_diam);
    fillNum(6, b_count);

    for (int i = 0; i < 7; ++i) m_combos[i]->blockSignals(false);
}

Flange ShellFlangeConfigDialog::flangeFromManual() const
{
    Flange f;
    f.inner_d = m_mInnerD->value();
    f.outer_d = m_mOuterD->value();
    f.bolt_circle_d = m_mBoltCircD->value();
    f.flange_thickness = m_mFlangThk->value();
    f.bolt_d = m_mBoltD->value();
    f.bolt_count = m_mBoltN->value();
    f.name = "(пользоват.)";
    return f;
}

void ShellFlangeConfigDialog::onAccept()
{
    if (m_standardCheck->isChecked()) {
        m_current.name = m_combos[0]->currentText().toStdString();
        m_current.inner_d = m_combos[1]->currentData().toDouble();
        m_current.outer_d = m_combos[2]->currentData().toDouble();
        m_current.bolt_circle_d = m_combos[3]->currentData().toDouble();
        m_current.flange_thickness = m_combos[4]->currentData().toDouble();
        m_current.bolt_d = m_combos[5]->currentData().toDouble();
        m_current.bolt_count = m_combos[6]->currentData().toInt();
    }
    else {
        m_current = flangeFromManual();
    }
    accept();
}

```